

21BA030 - Ingénierie des exigences



Objectifs

- situer la discipline de l'ingénierie des exigences dans le contexte de l'ingénierie logicielle et de la business analyse
- maîtriser les mécanismes de gestion des exigences
 - de la compréhension du domaine à la validation du système
- maîtriser les techniques de représentation des exigences
 - collecter, analyser, spécifier, gérer et valider les exigences des utilisateurs en termes d'automatisation des processus d'affaires
- interagir avec les acteurs
 - dialoguer avec les utilisateurs
 - délivrer une information utile et complète aux développeurs

Les compétences clés visées par le cours



Analyser et
modéliser



Collaborer



Concevoir et
développer des
solutions

Plan du cours (1)

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Plan du cours (2)

- 2 chapitres introductifs pour comprendre le métier du business analyst son "core business" : l'ingénierie des exigences
- 1 Introduction à la business analyse
 - 1.1 Définition et enjeux
 - 1.2 Le métier de business analyst
 - 1.3 Le processus de la business analyse
- 2 Ingénierie des exigences
 - 2.1 Contexte de l'ingénierie logicielle
 - 2.2 Définition et enjeux
 - 2.3 Taxonomie des exigences
 - 2.4 Enjeux et défis de l'ingénierie des exigences

Plan du cours (3)

- 1 chapitre détaillé sur les différentes tâches à réaliser pour produire des exigences de qualité
- 3 Cycle de gestion des exigences
 - 3.1 Compréhension du domaine
 - 3.2 Elicitation des exigences
 - 3.3 Analyse des exigences
 - 3.4 Représentation des exigences
 - 3.5 Organisation des exigences
 - 3.6 Gestion des exigences
 - 3.7 Estimation des coûts

Plan du cours (4)

- 2 chapitres consacrés à la représentation graphique des exigences avec UML

- 4 Diagramme de cas d'utilisation

- 4.1 Cas d'utilisation
- 4.2 Acteur
- 4.3 Structuration des cas d'utilisation
- 4.4 Spécification des cas d'utilisation
- 4.5 Démarche de construction

- 5 Diagramme d'activité

- 5.1 Action
- 5.2 Arc entre des actions
- 5.3 Nœud
- 5.4 Partition
- 5.5 Données et persistance
- 5.6 Événement, poids de flux et décomposition
- 5.7 Exception et zone interruptible
- 5.8 Zone d'expansion

Plan du cours (5)

- 2 chapitres qui revisitent certains aspects clés de l'ingénierie des exigences avec BPMN

- 6 Réingénierie des processus avec BPM

- 6.1 Processus d'affaires (business process ou processus métier)
- 6.2 Cycle BPM
- 6.3 Exercice

- 7 Modélisation des processus avec BPMN

- 7.1 Activité
- 7.2 Séquencement
- 7.3 Données
- 7.4 Ressources
- 7.5 Compléments sur les événements

Plan du cours (6)

- on termine avec une introduction aux tests et une courte discussion sur l'utilisation de l'IA dans l'ingénierie des exigences
- 8 Tests
 - 8.1 Définition et enjeux
 - 8.2 Phasage des tests
 - 8.3 Types de tests
 - 8.4 Techniques de test
- 9 IA pour l'ingénierie des exigences
 - 9.1 Opportunités de l'IA
 - 9.2 Risques de l'IA
 - 9.3 Démarche de mise en œuvre de l'IA

Modalités

- support : notes de cours
 - syllabus reprenant les explications détaillées des concepts, des exercices dont la plupart sont résolus
- pédagogie hybride : 5 parcours pédagogiques avec :
 - une séance d'exercice en présentiel
 - des vidéos à travailler en autonomie avant la séance
 - des exercices et lectures à réaliser après la séance
- inscription sur <http://ichec.be/moodle/>
 - clé : IE17
 - notes de cours, vidéos, transparents, quizz, ressources complémentaires

Evaluation

- référence de la matière d'examen : **notes de cours** + autres contenus du site de cours
 - les lectures complémentaires (articles scientifiques, chapitres de livre, etc.) font partie de la matière d'examen
- examen écrit à livre fermé
 - questions de restitution et de compréhension
 - questions d'exercices

Exemples de questions (non exhaustif)

- expliquez le fonctionnement du Business Analysis Core Concept Model (BABOK) à partir du schéma suivant



- définissez la notion d'exigences. En quoi une exigence diffère-t-elle d'un besoin/but?
- expliquez précisément et complètement les concepts suivants termes suivants en 2 à 3 lignes : test de non-régression, défaut d'un logiciel, system analyst
- présentez une méthode pour établir des priorités entre des exigences
- comparez ingénierie des exigences et ingénierie logicielle
- présentez en détails la notion de modèle. Quel est l'utilité d'un modèle ?
- discutez en détails la traçabilité des exigences. Donnez un exemple complet
- présentez et comparez les différents langages pour représenter les exigences

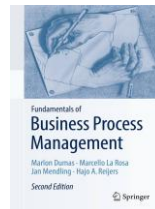
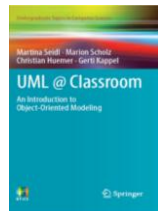
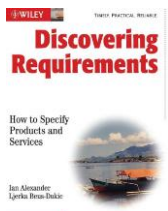
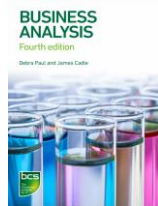
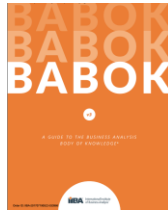
Exemples de questions (non exhaustif)

- présentez les 4 activités d'ingénierie des exigences
- présentez des techniques d'élicitation de groupe proposées par Alexander (Alexander I.F. and Beus-Dukic L., Discovering Requirements: How to Specify Products and Services, Wiley, 2009)
- présenter le cycle BPMN
- comparez les différentes techniques de test
- exercices de formulation d'exigences à partir d'un énoncé en langage naturel
- exercice de construction de diagramme /de cas d'utilisation/d'activité/BPMN
- exercice de vérification de la qualité d'un ensemble d'exigences
- choix de solutions et de priorités pour un cas donné
- etc.

Implication de l'étudiant

- avant les séances d'exercices : assimilation de la matière à travers les vidéos
 - **indispensable pour une participation efficace aux exercices**
- pendant les séances d'exercices : participation active
 - aux résolutions d'exercices
 - aux jeux de rôles
 - **pas de GSM ou de réseaux sociaux**
- entre les séances d'exercices : matière des vidéos et slides correspondants, préparation d'exercices, lectures, etc.
 - 4 à 6 h par semaine
- importance de rester bien à jour dans le cours dès le départ !
 - examen après 6 semaines de cours (seconde session : aout)

Références



Thierry Van den Berghe

Références

- International Institute of Business Analysis, A Guide to the Business Analysis Body of Knowledge® (BABOK® Guide), version 3.0, 2015
- Paul, D., & Cadle, J. (2020). Business Analysis (éd. 4ième). BCS Learning and Development.
- Milani Fr., Digital Business Analysis, Springer, 2019
- van Lamsweerde, A. *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Wiley Publishing, 1st edition, 2009
- Alexander I.F. and Beus-Dukic L., *Discovering Requirements: How to Specify Products and Services*, Wiley, 2009
- Seidl, M., Scholz, M., Huemer, C., Kappel, G., *UML @ Classroom*, Springer, 2015
- Dumas M., La Rosa M., Mendling Hajo J., Reijers A., *Fundamentals of Business Process Management*, Springer, 2013
- ISTQB, Syllabus Niveau Fondation, 2018

Thierry Van den Berghe



21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Introduction à la business analyse

- sources pour cette section :



- **définition de la business analyse**
- concept clés de la business analyse
- rôle du business analyst
- domaine de connaissance du business analyst
- processus de la business analyse

BABOK

- Edité par IIBA (International Institute of Business Analysis)

- fondé en 2003
- certifications
 - « Certified Business Analysis Professional » (CBAP)
 - « Certification of Competency on Business Analysis » (CCBA)
- chapitre belge : brussels.iiba.org



- BABOK (A Guide to the Business Analysis Body of Knowledge)

- précise les compétences et les connaissances que devraient maîtriser les business analysts
- exhaustif mais pas avec tous les détails
- très utile comme cadre de référence ... mais pas comme support de cours
- à destination des professionnels en premier lieu



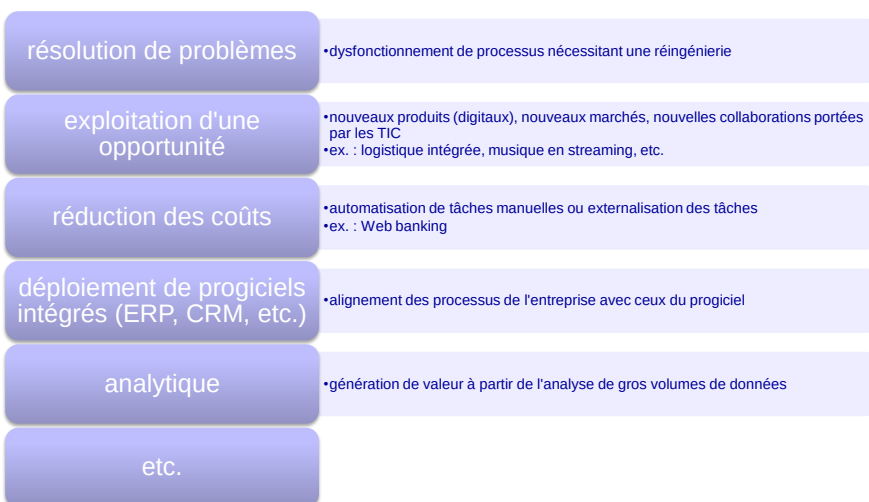
Importance des TIC et de la business analyse

- Pourquoi les TIC sont-elles omniprésentes dans tous les aspects des affaires ?
 - simplement parce qu'elles améliorent la performance des processus d'affaires
- Mais les TIC ont un potentiel d'apport de valeur ... uniquement si elles sont alignées avec les besoins des utilisateurs
- La business analyse participe à la mise en place de solutions impliquant l'IT et qui sont alignées sur les besoins
- La profession de business analyst trouve son origine dans :
 - la prolifération des systèmes d'information dans l'entreprise
 - la nécessité d'articuler business et IT

Business analyse : c'est quoi ?

- Définition : pratique d'activation du changement
 - → en définissant les besoins business
 - → en recommandant des solutions qui apportent de la valeur aux parties prenantes
- Les solutions proposées exploitent souvent les TIC
 - ...mais pas seulement
 - ex. : révision des processus sans changement du système d'information
 - ex. : changement dans les acteurs qui prennent en charge les processus

Projets types de la business analyse

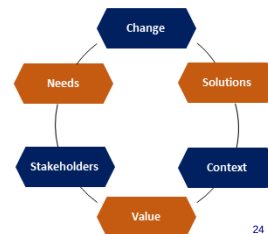


Introduction à la business analyse

- définition de la business analyse
- **concept clés de la business analyse**
- rôle du business analyst
- domaines de connaissance du business analyst
- processus de la business analyse

Business Analysis Core Concept Model (BABOK)

- Le BABOK introduit les concepts clés qui permettent de capter la nature de la business analyse
- **Changement** : action de transformation en réponse à un besoin
- **Besoin** : problème ou opportunité à traiter
- **Solution** : manière spécifique de répondre à un besoin dans un contexte donné
- **Partie prenante** : individus ou groupes impliqués dans le changement, les besoins ou la solution
- **Valeur** : apport, importance ou utilité de quelque chose pour une partie prenante dans une contexte donné
 - tangible ou intangible
- **Contexte** : circonstances qui influencent/sont influencées/exploquent le changement
 - régulation, culture d'entreprise, etc.



Exercice

- Proposez des relations entre les éléments du Core Concept Model

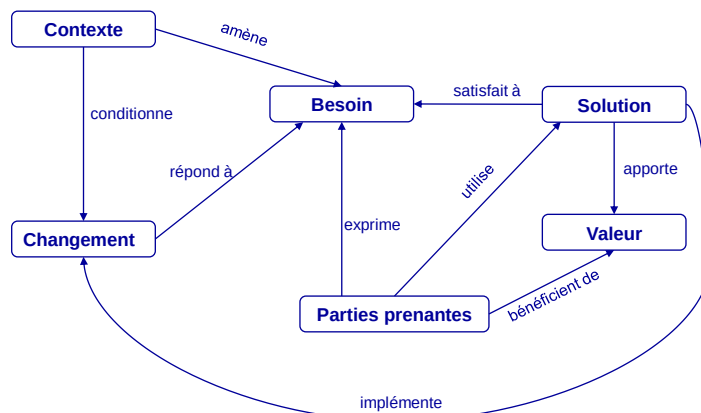


Thierry Van den Berghe

25

Exercice : exemple de solution

- Proposez des relations entre les éléments du Core Concept Model



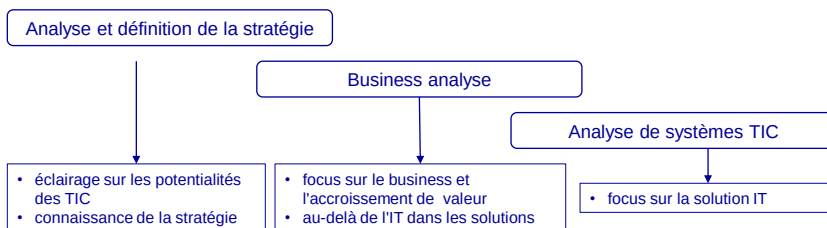
Thierry Van den Berghe

26

Business Analysis Core Concept Model : exemple

- Imaginons un producteur de barres chocolatées qui souhaite développer son activité et conquérir de nouveaux détaillants. Pour cela, il doit gérer la traçabilité de ses produits à l'aide d'un système de gestion de la traçabilité
 - **Contexte** : concurrence importante, exigences de certification de la qualité de la chaîne de production et de livraison
 - **Changement** : mise en place d'une gestion de la traçabilité
 - **Besoin** : nécessité de gérer la traçabilité
 - **Solution** : module logiciel permettant de tracer chaque produit dans son cycle de production et de livraison
 - **Valeur** : amélioration de la qualité grâce à la traçabilité, conformité aux exigences de certification, gain de la confiance des détaillants
 - **Parties prenantes** : agences de certification, détaillants, département qualité

Spectre de la business analyse



Mission de la business analyse (1)

- **investigation de la situation et analyse du problème**
 - recherche des causes racine à des problèmes informels et mal définis
 - identification des besoins émergents
 - niveau macro
- **étude de faisabilité et business case**
 - évaluation des solutions possibles pour répondre aux besoins du business
 - analyse de l'opportunité d'un changement
- **amélioration des processus d'affaires**
 - as is → to be, gap analysis
 - niveau micro

Mission de la business analyse (2)

- **définition des exigences**
 - élicitation, analyse et définition des exigences (ingénierie des exigences)
- **tests d'acceptance business**
 - soutien les acteurs business dans le test des nouveaux processus et des systèmes IT pour confirmer leur acceptation par le business
- **gestion du changement**
 - aide à la mise en place du changement
 - transition en douceur vers les nouveaux processus et systèmes

Introduction à la business analyse

- définition de la business analyse
- concept clés de la business analyse
- **rôle du business analyst**
- domaines de connaissance du business analyst
- processus de la business analyse



21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse**
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

1 B - rôle du business analyst



Introduction à la business analyse

- définition de la business analyse
- concept clés de la business analyse
- **rôle du business analyst**
- domaines de connaissance du business analyst
- processus de la business analyse

BABOK

- "Business analysts are responsible for discovering, synthesizing, and analyzing **information** from a variety of sources within an enterprise, including tools, processes, documentation, and stakeholders. The business analyst is responsible for **eliciting the actual needs of stakeholders**—which frequently involves investigating and clarifying their expressed desires—in order to determine underlying issues and causes.
- Business analysts play a role in **aligning** the designed and delivered **solutions** with the **needs** of stakeholders.
- In many cases, the business analyst will also work to **facilitate communication** between organizational units. In particular, business analysts often play a central role in **aligning** the needs of **business** units with the capabilities delivered by information technology, and may serve as a "**translator**" between those groups."



Rôle du business analyst (1)

- En amont du développement de la solution :
 - apporte la vision et la stratégie
 - conduit l'ingénierie des exigences
 - collabore au choix des solutions
- Pendant le développement de la solution :
 - accompagne le chef de projet et contribue à la priorisation des exigences
 - maintient la cohérence de la solution apportée par rapport aux exigences
- Après le développement :
 - valide le système livré par rapport aux exigences

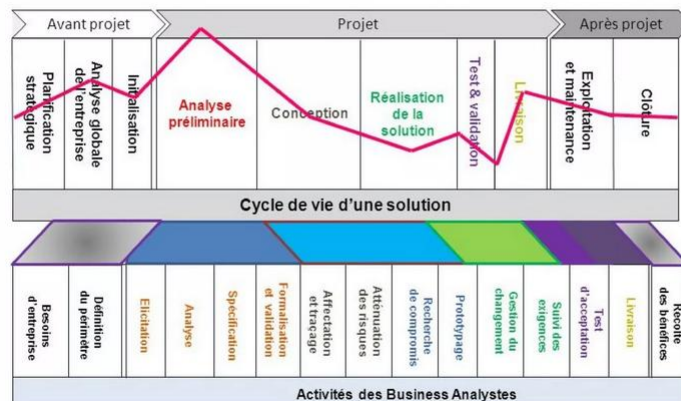
Thierry Van den Bergh

35

Rôle du business analyst (2)

IBA France
Chapter

La Business Analysis, dans un cycle « en V » de vie d'une solution



BA ou Professionnalisation de l'AMOA, Mercredi 29 Avril 2015, IIBA France – Antenne AURA - ADIRA

Thi

Déclinaisons du métier de BA



Enterprise business analyst

réflexion au niveau entreprise : problèmes/opportunités ? solutions business à développer ?



Digital business analyst

réflexion sur les opportunités des TIC pour l'amélioration des processus, du service et de l'expérience client



Technical business analyst

analyse des exigences détaillées dans le cadre d'un projet



Proxy business analyst

rôle de product owner dans le contexte Agile
gestion du product backlog, voix des utilisateurs, priorités dans les exigences, etc.

Source : BA

27

Les compétences visées par le master en BA



Analyser et modéliser



Collaborer



Concevoir et développer des solutions



Gérer un projet



Participer à la stratégie de gouvernance



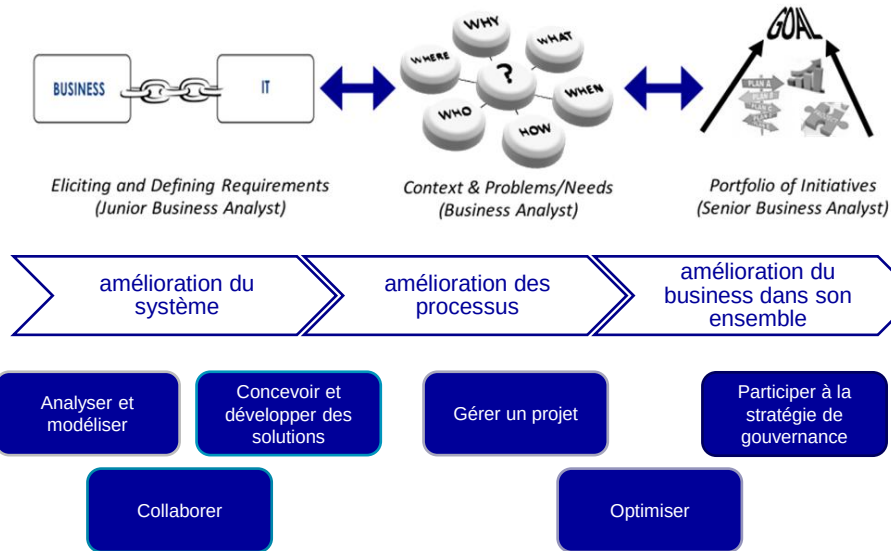
Optimiser

hard skills

soft skills

38

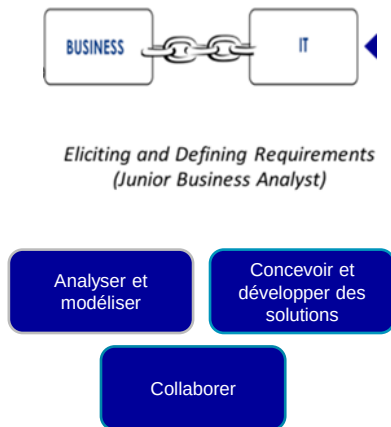
Gradation dans la business analyse



Thierry Van den Berghe

39

Gradation dans la business analyse (1)



- **Stade 1 : comprendre les besoins et les communiquer à l'IT**
 - implique le **business analyst junior**
 - élicitation et formalisation des besoins, transmission vers l'IT, évaluation de la solution
 - tester la solution
 - → amélioration du système en place

Thierry Van den Berghe

40

Gradation dans la business analyse (2)



*Context & Problems/Needs
(Business Analyst)*

Gérer un projet

Participer à la
stratégie de
gouvernance

Optimiser

- Stade 2 : comprendre les motivations du changement (business drivers)
 - implique le **business analyst expérimenté**
 - cadrage, motivation et opportunités du projet
 - gap analysis, réingénierie des processus
 - → amélioration des processus en place

Thierry Van den Berghe

41

Gradation dans la business analyse (3)



*Portfolio of Initiatives
(Senior Business Analyst)*

Gérer un projet

Participer à la
stratégie de
gouvernance

Optimiser

- Stade 3 : comprendre les objectifs business de haut niveau à atteindre et décider des composants à adapter
 - implique le **business analyst senior**, qui intègre le management stratégique
 - évaluation des alternatives et choix en termes de projets
 - gestion d'un portefeuille de projets, établissement de priorités, alignement stratégique
 - ex. : choix des projets à lancer, choix de changer le SI ou non, etc.
 - → amélioration du business dans son ensemble

Thierry Van den Berghe

42

Le business analyst comme acteur de la transformation digitale

- L'application massive des TIC est progressive :

conversion analogue → digital

- conversion de produits/processus en digital
- ex. : Tax-on Web

transformation digitale

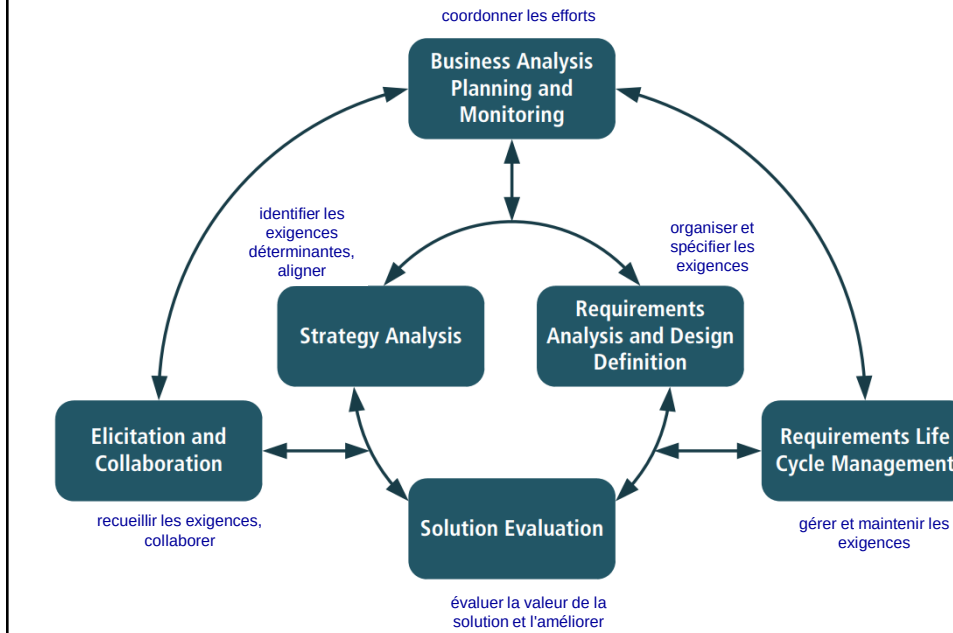
- changement dans les processus/produits grâce aux TIC
- nouveaux business models (streaming, réseaux sociaux, etc.)
- locale (projet) ou globale (corporate, stratégique)

- La BA accompagne l'entreprise dans la transformation digitale
 - identification des opportunités des TIC pour optimiser l'existant, au-delà de la résolution immédiate de problèmes
 - → vision à moyen et long terme, proactivité

Introduction à la business analyse

- définition de la business analyse
- concept clés de la business analyse
- rôle du business analyst
- **domaines de connaissance du business analyst**
- processus de la business analyse

Domaines de connaissances (BABOK) (1)



Domaines de connaissances (BABOK) (2)

- **Business Analysis Planning and Monitoring** : comment coordonner les efforts des business analysts et des autres parties prenantes (définition d'un cadre méthodologique, planification des tâches, etc.)
- **Elicitation and Collaboration** : comment recueillir et valider les exigences auprès des parties prenantes ? Comment gérer la collaboration avec les parties prenantes ?
- **Requirements Life Cycle Management** : comment gérer et maintenir les exigences de leur découverte à leur achèvement ?
- **Strategy Analysis** : comment identifier les exigences déterminantes d'un point de vue stratégique ou tactique, et aligner le projet sur les objectifs stratégiques de l'organisation ?
- **Requirements Analysis and Design Definition** : comment organiser et spécifier les exigences ? Comment dégager une solution répondant aux exigences en apportant le plus de valeur possible ?
- **Solution evaluation** : comment évaluer la valeur de la solution et comment l'améliorer

Compétences du business analyst

Compétences relationnelles

- Soft skills (communication, facilitation, travail en équipe)
- Esprit analytique et résolution de problèmes
- Professionnalisme
- Flexibilité

Connaissances business et TIC

- Connaissance du domaine d'application (structure, architectures business)
- Calcul d'opportunité (ROI) - Business case development
- Technologies digitales

Techniques de business analyse

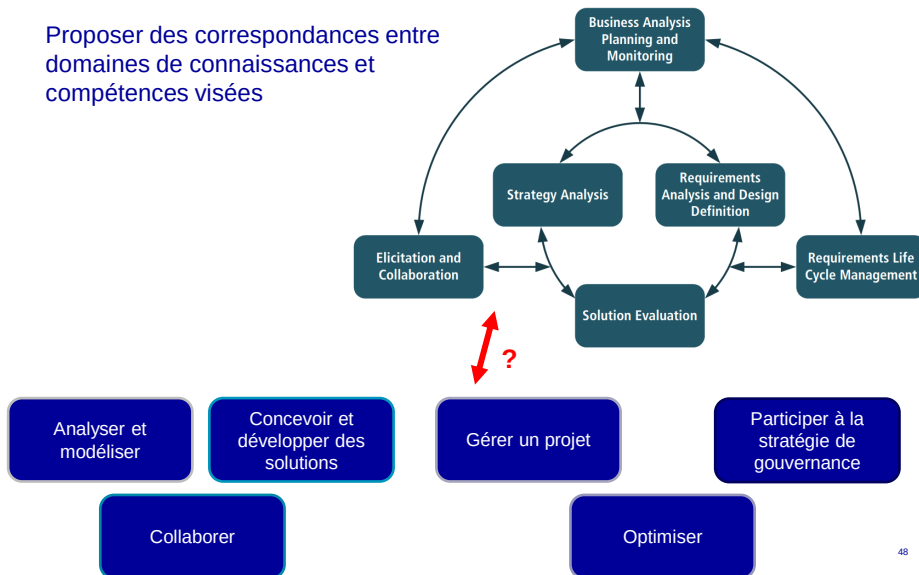
- Analyse de la stratégie (génération d'idées, gap analysis, gestion de portefeuilles de projets)
- Ingénierie des exigences (investigation, modélisation des processus et des données)
- Gestion de projet
- Ingénierie des exigences

Thierry Van den Bergh

47

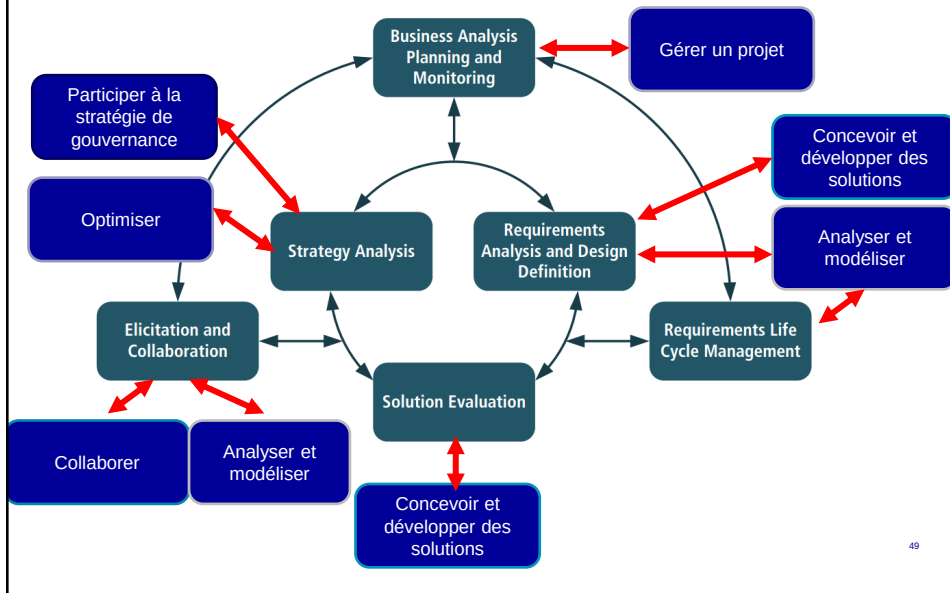
Exercice IEX02

Proposer des correspondances entre domaines de connaissances et compétences visées



48

Exercice IEX02 : solution



49

Introduction à la business analyse

- définition de la business analyse
- concept clés de la business analyse
- rôle du business analyst
- **domaines de connaissance du business analyst**
 - zoom sur la résolution de problèmes
- processus de la business analyse

Modèle de résolution de problèmes

1 Compréhension du problème

- 1.1 appréhender et décrire le problème
- 1.2 documenter avec des données quantitatives et qualitatives
- 1.3 déterminer les causes racines

2 Définition de la solution

- 2.1 générer des idées
- 2.2 analyser les idées et proposer des solutions
- 2.3 évaluer les solutions

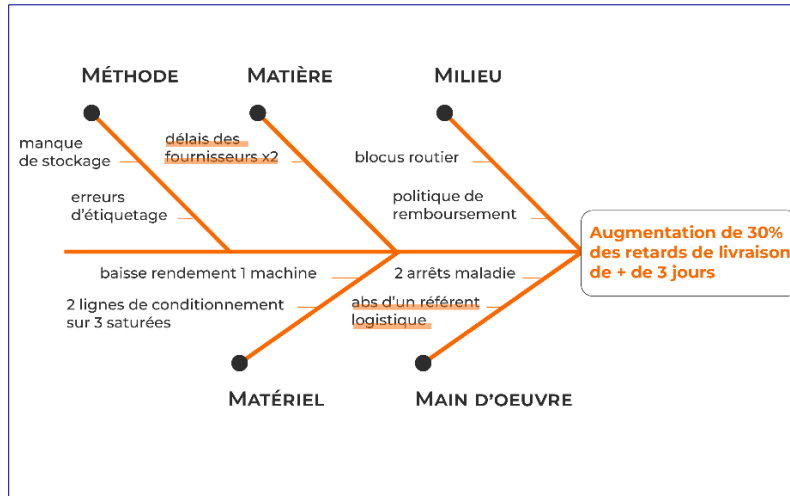
3 Mise en œuvre

- 3.1 détailler les solutions choisies et leur mise en œuvre

Exemples d'outils : 3.1 - décrire le problème



Exemples d'outils : 1.3 déterminer les causes racines



Exemples d'outils : 2.1 générer des idées



Exemples d'outils : 2.3 évaluer les solutions

Échelle de positionnement : 0 = nul, 1 = pas bon, 2 = moyen, 3 = bien, 4 = très bien.

CRITÈRES	Pondé - ration	SOLUTIONS							
		TGV 1re classe	Total	AVION	Total	VOITURE	Total	AUTO STOP	Total
Rapidité	4	3	12	3	12	1	4	0	0
Confort	3	3	9	2	6	0	0	1	3
Sécurité	2	3	6	3	6	1	2	0	0
Coût	1	2	2	0	0	0	0	3	3
TOTAL			29		24		6		6

Exercice IEX03 (1)

- **IEX03** : Appliquez une approche de résolution de problème dans le contexte d'une administration régionale qui gère des dossiers de demandes de primes à la rénovation durable déposées par les citoyens.
- On vous demande, sur base d'interviews :
 - Un mind mapping pour décrire le problème de satisfaction des citoyens et sa situation
 - Des données sur la performance (choisissez des indicateurs et trouvez des données relatives aux administrations sur le Web)
 - Un diagramme d'Ishikawa pour identifier les causes du problème
 - La tenue d'une séance de brainstorming
 - Une liste de solutions prioritaires

- **Interview d'un citoyen (adapté de Copilot 20/12/2024)**
- **Interviewer** : Bonjour, merci de prendre le temps de nous parler aujourd'hui. Pouvez-vous nous expliquer votre expérience avec l'administration communale concernant votre demande de prime à la rénovation durable ?
- **Citoyen mécontent** : Bonjour. Oui, bien sûr. Franchement, c'est un vrai parcours du combattant. J'ai déposé ma demande il y a plus de six mois et je n'ai toujours pas de réponse définitive. Les délais sont interminables et chaque fois que je contacte l'administration, on me dit que mon dossier est en cours de traitement.
- **Interviewer** : Cela doit être très frustrant. Pouvez-vous nous en dire plus sur les difficultés que vous avez rencontrées ?
- **Citoyen mécontent** : Absolument. D'abord, les formulaires sont extrêmement compliqués et toujours en format papier : quasi rien n'est en ligne. Il y a tellement de documents à fournir et chaque fois que je pense avoir tout complété, on me demande encore autre chose. Ensuite, il y a le manque de communication. J'ai envoyé plusieurs emails et passé des appels, mais les réponses sont souvent vagues ou inexistantes, et de la part de personnes différentes qui ne semblent pas connaître l'historique du dossier. On a l'impression que personne ne sait vraiment ce qui se passe avec mon dossier.
- **Interviewer** : Avez-vous reçu des explications sur les raisons de ces délais et de cette complexité ?
- **Citoyen mécontent** : Pas vraiment. On m'a dit que c'était dû à un grand nombre de demandes et à des ressources limitées, mais cela ne justifie pas un tel manque d'efficacité. J'ai l'impression que l'administration n'est pas bien organisée et que les citoyens en paient le prix.
- **Interviewer** : Que souhaiteriez-vous voir changer dans ce processus ?
- **Citoyen mécontent** : J'aimerais que l'administration soit plus transparente et réactive. Il faudrait simplifier les formulaires, améliorer la communication avec les citoyens et mettre tout ça en ligne. Un suivi plus rigoureux des dossiers serait aussi nécessaire pour éviter ces longs délais. En somme, il faut rendre ce processus plus accessible et moins stressant pour les gens qui veulent simplement améliorer leur habitat de manière durable.
- **Interviewer** : Merci beaucoup pour votre témoignage. Nous espérons que votre situation s'améliorera bientôt.
- **Citoyen mécontent** : Merci à vous. J'espère aussi que les choses changeront pour le mieux.

Thierry Van den Berghe

57

- **Interview d'un agent (adapté de Copilot 20/12/2024)**
- **Interviewer** : Bonjour, merci de nous accorder cet entretien. Pouvez-vous nous parler de votre expérience en tant qu'agent chargé des dossiers de demandes de primes à la rénovation durable ?
- **Agent administratif** : Bonjour. Eh bien, pour être honnête, c'est assez difficile en ce moment. Nous sommes débordés par le nombre de demandes et la procédure est extrêmement complexe. Je dois avouer que je ne me sens pas suffisamment formé pour gérer tout cela efficacement.
- **Interviewer** : Cela doit être très stressant. Quelles sont les principales difficultés que vous rencontrez ?
- **Agent administratif** : Oui, c'est vraiment stressant. La principale difficulté est la complexité des procédures. Il y a tellement de règles et de documents à vérifier que cela devient vite écrasant. De plus, nous n'avons pas reçu une formation adéquate pour bien comprendre toutes les étapes du processus. Cela entraîne des erreurs et des retards, ce qui est frustrant pour les citoyens et pour nous. La plupart des traitements sont manuels car l'informatique ne prend pas le temps de nous développer un programme.
- Sans parler des changements fréquents de la procédure, dès que les politiques s'en mêlent...
- **Interviewer** : Comment cette situation affecte-t-elle votre motivation au travail ?
- **Agent administratif** : Honnêtement, c'est démotivant. On a l'impression de ne jamais pouvoir rattraper le retard et de toujours être sous pression. Cela affecte aussi notre moral, car nous voulons aider les citoyens, mais nous sommes limités par le système et le manque de ressources. Plusieurs de mes collègues sont absents, et me refilent des dossiers que je ne connais pas bien.
- **Interviewer** : Que pensez-vous qu'il faudrait changer pour améliorer la situation ?
- **Agent administratif** : Il faudrait simplifier les procédures et offrir une meilleure formation aux agents. Un renforcement des effectifs serait également nécessaire pour gérer le volume de demandes. Enfin, une meilleure communication interne et avec les citoyens pourrait aider à réduire les frustrations et à rendre le processus plus fluide.
- **Interviewer** : Merci beaucoup pour votre témoignage. Nous espérons que des améliorations seront apportées rapidement.
- **Agent administratif** : Merci à vous. J'espère aussi que les choses changeront pour le mieux, pour nous et pour les citoyens.

Thierry Van den Berghe

58

Green IT

- Un constat alarmant
 - le digital consomme 10% de l'électricité mondiale
 - un Français possède 15 équipements connectés en moyenne
- → Green IT : ensemble de pratiques pour réduire l'empreinte environnementale du numérique
- Rôle du business analyst
 - choix de la solution la moins impactante
 - sobriété numérique
 - pousser au développement éco-responsable

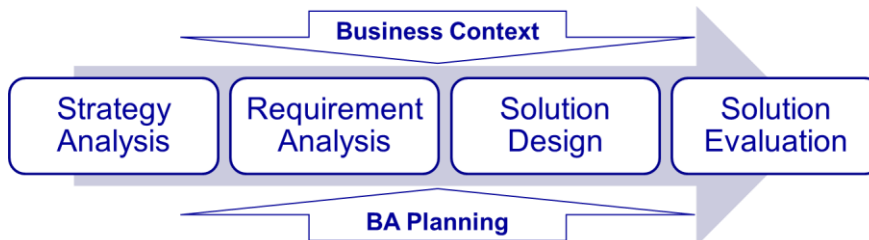
65

Introduction à la business analyse

- définition de la business analyse
- concept clés de la business analyse
- rôle du business analyst
- domaines de connaissance du business analyst
- **processus de la business analyse**

Processus de la business analyse

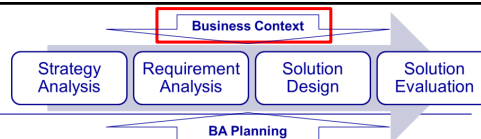
- Pas de règle ni de processus universels
 - le choix du processus dépend du problème à résoudre
 - les étapes du processus ne sont pas systématiquement exécutées en séquence, mais avec des allers-retours, des itérations et du parallélisme
- Processus typique (basé sur BABOK) :



Thierry Van den Berghe

67

Business context

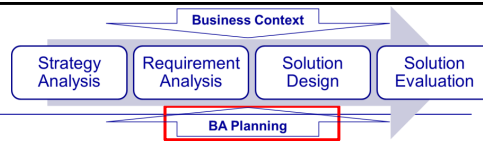


- Recouvre les circonstances qui influencent/sont influencées/exploquent le changement
 - les éléments de contexte impactent toutes les étapes de la business analyse
 - ex. : business model, valeurs, stratégie, régulation, structure organisationnelle, etc.
 - ex. du private banking : business model basé sur la qualité de la relation client → qualité > coût → processus flexibles et personnalisables
 - ex. du retail banking : business model basé sur de gros volumes (transactions, clients, services) → coût > qualité → processus standards
- → voir la section "compréhension du domaine et élicitation"

Thierry Van den Berghe

68

BA Planning



- Recouvre les activités :
 - de cadrage (problème, objectifs, valeur attendue)
 - de choix d'outils et de méthodes
 - d'identification et de planification des tâches et livrables
- Activités de fond de la gestion de projet
- Livrables clés : Business Analysis Plan, Stakeholder Engagement
- → voir la section sur "cycle de développement"

Zoom : contenu d'un Business Analysis Plan (1)

- Cadrage
 - description du besoin (problème/opportunité)
 - ex. : un parc d'attractions a vu sa fréquentation augmenter fortement ces 5 dernières années, et les solutions en place pour gérer les visiteurs et l'espace disponible est dénoncé par plusieurs plaintes. Une meilleure solution doit être mise en place pour augmenter la satisfaction client et poursuivre la croissance
 - objectifs du travail de business analyse
 - identifier les options de solution, fournir assez de données pour la prise de décision sans détailler les options
 - ex. de solutions pour améliorer l'expérience des visiteurs : agrandissement du parc, contrôle de nombre d'entrées par période et sur réservation, dynamic pricing
 - valeur attendue, effets désirés
 - meilleure satisfaction client, accroissement du nombre de visiteurs

Zoom : contenu d'un Business Analysis Plan (2)

- **Choix d'outils et de méthodes**
 - typiquement : approche prédictive (classique) vs approche adaptative (Agile)
 - recommandation en fonction du type de projet (taille, complexité, etc.), de la volatilité des exigences, etc.
- **Identification et de planification des tâches et livrables**
 - quelles sont les tâches et les livrables pour déployer la solution ? Quelle planification de haut niveau ? Estimation des ressources et des coûts
 - ex. de tâches :
 - définition du "as-is"
 - définition du "to-be"
 - analyse de risques
 - choix de la solution
 - design de la solution choisie
 - développement de la solution et évaluation

Thierry Van den Berghe

71

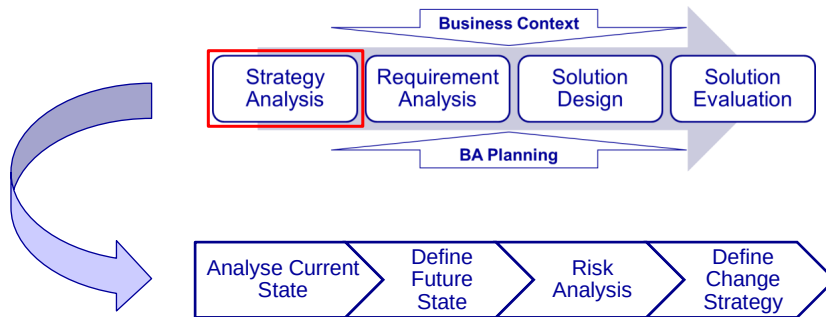
Zoom : contenu d'un Business Analysis Plan (3)

- **Choix de méthode**

Table 6.1 Aspects to factor in when considering approach

Predictive approach	Factor	Adaptive approach
A larger and more complex project	Project size and complexity	A smaller and less complex project
Customers have difficulties being extensively involved during the project duration	Customer/ stakeholder availability	Customers are willing and available to frequently be involved during project duration
Unknown or several complex integrations required	Integration level	None or few simple integrations required
Budget/time schedule is fixed and difficult to change/adapt	Flexibility and tolerance to changes	There is flexibility (budget/time)
Solution requires full feature set to be delivered	Time to Market	Solution can be initially launched with limited feature

Strategy Analysis



Thierry Van den Berghe

73

Strategy Analysis : Analyse Current State

- Description de l'existant (situation "as is")
 - livrable clé : Current State Description (métier, capacités, culture, ressources, infrastructure, etc. en place dans l'organisation)
- Identification des besoins business (problèmes/opportunités)
 - ex. de problèmes/opportunités : plaintes client, perte de revenu, opportunité de nouveaux marchés / produits / technologies
 - livrables clés : Business Requirement (problème/opportunité sur base de l'état courant décrit à un **niveau macro**)
- Nombreuses techniques (business canevas, PEST - **P**olitique **E**conomique **S**ocial **T**echnologique , etc.)
- → sections "compréhension du domaine et élicitation" et "cycle BPMN"

Thierry Van den Berghe

74

Strategy Analysis : Define Future State

- Description de l'état futur souhaité et motivant le changement (situation "to be")
- Quels buts et objectifs faut-il atteindre ?
 - BABOK : the purpose of Define Future State is to determine the set of necessary conditions to meet the business need
- Délivrables clés :
 - Business Objectives : orientation souhaitée pour atteindre l'état futur
 - Future State Description : description des composants à adapter (capacités, ressources, infrastructure, etc.) pour atteindre l'état futur
 - Potential Value : valeur espérée grâce à l'implémentation de l'état futur

Strategy Analysis : Risk analysis

- Compréhension des risques liés à la réalisation de l'état futur
- Identification des stratégies pour réduire
 - la probabilité de survenance ou
 - l'impact du risque
- Délivrables clés : analyse de risques (identification et caractérisation des risques, contremesures, etc.)

Strategy Analysis : Define Change Strategy

- Description des solutions alternatives pour activer le changement
 - ex. de solutions alternatives : développement sur mesure, progiciel, adaptation des processus, etc.
 - Gap Analysis :
 - comment passer de l'état "as is" à l'état "to be"
 - quelles sont les solutions possibles ?
 - quelle est la portée de chaque solution alternative
 - livrable clé : Solution Scope
- Evaluation des solutions alternatives et sélection de la meilleure
 - exemples de critères décisionnels : impact de la solution (bénéfices), respects des contraintes (délais, coûts, etc.), ampleur du changement, faisabilité, etc.
 - Livrable clé : Change Strategy (approche pour piloter le changement)
- Voir la section "BPMN"

Zoom : exemple as is → to be (1)

- Analyse as is :
 - contexte :
 - une administration régionale gère des dossiers de demandes de primes à la rénovation durable adressées par les citoyens
 - à faire : description des processus **en place** (soumission d'une demande, examen d'une demande, réponse à une demande)
 - mesure de la performance du système en place
 - ex. de KPI : délai moyen du traitement d'une demande, satisfaction des citoyens
 - problème
 - délai trop long dans la prise de décision du montant éligible et l'octroi des primes
 - mécontentement des parties prenantes (citoyen, agent)
 - → l'administration souhaite un changement pour résoudre ce problème

Zoom : exemple as is → to be (2)

- Analyse to be :
 - fixation d'objectifs afin de résoudre le problème identifié en amont
 - ex. : réduction du temps moyen du traitement à 3 mois
 - ex. : augmentation du niveau de satisfaction de 20 %
 - révision des processus et des ressources

Zoom : exemple as is → to be (3)

- Gap analysis :
 - identification des facteurs limitants
 - ex. : manque d'automatisation des processus
 - ex. : nombre d'intervenants trop nombreux
 - ex. : motivation et formation du personnel irrégulières
 - identification des solutions possibles (bridging the gap)
 - automatisation de tâches encore manuelles
 - optimisation des processus en place (suppression des tâches peu porteuses de valeur, réduction du nombre d'intervenants, etc.)
 - à faire : description des processus **futurs** (soumission d'une demande, examen d'une demande, réponse à une demande)
 - plan de formation et de motivation du personnel

Zoom : exemple as is → to be (4)

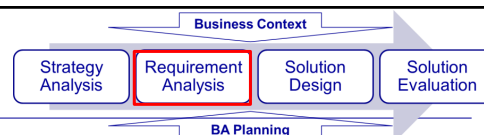
- Exemple d'évaluation des solutions possibles

Solution	Impact sur le délai	Impact sur la satisfaction	Coût	Niveau de risque
automatisation	***	**	*	**
optimisation des processus	**	**	**	*
formation et motivation	*	***	***	**

Thierry Van den Berghe

81

Requirement analysis

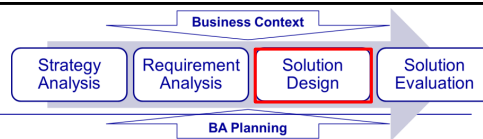


- Constitution d'un ensemble d'exigences complet, cohérent et porteur de valeur pour la solution retenue
 - les étapes précédentes (as is, to be, gap analysis) se cantonnent à une description des exigences à un niveau macro
 - ici : exigences détaillées
- Cf. ingénierie des exigences (couvert par la suite du cours)
- Délivrables clés : liste d'exigences, SRS (System Requirement Specification)

Thierry Van den Berghe

82

Solution Design



- Définition complète et détaillée de la solution proposée
 - compréhension des problèmes et des besoins : analyse **approfondie** de la situation actuelle sur base de la phase requirement analysis et des points d'amélioration nécessaires
 - présentation de la solution proposée : comment la solution va résoudre les problèmes identifiés, améliorer l'efficacité opérationnelle et répondre aux besoins spécifiques
 - portée du projet : fonctionnalités et les processus couverts par la solution
 - tests et validation : test et étapes de validation et d'assurance qualité
- Délivrable clé : Solution Design Document (SDD)
 - les livrables des différentes étapes décrivent le même projet, mais à degrés d'avancement et de précision différents → contenus communs

Exemple de SDD : contenu

1. Introduction
 - Description de la mission du projet
 - Parties prenantes
2. Exigences
 - Exigences métier
 - État actuel (as is)
 - Problèmes (as is)
3. Solution
 - Description détaillée
 - Portée
 - Jalons temporels
4. Tests à réaliser

Exemple de SDD : 1. Introduction

- Description de la mission du projet
 - dans la cadre de la gestion des PAE des étudiants, une procédure a été mise en place pour permettre aux étudiants de modifier leur PAE
 - l'objectif du projet est d'optimiser cette procédure pour l'accélérer et réduire les erreurs.
 - application concernée : application de gestion administrative des étudiants

Exemple de SDD : 1. Introduction

- Parties prenantes
 - consultées pour le projet :

Nom	Prénom	Département	Fonction
Durant	Nathalie	Secrétariat des étudiants	Secrétaire
Mahieu	Martine	Secrétariat des étudiants	Responsable programme
Pornel	Yannick	IT	Développeur
Van den zande	Alban	IT	Business Analyste

- utilisateurs de la solution :
 - étudiants
 - responsables des programmes
 - secrétaires

Exemple de SDD : 2. Exigences

- Exigences métier

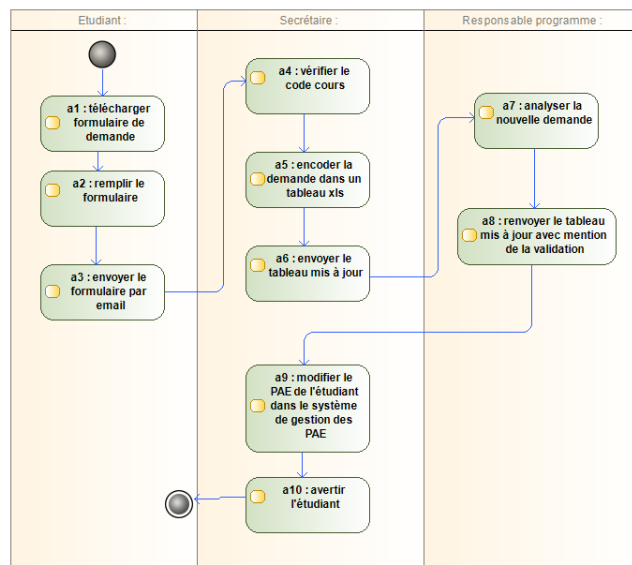
- en tant qu'étudiant, je veux introduire une demande de modification de mon PAE pour adapter mon programme à mon profil
- en tant que responsable de programme, je veux examiner une demande de modification de PAE pour statuer sur celle-ci
- en tant que responsable de programme, je veux accepter ou rejeter une demande de modification de PAE pour répondre à la demande de l'étudiant
- etc.

Thierry Van den Berghe

87

Exemple de SDD : 2. Exigences

- État actuel (as is)



Thierry Van den Berghe

Exemple de SDD : 2 . Exigences

- Problèmes (as is)
 - le processus en place implique des échanges de mails et de documents
 - formulaire de demande de changement de PAE
 - tableau récapitulatif des demandes réalisé en Excel
 - les problèmes sont les suivants :
 - encodage multiple de l'information avec des risques d'erreur
 - un intermédiaire est nécessaire (secrétaire) mais n'apporte pas de plus-value à l'information
 - les délais de traitement sont longs

Exemple de SDD : 3 . Solution

- Description
 - la solution préconisée consiste à revoir le processus en place et à l'automatiser
 - le processus sera intégré au flux d'information au sein du système central de gestion administrative afin de centraliser toute l'information utile au même endroit et sans redondance
 - cette centralisation permet d'automatiser de nombreuses tâches chronophages, comme la vérification des codes cours de la demande, et facilite l'accès et la circulation d'information
 - la solution permet de supprimer un intermédiaire (secrétaire)

Exemple de SDD : 3 . Solution

- Description (suite)

– la solution fonctionnera de la sorte :

1. encodage de la demande de changement via un formulaire électronique contenant les champs suivants moyennant identification :

Nom du champ	Type	Choix pré-enregistré	Contrainte
Matricule	Liste	Liste des étudiants	Obligatoire par identification
Code cours	Liste	Liste des UE accessibles à l'étudiant	Obligatoire
Nature du changement	Liste	Ajout, suppression	Obligatoire

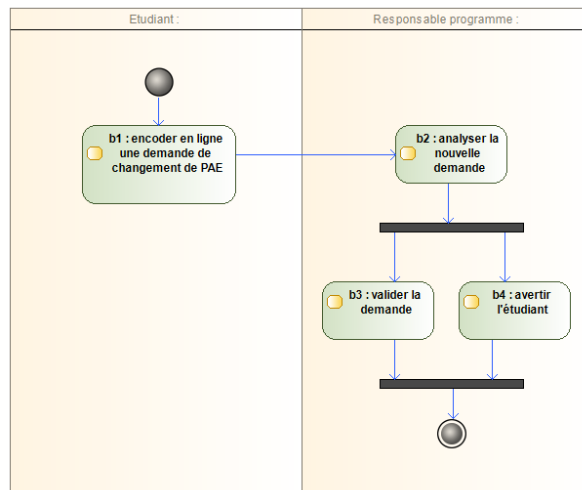
2. analyse de la demande
3. validation ou refus de la demande Signature électronique via la plateforme
4. communication de la décision

Thierry Van den Berghe

91

Exemple de SDD : 3 . Solution

- Description (suite)



Thierry Van den Berghe

92

Exemple de SDD : 3 . Solution

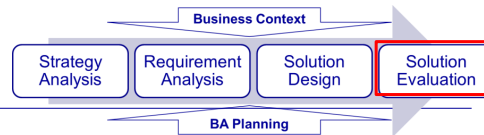
- Portée
 - le processus s'applique aux étudiants full time et au cours de l'institution
 - hors périmètre : étudiant en échange, cours suivis en dehors de l'institution (Erasmus, etc.)
- Jalons

Tâche	Responsable
T + 1 mois : Analyse business	IT -Alban
T + 3 mois : Développement	IT - Yannick
T + 4 mois : Tests	Secrétariat des étudiants - Nathalie

Exemple de SDD : 4 . Tests à réaliser

Tâche	Réponse système attendue
Encodage du formulaire de demande de changement du PAE	Création d'une demande de changement de PAE sauvegardée dans un statut « ouvert »
Analyse de la nouvelle demande	Affichage des données étudiant, de son PAE complet et des UE à modifier
Validation d'une demande	Marquage la demande comme acceptée ou refusée. Modification du PAE en cas d'acceptation. Sauvegarde de la demande dans un statut « traitée »
Avertissement de l'étudiant	Génération d'un mail de décision à destination de l'étudiant

Solution Evaluation

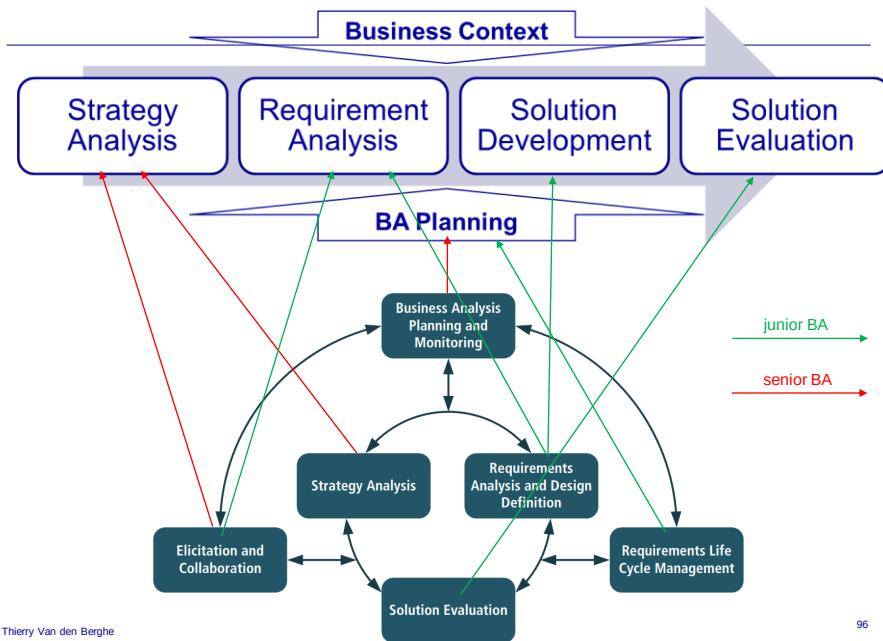


- Vérification que la solution atteint les objectifs fixés et répond aux exigences des parties prenantes
- Activités
 - planification des tests
 - création des tests
 - réalisation de tests
 - etc.
- Voir section "Tests"
- Délivrable clé : Evaluation Report

Thierry Van den Berghe

95

Processus et compétences de la business analyse



Thierry Van den Berghe

96

Plan

- IA et business analyse



- Opportunités
- Risques
- Démarche

IA dans l'entreprise

- Développement et impact fulgurants
 - technologie de rupture
- IA = automate efficace pour des tâches répétitives
 - ex. : IA générative = perroquet statistique
 - pas de compréhension, ni d'intelligence, ni de créativité... pour le moment
- IA = assistant digital
 - l'IA demande un apprentissage, une supervision et une validation des résultats
 - l'acteur humain est in fine le seul responsable
 - IA ≠ remplaçant

IA dans l'ingénierie des exigences

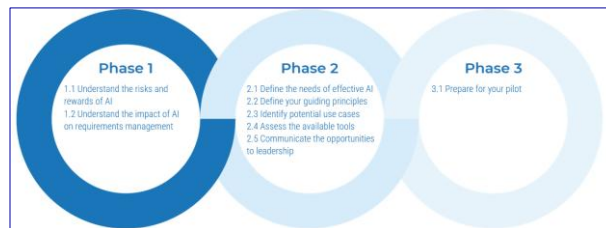
- Nombreuses applications
 - retranscription d'interview
 - traduction
 - reporting
 - découverte d'exigences
 - présentation d'une entreprise
 - identification de risques

Risques de l'IA

- Risques de fiabilité
 - ex. : biais induits par des données d'apprentissage
- Risques de sécurité
 - nouvelles attaques ciblant les système IA
 - ! données confidentielles
- Risques de confiance
 - opacité du "raisonnement" de l'IA

Mise en œuvre de l'IA dans la business analyse

- Prérequis :
 - support du management, vision et gouvernance
 - business analysts formés
 - processus et outils optimisés
 - support opérationnel
- La mise en œuvre est progressive :



Thierry Van den Bergh

101



ICHEC
BRUSSELS MANAGEMENT SCHOOL

21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences**
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Plan

- **définition de l'ingénierie logicielle**
- **4 P de l'ingénierie logicielle**
- **cycles de développement**
- **croisement de disciplines**

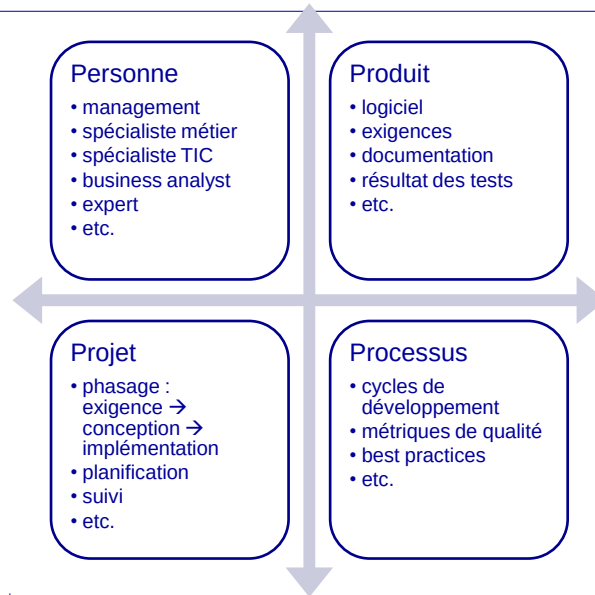
Thierry Van den Berghe

Ingénierie logicielle : définition

- ingénierie : développement de solutions à des problèmes pratiques sur base de connaissances scientifiques
 - ex. : construire un pont résistant à partir des lois de la physique
- ingénierie logicielle : "application d'approches systématiques, rigoureuses, quantifiables pour le développement, la mise en œuvre et la maintenance d'un logiciel"
 - IEEE - Institute of Electrical and Electronics Engineers 
- software engineering is human intensive (J. Mylopoulos)

Thierry Van den Berghe

Les 4 P de l'ingénierie logicielle (1)



Thierry Van den Berghe

105

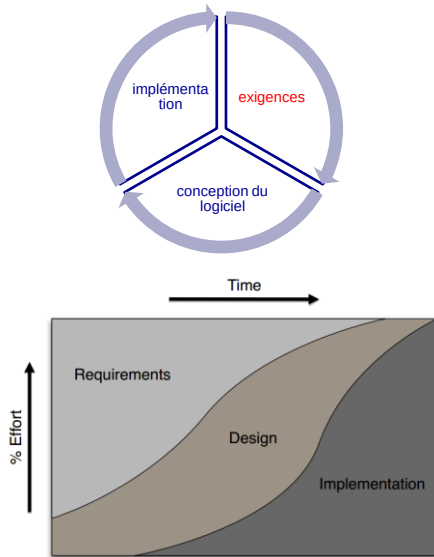
Les 4 P de l'ingénierie logicielle (2)

- **Personne** : des développeurs produisent du logiciel pour des utilisateurs → communication cruciale
- **Produit** : la finalité est de délivrer un produit logiciel qui répond aux exigences métier, mais pas seulement
- **Projet** : la production de logiciel est complexe et doit être organisée comme tout autre projet d'entreprise
 - nombreuses tâches interdépendantes
 - nombreux intervenants
 - contraintes de délai, de coût et de qualité
- **Processus** : un projet se structure selon un processus défini
 - ex. : Waterfall, Unified Process, Scrum

Thierry Van den Berghe

106

Projet : phasage générique des activités



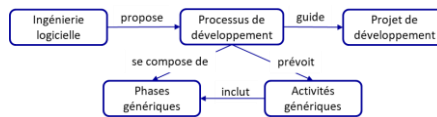
Thierry Van den Berghe

107

- enchaînement des activités selon des cycles de développements
 - ex. : cascade, itérations, etc.
- en pratique : morcellement plus fin des activités
 - voir plus loin : Zoom sur les cycles de développement

Zoom sur les cycles de développement

- le développement d'un logiciel est progressif
 - la complexité impose une décomposition
 - un projet est souvent organisé en phases
 - une phase est le lieu d'activités

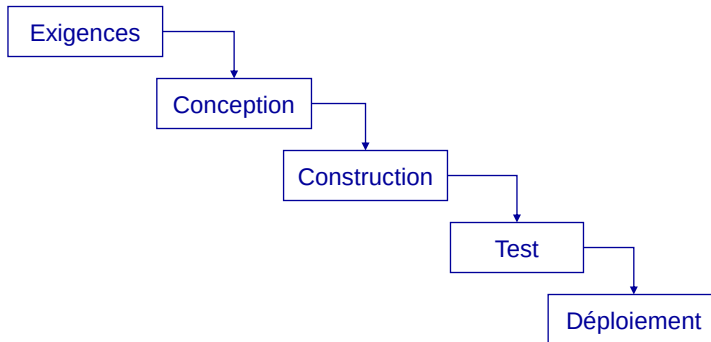


- le cycle de développement précise les interrelations entre les phases :
 - dans quel ordre et à quelle fréquence exécuter les phases?
 - quels critères pour passer d'une phase à l'autre?
 - quel livrables pour chaque phase?

Thierry Van den Berghe

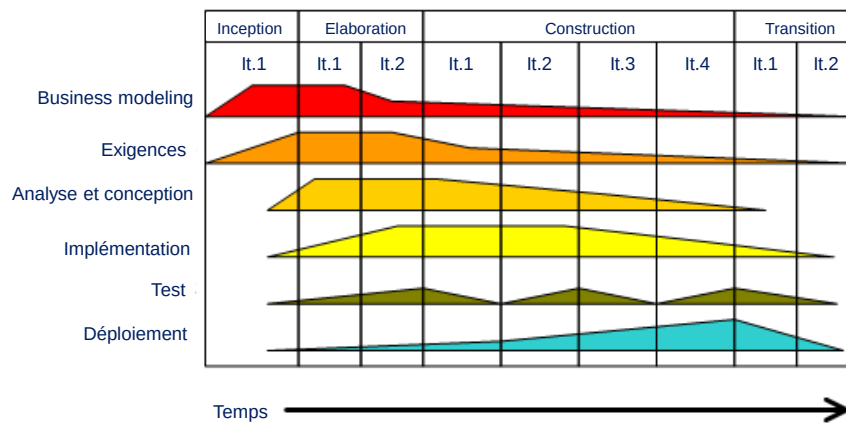
Processus en cascade

- ! exigences
- complexité
- gestion du changement
- lourdeur des tests



109

Unified Process (UP)



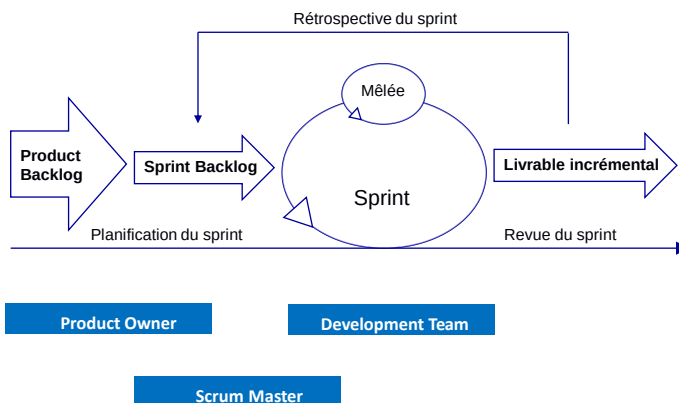
110

Agile

- Agile Manifesto
 - des interactions entre les personnes plutôt que des processus et outils
 - des collaborations plutôt que des contrats
 - du logiciel plutôt que de la documentation
 - de l'adaptation plutôt qu'un plan figé

111

Exemple : Scrum



112

Plan

- **définition**
- taxonomie des exigences
- enjeux et défis des exigences
- cycle de l'IE

Thierry Van den Berghe

Exigence utilisateur : définitions (1)

- énoncé prescriptif sur les **fonctionnalités**, les **qualités** et les **contraintes** de développement d'un logiciel (van Lamsweerde)
- représentation **utilisable** d'un besoin. Une exigence est centrée sur le type de **valeur** qui peut être apportée (BABOK)
- "(1) A condition or capability needed **by a user** to solve a problem or achieve an objective. (2) A condition or capability that must be met/possessed **by a system** or system component to satisfy a contract, standard, specification or other formally imposed documents. (3) A **documented representation** of a condition or a capability as in (1) or (2) above" (IEEE)

Thierry Van den Berghe

114

Exigence utilisateur : définition (2)

- → énoncé
- ...d'une capacité que doit posséder le système à construire pour répondre à un objectif donné
- ...ou d'une contrainte que le système doit respecter
 - doit être documenté et utilisable pour guider le développement
 - concerne le logiciel mais correspond à un objectif utilisateur
 - sous-entend une implémentation possible avec les TIC disponibles
- ne pas confondre avec :
 - besoin : attente non (encore) formulée d'un acteur, et donc inutilisable
 - but : besoin ou exigence de haut niveau, éventuellement vague
- dénominations : requirement, spécification fonctionnelle

Exemples d'exigences

- un lecteur doit pouvoir réserver un ouvrage qu'il souhaite emprunter
- le bibliothécaire doit pouvoir définir des codes de classification des ouvrages
- le système de réservation d'ouvrage doit être accessible à un utilisateur non formé aux TIC
- seul le bibliothécaire peut prolonger la durée d'un prêt
- un lecteur ne peut emprunter plus de 3 exemplaires d'un ouvrage au même moment

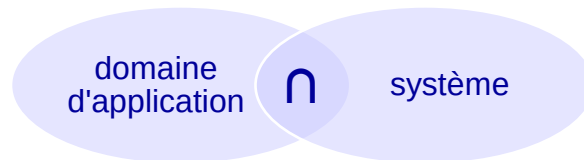
← exigence fonctionnelle

← exigence non fonctionnelle

← contrainte

Ingénierie des exigences : définition (1)

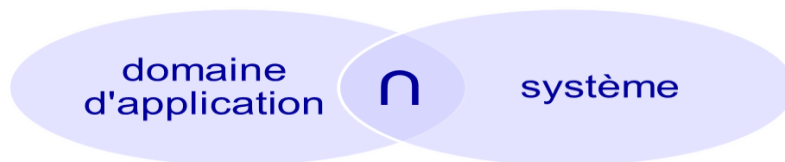
- processus de découverte, de compréhension, d'analyse et de formulation du problème adressé par le système à construire
 - système = système reposant sur du logiciel installé sur un ordinateur
 - dénominations : analyse fonctionnelle, business analysis, etc.
- le problème adressé doit être placé dans un contexte plus large : le domaine d'application
 - partie de monde qui interagit avec le système
 - intersection {système, domaine d'application} = interface du système
 - interface = partie visible par l'utilisateur



Thierry Van den Berghe

117

Ingénierie des exigences : définition (2)

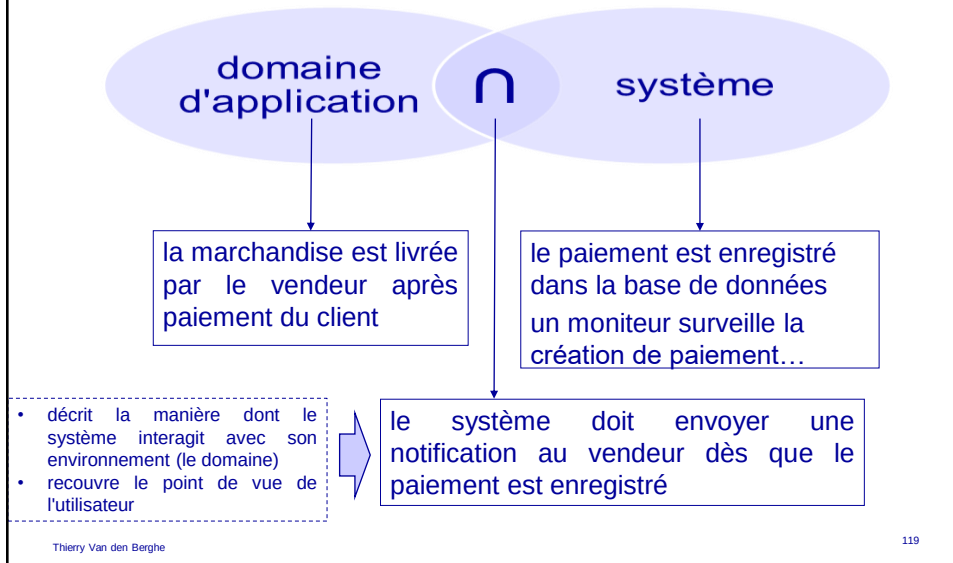


<u>domaine + $\cap$</u>	<u>système</u>
• ingénierie des exigences : traite des effets du système sur le domaine et de la connaissance du domaine • centrée sur le "quoi"	• conception et implémentation : traite du développement logiciel • centrées sur le "comment"

Thierry Van den Berghe

118

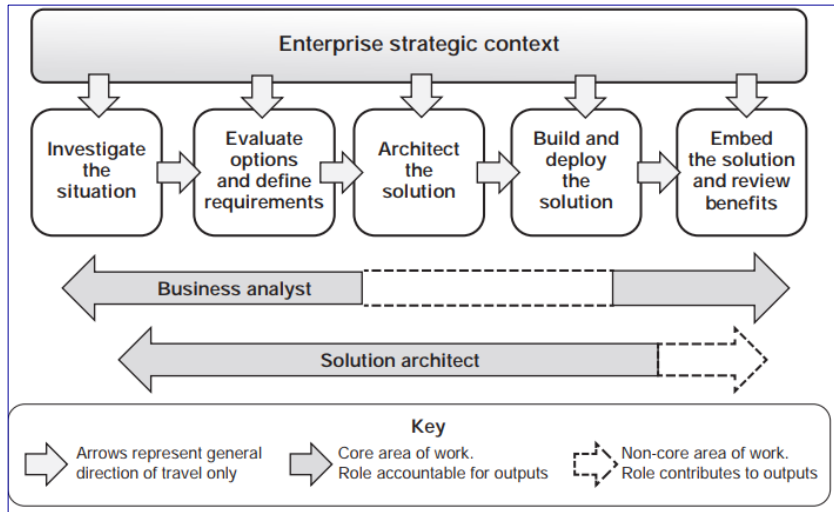
Exemple



Ingénierie logicielle vs. ingénierie des exigences

- 2 approches complémentaires et intégrées pour développer un système logiciel en ligne avec les besoins des utilisateurs
 - les exigences guident la conception et l'implémentation
- ingénierie des exigences : building the right system
 - focus sur les processus et leur automatisation
 - focus sur la définition des capacités pertinentes du système
 - couvre surtout les phases en amont des cycles de développement
 - activité clé du Business Analyst
- ingénierie logicielle : building the system right
 - focus sur la qualité du système
 - ex. : fiabilité, sécurité, clarté du code, etc.
 - couvre surtout les phases en aval des cycles de développement
 - activité clé des System Analysts, System Developers, etc.

Business analyst et solution architect (Paul & Cadle, 2020)



Thierry Van den Berghe

121

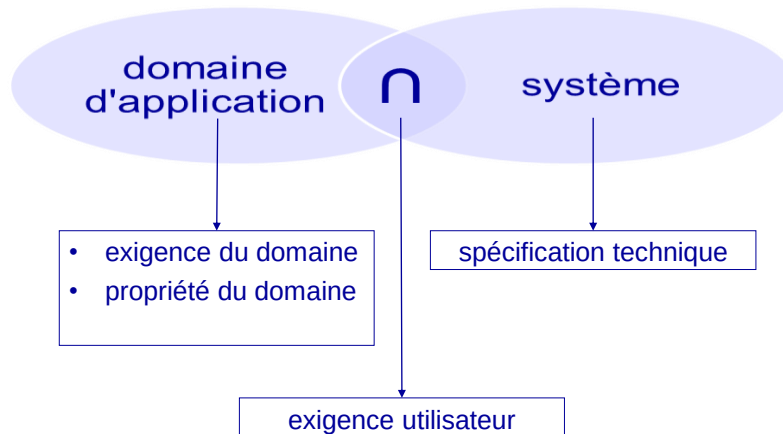
Plan

- définition
- **taxonomie des exigences**
- enjeux et défis des exigences
- cycle de l'IE

Thierry Van den Berghe

122

Variété des exigences (1)



Thierry Van den Berghe

123

Variété des exigences (2)

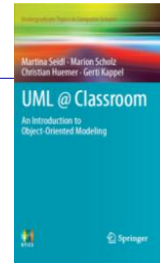
- **exigence du domaine**
 - énoncé du domaine à propos de ses objectifs et de ses besoins
 - prescriptif
 - ex. : un professeur doit pouvoir emprunter 3 ouvrages maximum
 - peut être générique et partiellement défini (vision, objectifs stratégiques)
 - ex. : la bibliothèque a pour la mission de fournir le support scientifique aux activités de recherche et d'enseignement des professeurs
- **propriété du domaine**
 - énoncé du domaine indépendant du comportement du système et à vérifier en toutes circonstances
 - descriptif
 - ex. : le même ouvrage ne peut pas être prêté à deux lecteurs différents en même temps

Thierry Van den Berghe

124

Prescriptif versus descriptif

- "*Descriptive models* show a part of the reality to make a specific aspect easier to understand. For example, a city map describes a city in such a way as to help a non-local person to find routes within the city.
- In contrast, *prescriptive models* are used to offer a construction manual for the system to be developed. "



Variété des exigences (3)

- exigence utilisateur
 - rappel : énoncé d'une capacité que doit posséder le système à construire pour répondre à un objectif donné ou d'une contrainte
 - énoncé à propos du comportement externe du système, tel que perçu par l'utilisateur
 - prescriptif (capacité) ou descriptif (contrainte)
 - ex. :
 - le bibliothécaire doit pouvoir enregistrer maximum 3 prêts simultanés pour un même professeur
 - le système doit afficher un message d'erreur si on tente d'encoder un nouveau prêt pour un professeur pour lequel 3 prêts sont ouverts
 - correspondance aux exigences du domaine

Spécifications techniques

- énoncé concernant le fonctionnement interne du système
 - prescriptif
 - ex . :
 - la table "Emprunts en cours" doit stocker les prêts d'ouvrages auprès des lecteurs
 - la table "Emprunts en cours" doit contenir les colonnes "matricule lecteur", "cote ouvrage", "date de début de prêt", "date de fin de prêt"
 - le programme de création d'emprunts doit vérifier qu'il n'y pas plus de 3 emprunts simultanés pour le même matricule de lecteur
 - relève de la solution ("comment" plutôt que le "quoi")
 - correspondance aux exigences utilisateur
 - dénomination : exigences système, exigences de la solution

Liens entre les exigences

- voir plus loin : traçabilité

• un professeur doit pouvoir emprunter 3 ouvrages maximum



- le bibliothécaire doit pouvoir enregistrer maximum 3 prêts simultanés pour un même professeur
- le système doit afficher un message d'erreur si on tente d'encoder un nouveau prêt pour un professeur pour lequel 3 prêts sont ouverts



- la table "Emprunts en cours" doit stocker les prêts d'ouvrages auprès des lecteurs
- la table "Emprunts en cours" doit contenir les colonnes "matricule lecteur", "cote ouvrage", "date de début de prêt", "date de fin de prêt"
- le programme de création de d'emprunts doit vérifier qu'il n'y pas plus de 3 emprunts simultanés pour le même matricule de lecteur

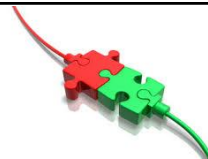
exigences du domaine



exigences utilisateur



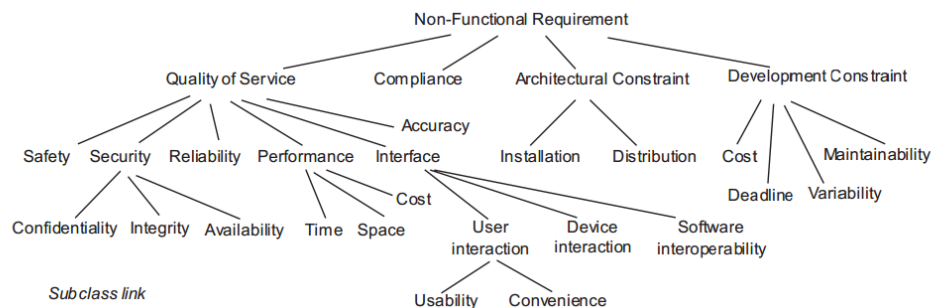
spécifications techniques



Catégories d'exigences

- exigence fonctionnelle
 - description d'une fonctionnalité que le futur système doit fournir
- exigence non fonctionnelle
 - définit une contrainte sur la manière dont le système réalise les fonctionnalités
 - caractérise une propriété ou qualité désirée du système
 - ex. : contrainte de performance ou de sécurité

Exigences non fonctionnelles (source : van Lamsweerde)



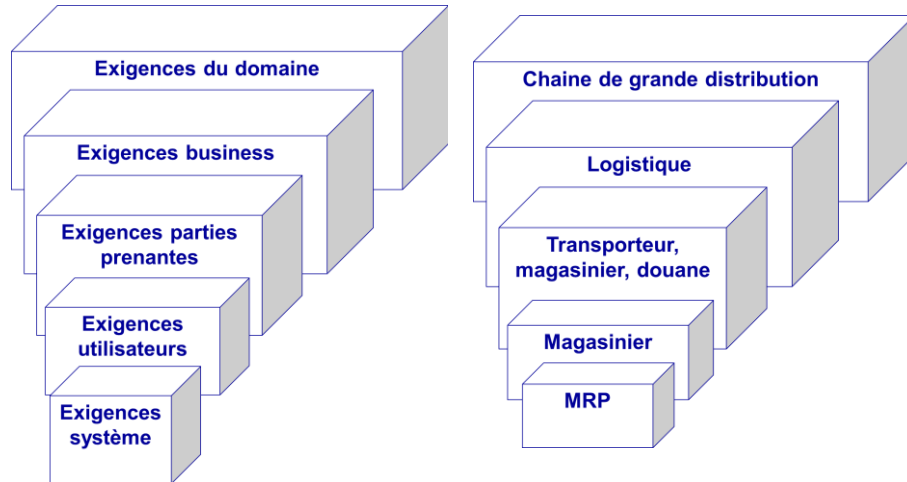
Importance des exigences non fonctionnelles

- souvent sous-estimées
- prise en charge tardive (après la mise en production)
... sous la pression des délais
- dangers :
 - dépassements de coûts et de délais, par exemple quand un logiciel peu performant doit être revu ou quand le matériel doit être renforcé
 - périodes d'interruption, par exemple lors de cyberattaques portant sur un logiciel mal sécurisé
 - faible satisfaction des utilisateurs, par exemple lorsque des exigences non fonctionnelles en termes d'ergonomie n'ont pas été prises en compte

BABOK

- **"Business Requirements** are higher-level statements of the goals, objectives, or needs of the enterprise. They describe the reasons why a project has been initiated, the objectives that the project will achieve, and the metrics that will be used to measure its success. Business requirements describe needs of the organization as a whole, and not groups or stakeholders within it...
- **Stakeholder Requirements** are statements of the needs of a particular stakeholder or class of stakeholders. They describe the needs that a given stakeholder has and how that stakeholder will interact with a solution. Stakeholder requirements serve as a bridge between business requirements and the various classes of solution requirements...
- **Solution Requirements** describe the characteristics of a solution that meet business requirements and stakeholder requirements.
- They are frequently divided into sub-categories, particularly when the requirements describe a software solution:
 - **Functional Requirements** describe the behavior and information that the solution will manage. They describe capabilities the system will be able to perform in terms of behaviors or operations—specific information technology application actions or responses.
 - **Non-functional Requirements** capture conditions that do not directly relate to the behavior or functionality of the solution, but rather describe environmental conditions under which the solution must remain effective or qualities that the systems must have. They are also known as quality or supplementary requirements..."

Catégories d'exigences



Thierry Van den Berghe

135

Plan

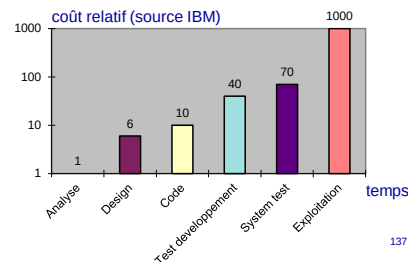
- définition
- taxonomie des exigences
- **enjeux et défis des exigences**
- cycle de l'IE

Thierry Van den Berghe

136

Enjeu de l'ingénierie des exigences

- conditionne la satisfaction des utilisateurs
 - problème mal défini → mauvaise solution
- des insuffisances au niveau des exigences sont à la source de nombreuses difficultés
 - dépassement des coûts et des délais, voire abandon du projet de développement
 - logiciel de piètre qualité
 - manque d'adhésion des utilisateurs
- le coût de correction d'erreurs évolue très rapidement avec l'avancement du projet



Thierry Van den Bergh

137

Facteurs de succès et d'échec des projets TIC

Project success factors		Project impaired factors	
Related to RE : 40%		Related to RE : 44,1%	
1. User involvement	15,9%	1. Incomplete requirements	13,1%
2. Executive management support	13,9%	2. Lack of user involvement	12,4%
3. Clear statement of requirements	13,0%	3. Lack of resources	10,6%
4. Proper planning	9,6%	4. Unrealistic expectations	9,9%
5. Realistic expectations	8,2%	5. Lack of executive support	9,3%
6. Smaller project milestones	7,7%	6. Changing requirements & specifications	8,7%
7. Competent staff	7,2%	7. Lack of planning	8,1%
8. Ownership	5,3%	8. Didn't need it any longer	7,5%
9. Clear vision & objectives	2,9%	9. Lack of IT management	6,2%
10. Hard-working, focused staff	2,4%	10. Technology illiteracy	4,3%
Other	1,9%	Other	9,9%

(Standish Group, The CHAOS report, 1994-1998)

sources de retard ou de non satisfaction des utilisateurs
(Standish Group, 2006)



Thierry Van den Bergh

138

Défis de l'ingénierie des exigences (1)

- défi : question non encore résolue selon la littérature
 - porteur d'incertitude et de risque
 - combat qui n'est pas (encore) gagné
- le traitement de problèmes complexes et mal définis comme des problèmes de gestion
 - ...avec des systèmes TIC qui ne tolèrent pas l'incertitude
- l'obligation de combiner des "hard skills"
 - compétences analytiques et techniques
 - modélisation, etc.
- ...et des "soft skills"
 - compétences relationnelles et organisationnelles
 - gestion d'équipe, communication, négociation, etc.

Thierry Van den Berghe

139

Défis de l'ingénierie des exigences (2)

- l'existence d'une large palette de méthodes, d'activités, de techniques et d'outils... à choisir en fonction de chaque situation
 - pas de recette magique ni de solution unique
 - cf. BABOK pour un inventaire (non exhaustif)

Chapter 10: Techniques

10.1	Acceptance and Evaluation Criteria	217	10.33	Prioritization	311
10.2	Backlog Management	220	10.34	Process Analysis	314
10.3	Balanced Scorecard	223	10.35	Process Modelling	318
10.4	Benchmarking and Market Analysis	226	10.36	Prototyping	322
10.5	Brainstorming	227	10.37	Reviews	326
10.6	Business Capability Analysis	230	10.38	Risk Analysis and Management	329
10.7	Business Cases	234	10.39	Roles and Permissions Matrix	333
10.8	Business Model Canvas	236	10.40	Root Cause Analysis	335
10.9	Business Rules Analysis	240	10.41	Scope Modelling	338
10.10	Collaborative Games	243	10.42	Sequence Diagrams	341
10.11	Concept Modelling	245	10.43	Stakeholder List, Map, or Personas	344
10.12	Data Dictionary	247	10.44	State Modelling	348
10.13	Data Flow Diagrams	250	10.45	Survey or Questionnaire	350
10.14	Data Mining	253	10.46	SWOT Analysis	353
10.15	Data Modelling	256	10.47	Use Cases and Scenarios	356
10.16	Decision Analysis	261	10.48	User Stories	359
10.17	Decision Modelling	265	10.49	Vendor Assessment	361
10.18	Document Analysis	269	10.50	Workshops	363
10.19	Estimation	271			
10.20	Financial Analysis	274			
10.21	Focus Groups	279			
10.22	Functional Decomposition	283			
10.23	Glossary	286			
10.24	Interface Analysis	287			
10.25	Interviews	290			
10.26	Item Tracking	294			
10.27	Lessons Learned	296			
10.28	Metrics and Key Performance Indicators (KPIs)	297			
10.29	Mind Mapping	299			
10.30	Non-Functional Requirements Analysis	302			
10.31	Observation	305			
10.32	Organizational Modelling	308			

Thierry Van den Berghe

140

Défis de l'ingénierie des exigences (3)

- la mise en place d'un dialogue constructif entre plusieurs groupes d'acteurs parfois forts différents
 - spécialistes TIC, spécialistes du domaine, experts externes, etc.
 - diversité dans les cultures, les intérêts, le vocabulaire, etc. → à concilier
- la difficulté des utilisateurs à exprimer leurs exigences
 - SI = immatériel, donc difficile à imaginer pour l'utilisateur
 - les systèmes à construire sont parfois novateurs, et l'expérience des utilisateurs est réduite
 - rappel (Agile) : les spécifications émergent progressivement au fur et à mesure de l'avancement du projet

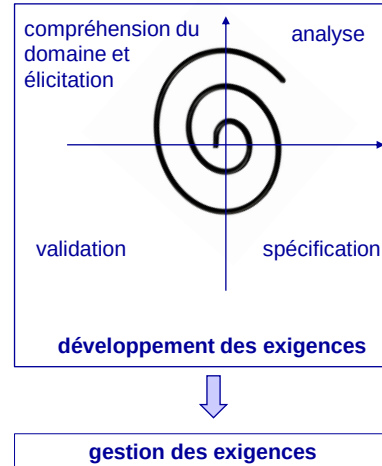
Plan



- définition
- taxonomie des exigences
- enjeux et défis des exigences
- **cycle de l'IE**

Activités de l'ingénierie des exigences

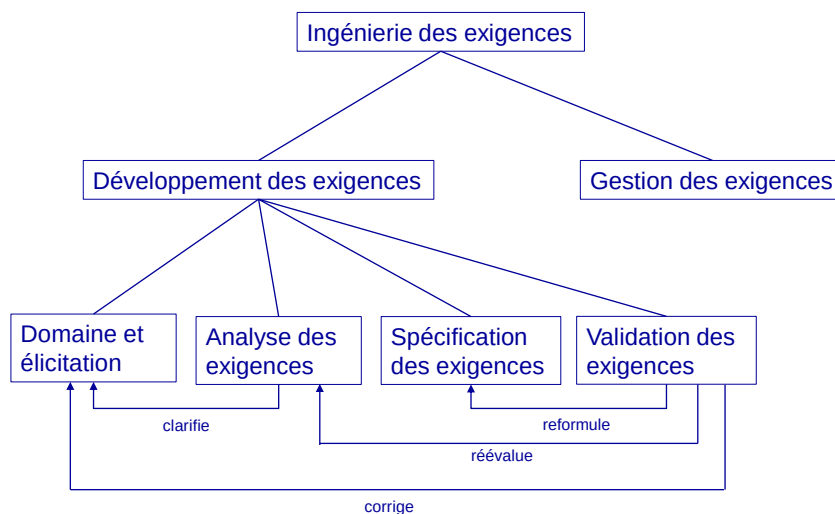
- les activités de l'IE sont interdépendantes
 - elles se succèdent dans le temps
 - la sortie de l'une peut constituer une entrée de l'autre
- en pratique : les activités sont itérées, et se superposent avec des allers-retours



Thierry Van den Berghe

143

Nature des interconnexions entre les activités



Thierry Van den Berghe

source : van Lamsweerde

144



21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences**
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Plan

- **domaine et acteurs**
- **élicitation**
- **sources de connaissances**
 - existant
 - interview
 - observation

Activités de l'IE : compréhension du domaine et élicitation

- compréhension du domaine
 - étudier les particularités du domaine
 - étudier le **système existant** et les problèmes qu'il ne sait pas résoudre
 - se faire une idée des acteurs, de l'organisation elle-même et de son environnement
 - bien utile au Business Analyst pour dialoguer avec les acteurs
 - une expérience du domaine d'application est un avantage
- élicitation des exigences
 - comprendre les besoins des acteurs pour les expliciter en exigences
 - voir comment résoudre les problèmes posés avec le **futur système**
 - ex. : comment optimiser un processus d'affaire via l'automatisation de certaines tâches

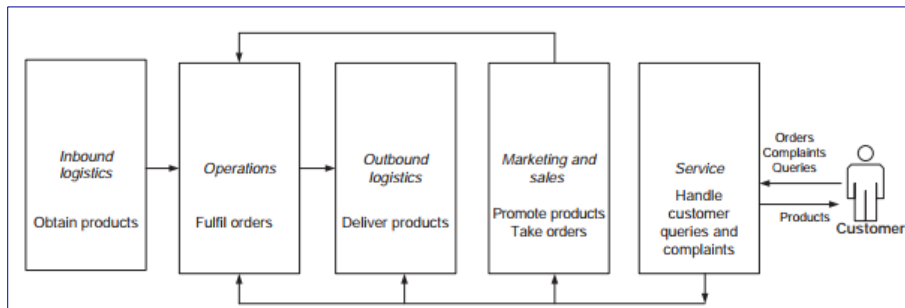
BABOK

- "Business analysis involves understanding **how organizations function** to accomplish their purposes, and defining the capabilities an organization requires to provide products and services to external stakeholders. It includes the definition of **organizational goals**, how those goals connect to specific objectives, determining the courses of action that an organization has to undertake to achieve those goals and objectives, and defining how the various **organizational units** and stakeholders within and outside of that organization interact."

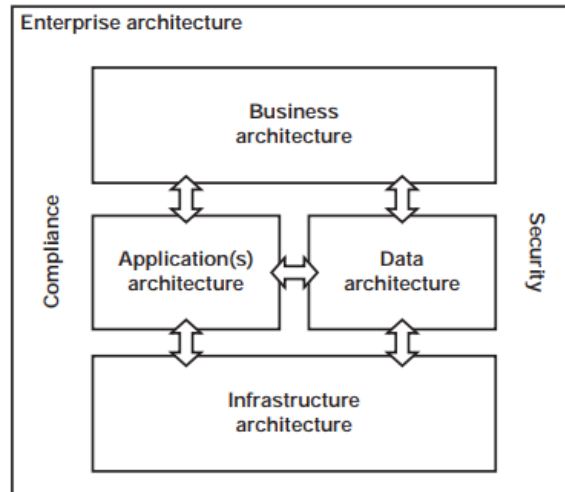
Plusieurs niveaux d'abstraction interdépendants

- **niveau entreprise**
 - activités de haut niveau pour fournir un service ou un produit au client
 - ex. : schémas de chaînes de valeur
- **niveau processus**
 - processus d'affaires détaillés qui répondent à l'occurrence d'un événement
 - ex.: diagrammes d'activité UML ou diagrammes BPMN
- **niveau acteur**
 - actions réalisées par un même acteur à un moment donnée et à un endroit donné
 - texte, pré/post condition

Exemple : chaine de valeur



Exemple : architecture d'entreprise



Thierry Van den Berghe

152

Exemple : PESTEL



Thierry Van den Berghe

153

Exemples d'éléments du domaine

- description de l'organisation
 - présentation générale
 - nature du métier, produit, etc.
 - stratégie
 - objectifs stratégiques, vision, mission, valeurs, KPI, etc.
 - environnement de l'organisation
 - partenaires, concurrence, marché, régulation, etc.
 - structure organisationnelle
 - rôles, acteurs clés, parties prenantes (stakeholders), organigramme
- description du système existant
 - architecture
 - fonctionnalité
 - principales lacunes

Acteurs

- acteur ou partie prenante : quelqu'un qui a un intérêt dans la réussite du projet de développement
- "C : someone who is committed to the project
- R : represents a group of stakeholders
- A : is authorized to make decisions
- C : will collaborate with team members
- K : is knowledgeable"

Identifier les acteurs

- l'identification des acteurs est cruciale pour ne pas omettre certaines exigences
 - sources d'informations : organigramme, sponsor, template qui reprend une liste de rôles possibles, etc.
- attitudes des acteurs par rapport au projet
 - positive : utilisateurs champions, contributeurs importants
 - négative : ne perçoivent pas la valeur du projet, voient d'abord les difficultés plutôt que les solutions, sont hostiles au changement → mettre en place une collaboration pour gagner l'adhésion
- rappel : facteur clé de succès !

Exemples de template

Operational roles

- Normal operations
- Maintenance
- Support
- Helpdesk
- Training
- Planning, scheduling
- Control, management, administration

Beneficiaries

- Functional beneficiary
- Social beneficiary, etc (if indirect benefits exist)
- Financial beneficiary
- Political beneficiary (not only professional politicians)

Interfacing roles

- Owner of interfacing system (sharing data, etc)
- Neighbouring business (mutual benefit)
- Interoperating service (sharing facilities/equipment)

Surrogate roles (representing or working on behalf of others)

- Champion
- Purchaser (roles here vary widely and often overlap)
 - Internal customer
 - Procurement
 - Project sponsor
 - Marketing (representing the consumer)
 - Product manager
- Developer (many roles possible here)
 - Requirements analyst
 - Designer
 - User interface designer
 - Programmer
 - Tester (very useful in requirements work)
 - Technical writer
- Manufacturer/subcontractor/supplier
- Expert/consultant (many more roles possible here)
 - Human factors/ergonomics
 - Safety engineer
 - Security engineer
 - Simulation/modelling expert
 - Legal opinion
 - Translator, cultural advisor, etc
- Regulator
 - Parliament, statute law
 - Government departments, regulations
 - Statutory regulators
 - Standards bodies (national and international)
 - Voluntary/industry regulators

source : Alexander

Hybrid roles

- User (= Normal Operator + Functional Beneficiary)
- Consumer (= Mass-market Purchaser + User)
- Service Provider (= Operational Support + Maintenance + Helpdesk, and sometimes Developer, Subcontractor, Manufacturer)
- Risk and Revenue-Sharing Partner (= Developer + Manufacturer + Financial Beneficiary), etc

Negative/hostile roles

- Vandal, graffiti artist
- Thief
- Hacker, virus writer
- Competitor
- Industrial espionage (via malware, etc)
- Fraudster
- Disgruntled employee
- Trades union
- Political opponent
- The public, residents' association, etc
- Activist, environmental pressure group, etc
- Military enemy, terrorist

Une longue histoire d'opposants...



Grève des facteurs(Belga) La Libre Belgique, 13/09/2006 :

La situation est tendue : le système **Géoroute 2** est une sorte de chaîne ou tout le monde doit faire plus, plus vite, et les agents ont déjà des journées plus que complètes.», affirme Eve Izquierdo, déléguée CGSP.

Grève des postiers de Flémalle, La Libre Belgique, 30/01/2007

Les postiers se positionnent contre le logiciel **Géoroute 2**,...

Le bureau de distribution s'oppose à **Georoute 3**, Le soir, 17/2/2009

Bpost : la CSC ne veut plus discuter, La Libre Belgique , 19/07/2012

La direction est suspectée de faire passer en douce la réforme de **Géoroute**.

Thierry Van den Berghe

Documenter les acteurs (1)

coordonnées	rôle	contacts	priorité	types d'exigences	techniques d'éllicitation
Jean Dupond	key user	réunion les 2/2 ; 8/2	élevé	fonctionnel	interview
Anne Durant	responsable sécurité	mail du 14/2, réunion du 4/3	moyenne	sécurité, régulation, données personnelles	échange de textes légaux par mail
Henry Golant	comptable		faible	interface avec le système comptable	échange de spécifications techniques

Thierry Van den Berghe

159

Documenter les acteurs / BABOK (2)

Figure 10.43.1: Stakeholder Matrix

Influence of Stakeholder	High	Ensure stakeholder remains satisfied.	Work closely with stakeholder to ensure that they are in agreement with and support the change.
	Low	Monitor to ensure stakeholders interest or influence do not change.	Keep informed; stakeholder is likely to be very concerned and may feel anxious about lack of control.
		Low	High
		Impact on Stakeholder	

- **Responsible (R)**: the persons who will be performing the work on the task.
- **Accountable (A)**: the person who is ultimately held accountable for successful completion of the task and is the decision maker. Only one stakeholder receives this assignment.
- **Consulted (C)**: the stakeholder or stakeholder group who will be asked to provide an opinion or information about the task. This assignment is often provided to the subject matter experts (SMEs).
- **Informed (I)**: a stakeholder or stakeholder group that is kept up to date on the task and notified of its outcome. Informed is different from Consulted as with Informed the communication is one-direction (business analyst to stakeholder) and with Consulted the communication is two-way.

Thierry Van den Berghe

Figure 10.43.3: RACI Matrix

Change Request Process	RACI		
Executive Sponsor		A	
Business Analyst		R	
Project Manager		C	
Developer		C	
Tester		I	
Trainer		I	
Application Architect		C	
Data Modeller		C	
Database Analyst (DBA)		C	
Infrastructure Analyst		C	
Business Architect		R	
Information Architect		C	
Solution Owner		C	
Subject Matter Expert (SME)		C	
Other Stakeholders	R	C	I
(varies)			

Collaboration et communiquer avec les acteurs

- **planifier la collaboration** :
 - définir les modalités de collaborations en fonction du profil des acteurs
 - timing et fréquence des interactions, localisation, outils supports à la communication (mail, wiki, etc.)
- **évaluer les besoins en communication**
 - que faut-il communiquer, à qui et quand ?
 - quel média de communication ?
 - quel niveau de détails dans la communication ?
 - quel niveau de formalisme ?
 - → plan de communication

Thierry Van den Berghe

161

Acteurs d'un projet – BABOK (1)

- **Business Analyst** : intermédiaire entre utilisateurs et développeurs
- **Client** : bénéficiaire des produits et services de l'organisation
- **Expert du domaine** : référents disposant d'une connaissance approfondie de l'organisation et de ses besoins
- **Utilisateur final** : acteur qui interagit avec le système
- **Expert technique** : spécialiste des techniques à mettre en œuvre pour la réalisation du système l'implémentation (développeur, administrateurs de bases de données, ergonomes, etc.)
- **Support opérationnel** : regroupe les responsables de la gestion opérationnelle et de la maintenance du système (help desk, release manager, web master, etc.)
- **Chef de projet** : responsable des différents aspects de la gestion du projet de développement (respect des objectifs, gestion d'équipe, planification, etc.)

Thierry Van den Berghe

162

Acteurs d'un projet – BABOK (2)

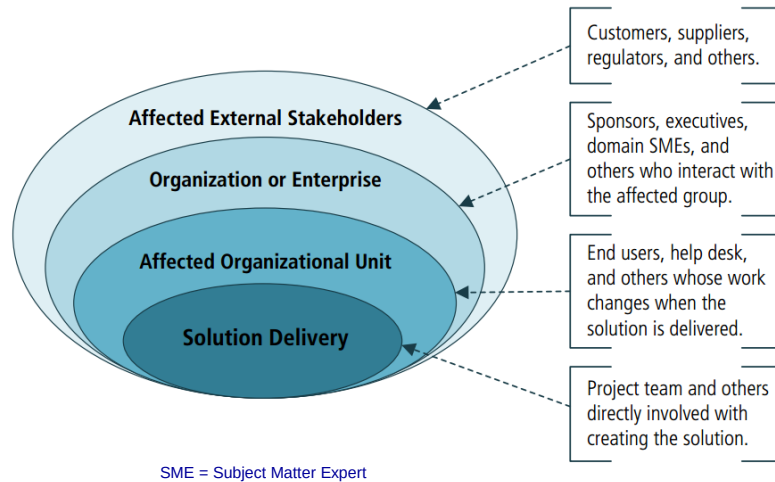
- **Régulateur** : responsable de la définition et du respect de standards (réglementation, réglementations métier, etc.)
- **Sponsor** : responsable du lancement du projet, et du contrôle de son budget et de sa réalisation
- **Fournisseur** : tiers qui fournissent des produits et services à l'organisation
- **Testeur** : responsable de la vérification de l'atteinte des objectifs par le système et de sa qualité
- Une même personne peut endosser plusieurs rôles
- Liste non exhaustive
 - partenaire - interface : expriment des exigences concernant la manière dont le futur système va interagir avec d'autres systèmes ou services
 - ex. : administration

Thierry Van den Berghe

163

Diagramme des acteurs – stakeholders (BABOK)

Figure 10.43.2: Stakeholder Onion Diagram



Thierry Van den Berghe

164

Exercice

- décrivez le contexte de votre stage en présentant l'entreprise et les acteurs de votre (vos) projet(s)

Thierry Van den Berghe

165

Plan



- domaine et acteurs
- **élicitation**
- sources de connaissances
 - existant
 - interview
 - observation

Élicitation des exigences (1)

- identification des exigences **potentielles** relatives au futur système
 - elles seront confirmées dans les activités en aval
 - les exigences doivent intégrer :
 - les objectifs **d'amélioration** portés par le futur système
 - les **opportunités** portées par les dernières avancées des TIC
 - les **limites** de l'automatisation (frontière entre le système et son environnement)
 - les **contraintes** qui restreignent les solutions possibles
- ex. : personnaliser la communication avec les clients
 - ex. : recommandation automatique pour conseiller chaque client
 - ex. : réponse à une plainte client réalisée par un acteur humain
→ hors système
 - ex. : règlement sur la protection des données personnelles

Exemple Galion

- les **objectifs d'amélioration**
 - ouvrir le système existant sur l'Internet
 - amener des facilités de front office automatisé (e-shop)
 - amener des facilités logistiques
- les **opportunités**
 - automatisation de la vente → réponse à l'objectif de développement du volume d'affaires
 - présence sur le Web
- les **limites de l'automatisation**
 - la préparation de commande reste manuelle
- les **contraintes**
 - certains produits fragiles ne peuvent pas être expédiés au client

Thierry Van den Berghe

168

Élicitation des exigences (2)

- l'élicitation prend la forme d'**apprentissage coopératif** entre le Business Analyst et les utilisateurs
 - but : éliciter les bonnes exigences
 - rappel : bonne collaboration entre les acteurs = facteur clé de succès
- **remarque : un terme plus correct serait : élucidation**
 - recherche, découverte, acquisition et élaboration

Thierry Van den Berghe

169

Difficultés (1)

- **éparpillement de la connaissance**
 - détenue par plusieurs acteurs (utilisateurs différents, management, etc.)
 - localisée dans plusieurs sources (intranet, rapports d'activités, etc.)
 - conflits dans les points de vue des différents acteurs et sources
- **problèmes d'accès à la connaissance**
 - disponibilité des acteurs, notamment dans un contexte international
 - sources confidentielles
- **obstacles à la communication**
 - jargon inconnu de certains acteurs (TIC et domaine)
 - différences de culture

Difficultés (2)

- **connaissance non explicite**
 - les acteurs considèrent certaines connaissances comme implicites ou évidentes et ne prennent pas la peine de les formuler
 - tous les acteurs ne sont pas des pédagogues nés
 - rappel : difficulté d'imaginer un système immatériel qui reste à construire
- **difficultés socio-politique**
 - manque d'adhésion au projet, résistance au changement
- **instabilité**
 - changement des exigences, des acteurs, etc. ... en cours de projet
- **biais**
 - les acteurs se comportent différemment de ce qu'ils prétendent

Exemple

- le futur système doit prendre en charge le processus de décision d'acceptation de l'inscription en ligne d'un nouvel étudiant, mais...
 - les critères d'acceptation d'une inscription ne sont pas documentés
 - les critères varient d'un utilisateur à l'autre
 - certains utilisateurs sont plus soigneux pour étudier un dossier d'inscription devant le Business Analyst que lorsqu'ils travaillent seuls
 - certains utilisateurs refusent d'explicitier les critères d'acceptation car ils ne veulent pas être remplacés par une machine
 - certains utilisateurs renvoient à la "direction" qui est le seul décideur à leurs yeux

Interaction avec les acteurs

- comment répondre à ces difficultés?
- développer des compétences communicationnelles
 - avoir une grande faculté d'écoute
 - adopter des techniques de collaboration, un langage et un vocabulaire adaptés aux acteurs
 - devenir un partenaire de confiance, et surtout pas un auditeur
 - bien identifier et impliquer tous les acteurs
- s'assurer de l'adhésion des acteurs
 - motiver le projet, démontrer sa valeur, etc.
- reformuler la connaissance
 - intégration et structuration de la connaissance acquise sur le terrain
 - procédure de validation (formelle) de la reformulation avec les acteurs

Quelques derniers conseils

- comprendre le domaine d'application avant l'élicitation
- se rappeler les parties prenantes ne savent pas toujours ce dont elles ont besoin
- se rappeler que les parties prenantes ne communiquent pas toujours efficacement
- ne jamais faire d'hypothèse sur ce que les utilisateurs souhaitent
- éviter de donner davantage aux utilisateurs que ce qu'ils demandent
- accepter les changements dans les exigences à tout moment

Plan



- domaine et acteurs
- élicitation
- techniques d'élicitation
 - sources existantes
 - collaboration avec les parties prenantes
 - observation
 - prototypage
 - non fonctionnel
 - zoom sur les interviews

Types d'éllicitation (BABOK)

- différentes approches d'éllicitation sont possibles :
 - recherche et analyse d'informations existantes
 - démarche souvent réalisée sans les acteurs
 - site Web, BD existantes, etc.
 - collaboration avec les acteurs
 - interviews, workshops, etc.
 - expérimentation
 - identifier l'information inconnue des acteurs, et qui ne peut pas être explicitée sans certaines formes de tests
 - ex. : interface utilisateur
 - observation, prototype, proof of concept, etc.

Sources d'informations existantes

- relèvent d'une grande variété
 - descriptions existantes du domaine
 - système d'information existant
 - etc.
- permettent de se faire une première idée du domaine
- aident à préparer les entrevues avec les acteurs
 - en évitant de les solliciter à propos de connaissances disponibles par ailleurs
- soutiennent les entrevues avec les acteurs
 - pour disposer d'exemples concrets
 - pour préciser les propos des acteurs

Descriptions du domaine

- ex. : plaquette de présentation commerciale, catalogue de produits, site Web, rapport d'activités
- précises et tangibles
- souvent partielles
- information souvent biaisée car présentée en fonction d'objectifs autres que la spécification d'un système d'information

Système d'information existant

- documents d'entrée et de sortie, masques d'écran
 - ex. : bon de commande, facture, bon de fabrication, etc.
 - opportunité de critiquer l'existant
 - sources redondantes entre elles
 - trop d'instances et pas assez de concepts
- schémas de données et autre documentation technique
 - haut niveau de détail
 - obsolètes par définition
 - reverse engineering

Collaborations

- **interview**
 - typiquement : business analyst + utilisateur
 - but : transfert de connaissances + faire émerger les exigences et les solutions
- **workshop**
 - discussions de groupe avec un but fixé
 - recherche d'un consensus
 - nouvelles idées
 - etc.
 - portée plus ciblée mais plus diversifiée que les interviews
 - animé par un facilitateur

Thierry Van den Berghe

Workshop

- **interviews en groupes**
- **avantages**
 - stimulation mutuelle des participants
 - vision plus globale des exigences que lors de l'interview d'un seul acteur
 - opportunité de résoudre des conflits directement pour arriver rapidement à un consensus
- **inconvénients**
 - coût de mobilisation de plusieurs personnes
 - difficulté d'animer des discussions de groupes
 - équilibre des temps de parole
 - réduction de l'impact des conflits interpersonnels
 - etc.

Thierry Van den Berghe

181

Observation

- suppose qu'il est plus simple de comprendre le déroulement d'une tâche en l'observant qu'en se la faisant expliquer
 - ex. : lier les lacets d'une chaussure
- avantage
 - utile pour comprendre des processus difficiles à expliquer
 - adapté aux processus impliquant plusieurs acteurs n'ayant pas conscience du rôle des autres acteurs
- inconvénient
 - restreint à des processus facilement observables (assez rares en informatique de gestion)
 - assez coûteux

Prototypage

- construction de modèles provisoires ou éphémères du futur système
 - exécutable ou non
 - ex. : maquette d'écran
- avantages
 - facilite l'élicitation car offre une perspective tangible du futur système
 - première expérience du système qui amène à une meilleure compréhension des besoins
 - coût mieux maîtrisé car au low/no code
- inconvénients
 - coûts
 - risque de un prototype inachevé en production

Elicitation d'exigences non fonctionnelles

- nécessite des techniques spécifiques
- ex. : questionnaires ciblés
 - Quels sont les niveaux d'accès et les autorisations nécessaires pour les utilisateurs ?
 - Quelles sont les attentes en termes de temps de réponse pour les différentes fonctionnalités ?
 - Quelles sont les exigences en matière de disponibilité et de temps de disponibilité ?
 - Le système doit-il être compatible avec certaines plateformes (par exemple, navigateurs, systèmes d'exploitation) ?
 - Quelles sont les attentes en matière d'ergonomie et d'expérience utilisateur ?

Interviews (1)

- principale technique pour l'élicitation
 - permet de collecter beaucoup d'informations rapidement
- processus :
 - sélection des acteurs à interviewer en fonction de l'information cherchée
 - planification des interviews
 - déroulement des interviews avec enregistrement des réponses
 - retranscription des réponses dans des rapports d'interview
 - validation et mise à jour des rapports par les acteurs interviewés

Interviews (2)

- **interview structuré**
 - questions préparées à l'avance
 - permet d'aller à l'essentiel sans discussion périphérique
 - limite l'investigation de nouvelles idées
- **interview ouvert**
 - discussion informelle sans question fixée
 - permet de traiter de sujets auxquels le Business Analyst n'a pas pensé
 - mais on risque de passer à côté de questions essentielles au profit de détails
- **à rechercher : combinaison structuré/ouvert**
- **un interview est toujours soigneusement préparé !**

Interviews (3)

- **avantages**
 - discussion en profondeur
 - élicitation d'exigences non captables autrement (cachées, implicite)
 - validation en temps réel et feed-back direct
 - adaptable "au vol"
 - co-construction de solutions
 - implication de l'utilisateur

Interviews (4)

- risques
 - coûteux et chronophage
 - redondances
 - divergences et conflits dans les exigences
 - maîtriser les techniques d'interview demande de l'expérience
 - biais du business analyst
 - biais de l'utilisateur

Compétences du business analyst

- écoute active
- communication claire et structurée
- professionnalisme
- flexibilité
- esprit analytique

Bonnes pratiques (1)

- préparer l'interview
 - définir l'objectif et la cible
 - connaître son interlocuteur
 - préparer une liste des questions à l'avance
 - logistique
- mener activement
 - but, agenda, trace
 - premier tour de table
 - général → spécifique
 - questions ouvertes pour la fin
 - résumer
 - préciser la suite

Bonnes pratiques (2)

- garder le contrôle de l'interview
 - ! digressions, ! timing
 - couvrir les points prévus
- mettre l'interlocuteur en confiance
 - questions simples
 - s'adapter à l'interlocuteur
 - motiver, insister sur la valeur espérée
 - pas d'interview en présence du chef!
- être impertinent ... mais créatif
 - le pourquoi
 - orientation solution

Bonnes pratiques (3)

- préparer les questions
 - ! questions orientées exprimant une opinion
 - ! questions sans réponse
 - ! évidences
- garder une trace
 - rapporteur
 - modéliser en temps réel
 - enregistrer l'interview

Plan



- buts
- résolution de conflits
- gestion des risques
- choix d'alternatives
- priorités des exigences

Buts de l'analyse des exigences (1)

- l'élicitation peut produire des exigences :
 - conflictuelles, quand elles sont émises par des acteurs différents
 - alternatives, liées à des variations fonctionnelles
 - différentes options pour atteindre le même but
 - ex. : pour prolonger un prêt :
 - un lecteur envoie une demande de prolongation; le bibliothécaire modifie la date de fin **vs.** :
 - un lecteur modifie la date de fin du prêt par le système; le bibliothécaire valide cette date de fin
 - alternatives, liées à des variations de coûts
 - ex. : laisser certains traitements rares et complexes en manuel vs. automatiser le plus de traitements possibles
 - alternatives plus ou moins exposées aux risques
 - ex. : maîtriser les risques de sécurité mais avec une réduction de la convivialité ou de la fonctionnalité
 - ex. : paiement en ligne via digipass, limiter les montants des transactions sans contact

Buts de l'analyse des exigences (2)

- souvent, les exigences ne peuvent pas être toutes traitées
 - limites de budget, du délai et des ressources disponibles
 - les exigences doivent être **priorisées**, notamment en fonction de leur valeur et de leur coût de réalisation
 - ... mais :
 - les utilisateurs ne connaissent pas la complexité, et donc le coût du développement lié à une exigence
 - les développeurs ne connaissent pas la valeur apportée des exigences
 - techniques formelles d'établissement des priorités
- le but de l'analyse est :
 - de résoudre les conflits
 - de choisir les meilleures alternatives
 - de gérer les risques
 - d'établir des priorités

Nature des conflits (1)

terminologie – inconsistance-divergence

- problèmes de terminologie
 - un même concept est désigné différemment
 - ex. : un client et un patient dans un hôpital
 - un même terme recouvre plusieurs concepts
 - ex. : un cours est une AA ou une UE
- inconsistances
 - exigences impossibles à satisfaire simultanément
 - ex. : le nombre maximum d'ouvrages empruntables simultanément est
 - 3 ouvrages pour tous les lecteurs **vs.**
 - 5 ouvrages pour un professeur, et 3 pour un étudiant
 - ex. : le système de paiement
 - doit être facile d'utilisation et rapide à utiliser **vs.**
 - doit être hautement sécurisé

Nature des conflits (2)

- divergence
 - exigences impossibles à satisfaire sous certaines conditions
 - ex. : un lecteur doit rendre un ouvrage après 2 semaines maximum **vs.** un lecteur rend un ouvrage dès qu'il n'en a plus besoin
 - → conflit si un lecteur a besoin d'un ouvrage pendant plus de 2 semaines
- techniques de résolutions
 - terminologie : définir formellement les termes utilisés dans un glossaire
 - inconsistance et divergence : négocier un accord entre les acteurs concernés
 - cf. van Lamsweerde pour davantage de détails

Gestion des risques



- un risque lié à une exigence est un facteur incertain dont la survenance peut impacter négativement l'objectif du futur système
 - une analyse des risques permet de détecter les exigences sensibles et de prendre une décision à leur sujet
- un risque est caractérisé par une **vraisemblance** et un **impact**
 - analyser un risque = déterminer sa vraisemblance et son impact
 - des contremesures peuvent être activées pour contrôler les risques, i.e. réduire sa vraisemblance et/ou son impact
 - la gestion des risques est abordée en détails dans d'autres enseignements – cf. UE Gestion de projets

Exemple

- exigence : le client doit pouvoir annuler une commande quand elle n'est pas encore expédiée
- identification du risque
 - coût de remise en stock d'une commande à annulée
- analyse du risque
 - probabilité de survenance : moyenne
 - impact : modéré
 - le risque est considéré comme moyennement critique
- contrôle
 - mise en place d'une contre-mesure : fixer un délai de 2 heures maximum après la création d'une commande pour son annulation
 - 2 heures = temps moyen avant la préparation

Choix d'alternatives : enjeux

- souvent, le Business Analyst doit argumenter et poser des choix entre des alternatives
 - les alternatives peuvent être évaluées selon différents critères
 - les alternatives les mieux notées sont choisies
- le processus d'évaluation doit être explicité
 - pour permettre un débat
 - pour justifier formellement le choix

Choix d'alternatives : approche qualitative

- évaluation de l'impact des alternatives sur des objectifs à atteindre
 - ex. : objectifs non fonctionnels de performance, de sécurité et de coût
 - ex. : évaluation selon une échelle à 4 niveaux : --, -, +, ++

Options/objectifs	performance	sécurité	coût
virement préalable	--	++	+
paiement en ligne	++	-	-
paiement à la livraison	+	--	+

Choix d'alternatives : approche quantitative

- analyse multicritères pondérée
 - évaluation des alternatives par rapport à des critères pondérés
 - chaque alternative obtient un score calculé comme suit :
 - $\text{scoreTotal}(\text{alternative}) = \sum (\text{score}(\text{alternative}, \text{critère}) * \text{poids}(\text{critère}))$
- ex. : poids [1→5], score [1→10]

		score des alternatives	
critères	poids	évaluation en ligne	évaluation DRH
coût de mise en œuvre	3	8	2
coût en production	4	2	8
fiabilité	5	5	7
temps de réponse	3	8	3
image de modernité	2	9	2
objectivité	4	7	5
score total		127	106

Thierry Van den Berghe

203

Priorités (BABOK)

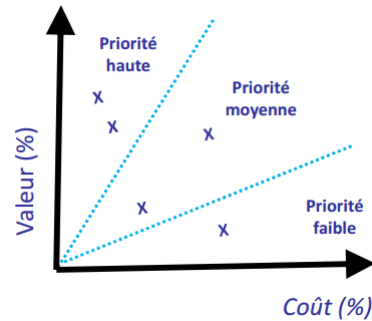
- classification des exigences par ordre d'importance relative
- facteurs à considérer :
 - **apport** : mesure de la contribution à la réalisation d'objectifs de l'exigence
 - **pénalité** : conséquence de ne pas implémenter l'exigence, comme ignorer une obligation légale
 - **coût**
 - **risque** : chance que l'exigence ne puisse pas apporter la valeur escomptée (faisabilité technique, rejet de la solution par l'utilisateur, etc.)
 - **interdépendance** : exigences qui ne peuvent être évitées sous peine d'entraver la réalisation d'autres exigences
 - **urgence temporelle** : exigences qui perdent leur valeur après un point dans le temps (activité cyclique)
 - **stabilité** : une exigence instable peut avoir une priorité plus faible

Thierry Van den Berghe

204

Méthode d'établissement de priorités

- classification des exigences réalisée en groupes d'acteurs
- démarche :
 - estimation de la contribution des exigences en termes de valeur et de coût
 - représentation graphique des exigences dans des zones de priorité
- plusieurs méthodes
 - Analytic Hierarchy Process (AHP)
 - priorisation Volere
 - Moscow



Thierry Van den Berghe

205

AHP (simplifié)

- étapes
 1. calcul de la contribution *relative* de chaque exigence à la **valeur** globale du projet
 - ex. : le paiement en ligne contribue-t-il plus à la valeur perçue par le client que le suivi d'expédition?
 2. calcul de la contribution *relative* de chaque exigence au **coût** global
 3. **comparaison** graphique des contributions **valeur/coût**
- échelle de contribution :
 - 1 : contribue à même hauteur
 - 3 : contribue légèrement plus
 - 5 : contribue plus
 - 7 : contribue nettement plus
 - 9 : contribue beaucoup plus

Thierry Van den Berghe

206

AHP (simplifié) : étape 1

- matrice de contributions relatives
- exemple : système de planification de réunion
- élément E_{ij} est l'inverse de l'élément E_{ji}

trouver la meilleure date
contribue nettement plus
à la valeur que
développer un Meeting
Assistant

Valeur	Trouver la meilleure date	Trouver la meilleure localisation	Paramétrage	Multilingue	Meeting Assistant
Trouver la meilleure date	1	3	5	9	7
Trouver la meilleure localisation	1/3	1	3	7	7
Paramétrage	1/5	1/3	1	5	3
Multilingue	1/9	1/7	1/5	1	1/3
Meeting Assistant	1/7	1/7	1/3	3	1

Thierry Van den Berghe

207

AHP (simplifié) : étape 2

- les valeurs sont normalisées au niveau des colonnes
 - $E_{ij, \text{normalisé}} = E_{ij} / (\sum_i E_{ij})$
 - la contribution relative de l'exigence au critère (valeur ou coût) est la moyenne des valeurs de la ligne correspondante → valeur en %

Valeur relative	Trouver la meilleure date	Trouver la meilleure localisation	Paramétrage	Multilingue	Meeting Assistant	Valeur relative
Trouver la meilleure date	0,56	0,65	0,52	0,36	0,38	0,50
Trouver la meilleure localisation	0,19	0,22	0,31	0,28	0,38	0,28
Paramétrage	0,11	0,07	0,10	0,20	0,16	0,13
Multilingue	0,06	0,03	0,02	0,04	0,02	0,03
Meeting Assistant	0,08	0,03	0,03	0,12	0,05	0,06

Thierry Van den Berghe

208

AHP (simplifié) : étape 2

idem
pour le
coût

coût	Trouver la meilleure date	Trouver la meilleure localisation	Paramétrage	Multilingue	Meeting Assistant
Trouver la meilleure date	1	1/3	1/5	1/5	1/7
Trouver la meilleure localisation	3	1	1/5	1/5	1/7
Paramétrage	5	5	1	1/3	1/5
Multilingue	5	5	3	1	1/3
Meeting Assistant	7	7	5	3	1

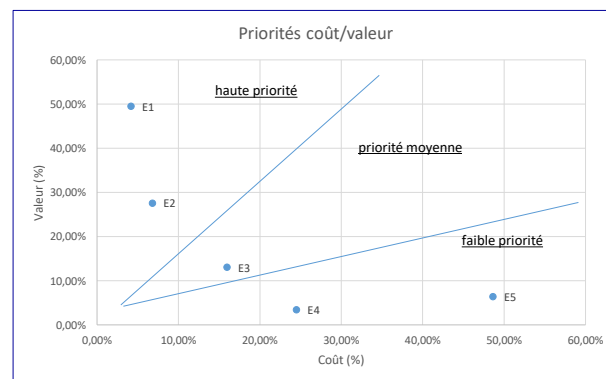
coût relatif	Trouver la meilleure date	Trouver la meilleure localisation	Paramétrage	Multilingue	Meeting Assistant	Coût relatif
Trouver la meilleure date	0,05	0,02	0,02	0,04	0,08	0,04
Trouver la meilleure localisation	0,14	0,05	0,02	0,04	0,08	0,07
Paramétrage	0,24	0,27	0,11	0,07	0,11	0,16
Multilingue	0,24	0,27	0,32	0,21	0,18	0,24
Meeting Assistant	0,33	0,38	0,53	0,63	0,55	0,49

Thierry Van den Berghe

209

AHP (simplifié) : étape 3

	Exigence	Valeur (%)	Coût (%)
Trouver la meilleure date	E1	49,51%	4,16%
Trouver la meilleure localisation	E2	27,59%	6,79%
Paramétrage	E3	13,05%	15,95%
Multilingue	E4	3,45%	24,49%
Meeting Assistant	E5	6,41%	48,61%



Thierry Van den Berghe

210

Evaluation de AHP

- ↑ reflète ce qui est important pour les utilisateurs
- ↑ permet de justifier le retrait ou le développement différé de certaines exigences (superflu ou "cherry on the cake")
- ↓ explosion combinatoire
 - regroupement d'exigences comparables dans plus petits ensembles
- ↓ subjectivité des évaluations
- ↓ pas de prise en compte de l'interdépendance entre les spécifications
 - ex. : suivi de commande par le client et l'import de données logistiques dans un e-shop
- remarque : la méthode complète propose des indicateurs de cohérence et de qualité non détaillés ici

Méthode MoSCoW : évaluation de la valeur

- acronyme : **M**ust have, **S**hould have, **C**ould have, **W**on't have
 - **Must have** : exigence incontournable pour le succès du projet
 - **Should have** : important mais pas nécessaire à court terme, à considérer plus tard
 - **Could have** : souhaitable mais pas nécessaire, souvent lié au confort, à considérer plus tard si les ressources le permettent
 - **Won't have** : exigence n'ayant pas fait l'objet d'un consensus au sein des parties prenantes, et donc non retenue dans le cadre du projet

Planning poker : évaluation du coût

- technique d'évaluation collective de la complexité de réalisation d'exigences
- déroulement (source : Wikipédia)
 1. "les participants s'installent autour d'une table, placés de façon que tout le monde puisse se voir.
 2. le responsable de produit explique à l'équipe un scénario utilisateur
 3. les participants posent des questions au responsable de produit, discutent du périmètre du scénario, évoquent les conditions de satisfaction qui permettront de le considérer comme "terminé".
 4. chacun des participants évalue la complexité de ce scénario, choisit la carte qui correspond à son estimation et la dépose, face vers le bas, sur la table devant lui.
 5. au signal du facilitateur, les cartes sont retournées en même temps.
 6. s'il n'y a pas unanimité, la discussion reprend.
 7. on répète le processus d'estimation jusqu'à l'obtention de l'unanimité."
 8. <https://www.youtube.com/watch?v=MrIZMuvjTws>

Thierry Van den Berghe

213

Plan

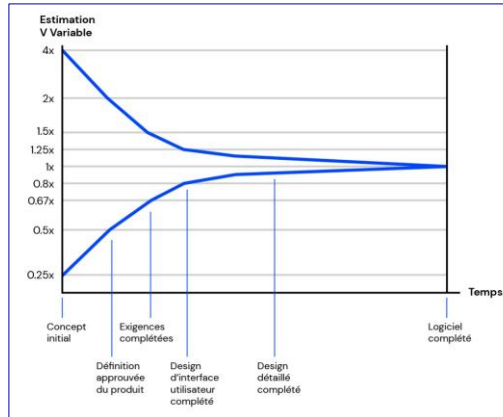


– estimation des coûts

Thierry Van den Berghe

La question qui fâche

- que doit répondre le business analyst à la question : "combien ça va coûter ?"
 - prudence !
 - estimation précise possible si les exigences sont complètes...au minimum
 - attention aux exigences non fonctionnelles
 - attention aux facteurs externes



Thierry Van den Berghe

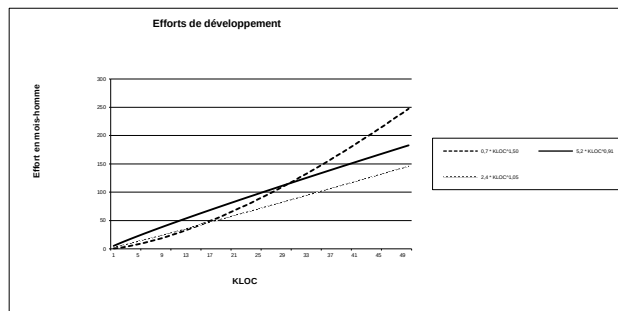
215

Estimation de l'effort de développement

- allure générale de l'effort E de développement (en personne/mois)

$$E = (a + b \text{ KLOC}^c) f(x_1, \dots, x_n)$$

- où a, b et c sont des constantes, $x_1, \dots, x_n$ des facteurs d'ajustement et KLOC = Kilo Lines Of Code
- modèle paramétrique



Thierry Van den Berghe

Constructive Cost Model simplifié (COCOMO)

Effort (personne/mois) : $E = a \text{ KLOC}^b \text{ EAF}$

Durée (mois) : $D = c E^d$

Nombre de ressources (personnes) : $N = E / D$

Constantes :

Type de projet	a	b	c	d
organique	2.4	1.05	2.5	0.38
semi-détaché	3.0	1.12	2.5	0.35
étendu	3.6	1.20	2.5	0.32

← Typique des applications de gestion

Thierry Van den Berghe

Effort Adjustment Factors-EAF (exemples)

Facteurs coût	très bas	bas	nominal	élevé	fort élevé	très élevé
Application						
fiabilité	0.75	0.88	1.00	1.15	1.40	
taille de la base de données		0.94	1.00	1.08	1.16	
complexité	0.70	0.85	1.00	1.15	1.30	1.65
Matériel (exemple)						
Temps de réponse			1.00	1.11	1.30	1.66
Personnel						
compétence des analystes	1.46	1.19	1.00	0.86	0.71	
expérience du domaine	1.29	1.13	1.00	0.91	0.82	
compétence des programmeurs	1.42	1.17	1.00	0.86	0.70	
expérience du langage de programmation	1.14	1.07	1.00	0.95		
Projet						
modernité des techniques	1.24	1.10	1.00	0.91	0.82	
outils CASE	1.24	1.10	1.00	0.91	0.83	
méthode de développement	1.23	1.08	1.00	1.04	1.10	

Thierry Van den Berghe

218

Exemples d'applications

- programme organique de 32 KLOC
 - $E = 2,4 * 32^{1,05} = 91$ personne/mois
 - $D = 2,5 * 91^{0,38} = 14$ mois
 - $N = 91/14 = 6,5$ personnes
- développement d'un OS de 35 millions de lignes
 - $E = 1\,021\,372$ personne/mois
 - $D = 210$ mois = +/- 17 ans
 - $N = 4878$ personnes
 - Remarque: première version de Windows: 1985!
 - Windows XP → Vista: 5000 personnes / 3 ans / 50 millions de lignes
- productivité implicite du modèle : 16 LOC par jour par personne
 - COCOMO inclut toutes activités hors codage (gestion du projet, etc.)

Thierry Van den Berghe

Analyse par points de fonction (FP)

- principe : identifier les éléments de l'application à construire (paramètres) et les pondérer en fonction de leur complexité
 - modèle d'estimation paramétrique
- objectif : dégager une estimation a priori du volume de l'application

Paramètre	Simple	Complexité moyenne	Elaboré
Entrées	poids = 3	poids = 4	poids = 6
Sorties	poids = 4	poids = 5	poids = 7
Requêtes	poids = 3	poids = 4	poids = 6
Fichiers ou tables	poids = 7	poids = 10	poids = 15

Thierry Van den Berghe

220

Analyse par points de fonction : exemple

Paramètre	Simple	Complexité moyenne	Elaboré	Total
Entrées	Saisie client $1 * 3 = 3$	$0 * 4 = 0$	Saisie facture Saisie produit $2 * 6 = 12$	$3 + 0 + 12 = 15$
Sorties	Fiche produit $1 * 4 = 4$	Facture Listing client $2 * 5 = 10$	$0 * 7 = 0$	$4 + 10 + 0 = 14$
Requêtes	$0 * 3 = 0$	Calcul CA/région $1 * 4 = 4$	Marge produit Marge client $2 * 6 = 12$	$0 + 4 + 12 = 16$
Fichiers ou tables	Client $1 * 7 = 7$	Produit Facture Stock $3 * 10 = 30$	$0 * 15 = 0$	$7 + 30 + 0 = 37$

nombre de paramètres * poids

Total des points de fonction :
t = 15 + 14 + 16 + 37 = 82

Thierry Van den Berghe

221

Exemples d'unités de paramètres

- **entrée : écran de saisie**
 - **évaluation de la complexité :**
 - nombre de zones de saisie/affichage
 - importance des contrôles de validité des données
 - importance de l'automatisme dans la saisie (remplissage automatique, proposition de données à l'utilisateur, etc.)
- **sortie: rapport à imprimer**
 - **évaluation de la complexité :**
 - difficulté de la mise en page
 - variété de l'information à représenter
 - construction de graphiques
 - **unité alternative : messages digitaux à envoyer vers d'autres systèmes**

Thierry Van den Berghe

Facteurs d'ajustement : fonctionnement

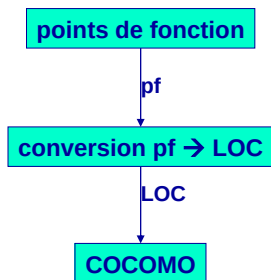
- des facteurs F_i sont évalués entre 0 (sans importance ou pas applicable) et 5 (très important)
- les valeurs des F_i sont combinées pour obtenir un nombre de points de fonction ajusté selon l'expression empirique suivante :

$$v = t * [0.65 + 0.01 * (\text{somme des } F_i)]$$

- exemples de F_i :
 - La performance est-elle critique ?
 - Le code produit doit-il être réutilisable ?
 - L'application doit-elle facilement évoluer ?
 - L'application doit-elle être simple d'utilisation ?

Thierry Van den Berghe

Conversion en lignes de code (LOC)



Outil de développement	LOC par point de fonction			
	Avg.	Median	Low	High
Access	35	38	15	47
Assembler	337	315	91	694
C	162	109	33	704
C++	66	53	29	178
COBOL	77	77	14	400
FoxPro	32	35	25	35
Informix	42	31	24	57
Java	63	53	77	-
JavaScript	58	63	42	75
JCL	91	123	26	150
JSP	59	-	-	-
Oracle	30	35	4	217
PeopleSoft	33	32	30	40
Perl	60	-	-	-
PL/1	78	67	22	263
Powerbuilder	32	31	11	105
Smalltalk	26	19	10	55
SQL	40	37	7	110
Visual Basic	47	42	16	158

Thierry Van den Berghe

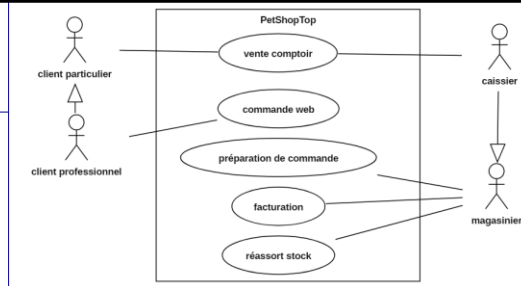
Estimation basée sur les cas d'utilisation

Step	Rule	Output
1	Classify actors: a) Simple, WF (Weight Factor) = 1 b) Average, WF = 2 c) Complex, WF = 3	Unadjusted Actor Weights (UAW) = Sum(#Actors * WF)
2	Classify use cases: a) Simple- 3 or fewer transactions, WF = 5 b) Average- 4 to 7 transactions, WF = 10 c) Complex- more than 7 transactions, WF= 15	Unadjusted Use Case Weights (UUCW) = Sum (#Use Cases * WF)
3	Calculate the Unadjusted Use Case Point (UUCP).	UUCP = UAW+ UUCW
4	Assign values to the technical and environmental factors [0..5], multiply by their weights [-1..2], and calculate the weighted sums (TFactor and EFactor). Calculate TCF and EF as shown.	Technical Complexity Factor (TCF) = $0.6 + (0.01 * TFactor)$ Environmental Factor (EF) = $1.4 + (-0.03 * EFactor)$
5	Calculate the adjusted Use Case Points (UCP).	UCP = UUCP * TCF * EF
6	Estimate effort (E) in person-hours.	E = UCP * PHperUCP where PHperUCP -> [20-36]

Thierry Van den Berghe

225

Exemple



	Acteurs		
	simple	average	complex
		client professionnel, caissier	client particulier, magasinier
nombre	0		2
poids	1	2	3
UAW	10		
	Use cases		
	simple	average	complex
	réassort stock	préparation de commande, facturation	vente comptoir, commande Web
nombre	1	2	2
poids	5	10	15
UUCW	55		
UUCP	65		
TCF	1		
EF	1		
UCP	65		
PHperUCP	25		
E(personne/heure)	1625		
E(personne/jour)	203,125		

226

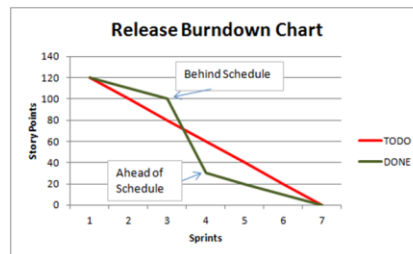
Estimation d'experts

- approche plus directe et plus intuitive que les modèles paramétriques
 - mais les paramètres des modèles sont aussi évalués par des acteurs humains...

Thierry Van den Berghe

Dans Scrum

- évaluation de story points d'une US
 - mesure relative de l'effort à produire
 - Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.)
 - fixé par consensus (Planning Poker ou autre)
- vélocité de l'équipe de développement : nbr de story points/sprint
- Burn down charts : affichent la progression du travail réalisé au cours du temps



Thierry Van den Berghe

228

Exemple

- taille du product backlog = 500 story points
- vélocité moyenne de l'équipe = 20 story points/sprint
- durée d'un sprint = une semaine
- salaire annuel d'un développeur = 100 000 €
- nombre de développeurs = 7, donc coût hebdomadaire = 13 462€
- → nombre de sprints nécessaires = $500 / 20 = 25$
- → durée = 25 semaines
- → coût du développement = $25 * 13\,462 = 336\,550$ €

Plan



- **représentation des exigences**
 - langage naturel libre
 - langage naturel structuré
 - user stories (US)
 - langage de modélisation graphique
 - modélisation
 - langage formel
- **organisation des exigences**
 - System Requirement Specification (SRS)
 - standards
 - cahier des charges

Spécification des exigences

- description détaillée des exigences
- **délivrable : document reprenant l'ensemble des exigences à destination des développeurs**
 - reprend davantage que les exigences du système futur
 - ex. : éléments de compréhension du domaine, objectifs, glossaire, parties prenantes, responsabilités, etc.
 - **System Requirement Specification** (SRS ou "**Sp**écification des **Requis** du **S**ystème")
- **comment représenter les exigences retenues?**
 - quel langage utiliser ?
 - règles au niveau micro
- **comment organiser ces exigences pour les transmettre à des tiers?**
 - règles au niveau macro

Thierry Van den Berghe

231

Représentation des exigences

- multitude de langages pour formuler les exigences
- **importance d'un "bon" langage de spécification**
 - pour pouvoir formuler toutes les exigences → **expressivité**
 - pour faciliter la communication entre les tous acteurs → **accessibilité**
 - pour être traité par des outils de génie logiciel → **formalisme**
- **expressivité, accessibilité et formalisme ne sont pas très compatibles**
 - ex. : un langage très formel perd souvent en accessibilité
 - différents langages avec leurs propres niveaux d'expressivité, d'accessibilité et de formalisme
- **à faire : trouver le meilleur compromis dans le choix du langage**
 - en fonction du profil des utilisateurs
 - en fonction du domaine d'application
 - etc.

Thierry Van den Berghe

232

Formalisme d'un langage

- le formalisme s'exprime à travers des règles qui contraignent l'expression
- plusieurs approches de la plus informelle à la plus formelle :
 - langage naturel libre
 - langage naturel structuré
 - langages graphiques
 - langages formels
- plus le langage est formel, plus un traitement automatique des exigences est possible
 - validation
 - génération automatique de spécifications techniques
 - etc.

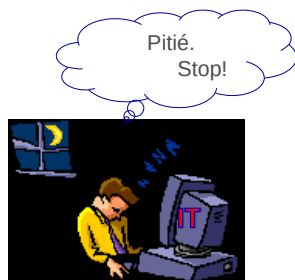
Plan

- Ingénierie logicielle
 - Ingénierie des exigences (IE)
 - Domaine et élicitation
 - Analyse des exigences
 - **Spécification des exigences**
 - Validation des exigences
 - Gestion des exigences
 - IE orientée vers les buts
 - Cas d'utilisation
 - Diagramme d'activité
 - Introduction à BPMN
 - Tests
- 
- **représentation des exigences**
 - langage naturel libre
 - langage naturel structuré
 - user stories (US)
 - langage de modélisation graphique
 - modélisation
 - langage formel
 - **organisation des exigences**
 - System Requirement Specification (SRS)
 - standards
 - cahier des charges

Langage naturel libre

- aucune contrainte dans la formulation
 - informel
 - approche la plus populaire ?
- avantages :
 - très expressif : pas de limite d'expressivité, formulation possible de n'importe quelle exigence
 - universel : adapté à tous les domaines d'application
 - très accessible : langage de tous les jours, accessible sans formation
- inconvénients :
 - pas formel : difficulté d'analyser les exigences automatiquement
 - difficulté de gérer un grand nombre d'exigences
 - limites intrinsèques dues à la richesse du langage

Trop d'information tue l'information



J'ai besoin de connaître des nombreuses informations sur mes employés. La plupart sont sympas mais il y a quand même quelques cas. Enfin bon. Chaque employé a un nom et un prénom. Y'en a qui ont jusque 5 prénoms. C'est dingue non? Mais nous avec un on a assez. Il faut aussi connaître leur âge ou leur date de naissance. C'est pas pour être indiscret ou leur offrir un cadeau d'anniversaire, mais c'est parce que le patron décide des augmentations un peu en fonction de l'ancienneté...



Limites intrinsèques du langage naturel

- **ambiguïté**
 - ex. : des chèques-repas sont distribués aux cadres et aux employés du département vente
- **silence**
 - ex. : le système détermine des remises
- **bruit**
 - ex. : les employés ne peuvent pas amener d'animaux dans les locaux
- **contradiction**
 - ex. : tous les lecteurs peuvent emprunter maximum 3 ouvrages
 - ex. : un étudiant peut emprunter maximum 5 ouvrages
- **sur-spécification**
 - ex. : spécifications concernant l'environnement du système
- **redondance**
- **référence en avant**
 - ex. : mentionner des codes qui seront expliqués plus tard dans les spécifications

Thierry Van den Berghe

237

Plan

- Ingénierie logicielle
 - Ingénierie des exigences (IE)
 - Domaine et élicitation
 - Analyse des exigences
 - **Spécification des exigences**
 - Validation des exigences
 - Gestion des exigences
 - IE orientée vers les buts
 - Cas d'utilisation
 - Diagramme d'activité
 - Introduction à BPMN
- 
- **représentation des exigences**
 - langage naturel libre
 - **langage naturel structuré**
 - user stories (US)
 - langage de modélisation graphique
 - modélisation
 - langage formel
 - **organisation des exigences**
 - System Requirement Specification (SRS)
 - standards
 - cahier des charges

Thierry Van den Berghe

Langage naturel structuré

- soumis à des règles de structure de la formulation
 - la liberté dans la formulation est limitée
 - ex. : template de formulation, user stories
- avantages
 - palie certaines limites du langage naturel libre, notamment l'ambiguïté
 - impose une formulation uniforme censée la rendre plus compréhensible
 - assez expressif
 - assez accessible (règles simples)
- inconvénients
 - difficulté de se plier à un template unique dans toutes les situations
 - limite à la créativité
 - peu formel

Thierry Van den Berghe

239

Template IEEE830

- chaque exigence prend la forme d'une phrase complète
- chaque exigence contient :
 - un **sujet** : système ou utilisateur
 - un **verbe**
 - doit (« shall ») pour exigences obligatoires
 - peut (« should ») pour exigences optionnelles
 - un **résultat** attendu
 - un éventuel **critère** de qualité
- ex. : le système doit permettre au bibliothécaire de créer un nouvel ouvrage avec toutes les données utiles pour la classification

Thierry Van den Berghe

240

Exemples IEEE

Order.Place:	The system shall let a Patron who is logged into the Cafeteria Ordering System place an order for one or more meals.
Order.Place.Register:	The system shall confirm that the Patron is registered for payroll deduction to place an order.
Order.Place.Register.No	If the Patron is not registered for payroll deduction, the system shall give the Patron options to register now and continue placing an order, to place an order for pickup in the cafeteria (not for delivery), or to exit from the COS.
Order.Place.Date:	The system shall prompt the Patron for the meal date (see BR-8).
Order.Place.Date.Cutoff:	If the meal date is the current date and the current time is after the order cutoff time, the system shall inform the patron that it's too late to place an order for today. The Patron may either change the meal date or cancel the order.

Attributs d'une exigence (1)

- les exigences sont documentées pour mieux les gérer
- attributs typiques (BABOK)
 - identifiant unique
 - auteur / propriété (utilisateur à la source de l'exigence)
 - niveau de complexité
 - priorité relative
 - risques (liste d'événements incertains qui peuvent impacter l'exigence)
 - stabilité (niveau de maturité de l'exigence)
 - statut (par exemple : proposé, accepté, vérifié, postposé, abandonné, réalisé)
 - urgence (si un délai existe spécifiquement)

Attributs d'une exigence (2)

The screenshot shows the 'Requirement: REQ019 - Manage Inventory' dialog box. On the left is a tree view with 'Properties' expanded, showing 'General', 'Rules', 'Requirements', 'Constraints', 'Scenarios', 'Related', 'Files', and 'Links'. The main area contains a text editor with the text: 'The system MUST include a complete inventory management facility to store and track stock of books for the on-line bookstore.' On the right is a properties panel with the following fields:

- Type: Functional (dropdown)
- Status: Approved (dropdown)
- Alias: Manage Inventory (text)
- Keywords: inventory, order, receive, store (text)
- Author: Paulene Dean (dropdown)
- Difficulty: Medium (dropdown)
- Priority: High (dropdown)
- Version: 1.3 (text)
- Phase: 2.0 (text)
- Package: Manage Inventory (text)
- Created: 07-03-05 (text)
- Modified: 28-02-17 12:27:15 (text)

At the bottom are buttons for 'OK', 'Annuler', 'Appliquer', and 'Aide'.

Exemple d'Enterprise Architect

Template VOLERE (www.volere.co.uk)

<u>Numéro de l'exigence</u> :	<u>Type d'exigence</u> :	<u>Événements / Cas d'utilisation</u> :
identifiant unique	référence au modèle	Liste d'événements / de use cases qui nécessitent cette exigence.
<u>Description</u> : objectif de l'exigence, en une phrase, généralement sous la forme « le produit devrait faire ceci pour telle personne »		
<u>Justification</u> : raison pour laquelle cette exigence est présente		
<u>Origine</u> : qui a émis cette exigence ?		
<u>Critère de satisfaction</u> : une mesure de l'exigence qui permet de tester si la solution proposée remplit l'exigence initiale.		
<u>Contentement du maître d'ouvrage</u> : degré de contentement du maître d'ouvrage si le produit final satisfait cette exigence. De 1 (pas intéressé) à 5 (très content).	<u>Mécontentement du maître d'ouvrage</u> : degré de mécontentement du maître d'ouvrage si le produit final ne satisfait pas cette exigence. De 1 (quasi indifférent) à 5 (très mécontent)	
<u>Exigences dépendantes</u> : liste d'exigences dont l'implémentation dépend de l'implémentation de celle-ci.	<u>Exigences conflictuelles</u> : exigences qui ne peuvent pas être implémentées si celle-ci l'est.	
<u>Documents relatifs</u> : référence à des documents qui illustrent et expliquent cette exigence.		
<u>Historique</u> : création, modifications, destruction apportées à l'exigence.		

A éviter (1)

- **ambiguïtés**
 - énoncés soumis à interprétation
 - "etc.", "ainsi de suite", cas particulier → signes d'ambiguïté
 - ex. : les étudiants doivent s'inscrire à 5 cours par session
- **exigences multiples**
 - une seule exigence par énoncé
 - "et", "ou", "ainsi que" → signes de mélange
 - ex. : un client peut ajouter un article à son panier ou le supprimer
- **clauses échappatoires**
 - énoncés couvrant plusieurs cas de figure
 - si possible : décomposer les cas
 - "si", "quand", "excepté" → signes d'échappement
 - ex. : un étudiant doit se présenter à l'examen excepté en cas de maladie

Thierry Van den Berghe

245

A éviter (2)

- **énoncés liés aux exigences techniques**
 - relève des phases en aval
 - termes techniques, références au matériel → signes de technicité
 - ex. : les données comptables doivent être stockées dans le cloud
- **spéculations et suggestions**
 - exigences qui pourraient être demandées sans certitude
 - "peut-être", "pourrait", "normalement", "souvent" → signes d'incertitude
 - ex. : normalement, un client doit régler sa facture avant expédition
 - ...mais caractériser le degré de volatilité d'une exigence (voir infra)
- **manque de précision**
 - expression de résultats ou de critères de qualité difficilement mesurables
 - "flexible", "le plus vite possible", "environ" → signes d'imprécision
 - ex. : un employé souffrant doit prévenir son employeur au plus vite

Thierry Van den Berghe

246

Plan



- **représentation des exigences**
 - langage naturel libre
 - langage naturel structuré
 - **user stories (US)**
 - langage de modélisation graphique
 - modélisation
 - langage formel
- **organisation des exigences**
 - System Requirement Specification (SRS)
 - standards
 - cahier des charges

Thierry Van den Berghe

User stories (US)

- description courte et simple d'une facilité exprimée du point de vue de l'utilisateur et qui apporte de la valeur à l'utilisateur
 - facilement compréhensible, orientée valeur et client, guide les itérations
 - mais : niveau de détails insuffisant, pas de de contexte ni de vue d'ensemble
- template de base :
en tant que <rôle>, je veux <fonctionnalité> afin de <valeur>
 - ex. : **en tant** qu'utilisateur, **je veux** pouvoir chercher mes clients par leur nom **afin de** les retrouver lorsque je reçois un appel de leur part
 - ex. : **en tant** qu'administrateur, **je veux** visualiser les log files des utilisateurs **afin de** détecter d'éventuels problèmes de sécurité

Thierry Van den Berghe

248

US : une approche portée par les méthodes AGILE

- <http://www.extremeprogramming.org/rules/userstories.html>
 - User Stories ... are used to create time **estimates** for the **release planning** meeting
 - User Stories are **written by the customers** as things that the system needs to do for them
 - User stories also **drive** the creation of the acceptance **tests**
 - User stories should only provide enough detail to make a reasonably low risk estimate of how long the story will take to implement. When the time comes to implement the story developers will go to the customer and receive a **detailed description of the requirements face to face**

US versus IEEE830 (1)

(source : <http://www.mountaingoatsoftware.com/articles/advantages-of-user-stories-for-requirements>)

- **finalité**
 - US : orienté but, guide le dialogue utilisateurs/informaticiens
 - IEEE830 : orienté exigence du système
- **niveau de détails**
 - US : limité à un relativement petit nombre de US de haut niveau d'abstraction (quelques dizaines)
 - US : détaillé par la suite oralement entre utilisateurs et informaticiens
 - IEEE830 : reprend tout le détail des exigences
 - IEEE830 : parfois de très nombreuses exigences de granularité fine
- **planification**
 - US : conditionne une itération du cycle de développement (quelques semaines dans le contexte Agile)
 - IEEE830 : n'impose aucune planification

US versus IEEE830 (2)

Dans un projet agile, les User Stories prennent la place des spécifications fonctionnelles détaillées que l'on trouve dans un projet cycle en V. Mais à la différence des spécifications fonctionnelles détaillées, **une User Story n'est pas un ensemble d'exigences fonctionnelles. Une User Story est plutôt un espace structuré, qui constitue un emplacement de choix pour entamer une conversation à propos de la manière de réaliser une fonctionnalité.** Il faut bien comprendre qu'une User Story est donc un emplacement propice à l'ajout de détails par la collaboration de l'équipe. C'est pour cette raison qu'il est inutile, voire dangereux, que le Product Owner écrive d'emblée une User Story avec tous les détails nécessaires pour la développer : on perdrait alors toute capacité à innover ensemble, à apporter des idées novatrices auxquelles le Product Owner n'a pas pensé, ou ne peut pas penser car ces idées sont directement liées à une connaissance technique pointue que seuls les développeurs possèdent.



Exemple de documentation des User Stories (1)

- **modèle des 3C (R. Jeffries)** : modèle de définition d'une US
- **card** : description écrite et tangible de l'US
- **conversation** : discussions orales pour expliciter les détails de l'US, complétées souvent par des documents
 - des règles de gestion (business rules ou critères d'acceptation)
 - des éléments de l'interface utilisateur (maquette)
 - → à reprendre sur la card
- **confirmation** : des tests d'acceptation pour valider l'implémentation
 - format Given-When-Then
 - Etant donné un contexte,
Lorsque l'utilisateur effectue certaines actions,
Alors on doit pouvoir constater telles conséquences
 - ex. (<http://referentiel.institut-agile.fr/gwt.html>) :
 - étant donné un solde positif de mon compte, et aucun retrait cette semaine,
lorsque je retire un montant inférieur à la limite de retrait,
alors mon retrait doit se dérouler sans erreur ni avertissement

Exemple de documentation des User Stories (2)

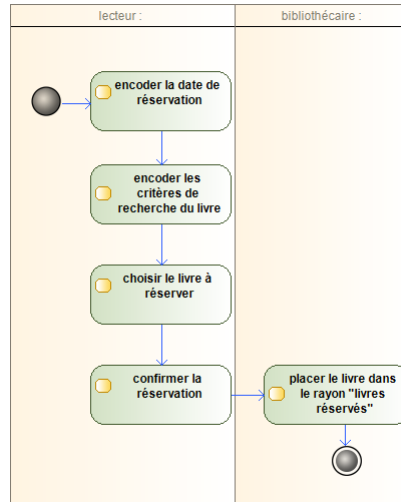
- Story
- Expected benefit
- Dimensions
 - User involved
 - Detailed actions
 - ex. BPMN, activity
 - Data
 - schema + samples
 - Interface
 - Constraints
 - Non functional requirements

Exemple de documentation des User Stories (3)

- Story
 - en tant que lecteur, je veux pouvoir réserver un livre que je souhaite emprunter afin d'être certain de sa disponibilité
- Expected benefit
 - le lecteur est assuré en ligne de la disponibilité du livre lorsqu'il souhaite l'emprunter, ce qui évite les déplacements et contacts superflus vers et avec le bibliothécaire
 - l'automatisation de la réservation permet au bibliothécaire de ne plus être sollicité par téléphone ou à son comptoir pour des réservations de livres
- Dimensions
 - User involved
 - lecteur, bibliothécaire

Exemple de documentation des User Stories (4)

- Dimensions
 - Detailed actions



Thierry Van den Berghe

255

Exemple de documentation des User Stories (5)

- Dimensions
 - Data



- Constraints
 - seuls les lecteurs en ordre de paiement de cotisation peuvent réserver un livre
 - un lecteur ne peut réserver que maximum 3 livres sur une même période
- Non functional requirements
 - l'interface utilisateur doit être responsive

Thierry Van den Berghe

256

Exemple de documentation des User Stories (6)

- Dimensions
 - Interface

Reservation

Date de début :

Date de fin :

Mot du titre :

Auteur :

liste livres

Cote	Titre	Auteurs	Reserver ?
			☒
			☐
⋮			

Thierry Van den Berghe

257

Exemple de critères d'acceptation

US — Formulaire de candidature

En tant que candidat,
je veux remplir un formulaire de candidature en ligne,
afin de postuler facilement à une offre d'emploi.

Critères d'acceptation :

- Le formulaire est accessible à partir de l'annonce d'offre d'emploi.
- Le formulaire comporte des champs obligatoires comme : nom, prénom, adresse e-mail, numéro de téléphone, lettre de motivation, etc.
- Un message d'erreur s'affiche en cas de champ obligatoire manquant.
- À la fin du processus, un message de confirmation indique que la candidature a été soumise avec succès.

Thierry Van den Berghe

258

Qualités attendues des US : modèle INVEST

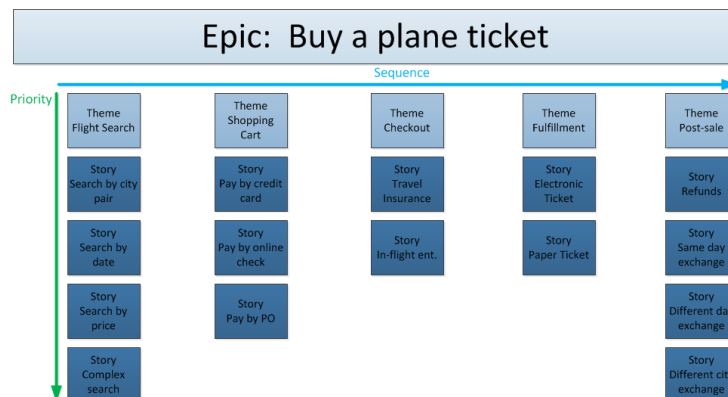
- *Independent* : les US sont indépendantes entre elles et peuvent être assorties de priorités
- *Negotiable* : les US captent l'essence d'une exigence, mais pas ses détails, qui peuvent être précisés conjointement entre utilisateurs et développeurs plus tard
- *Valuable* : les US impliquent de la valeur pour les utilisateurs
- *Estimable* : les développeurs doivent être capables d'estimer l'effort de développement de chaque US
- *Small* : typiquement 2 lignes
- *Testable* : la fonction liée à l'US doit pouvoir faire l'objet de tests

Thierry Van den Berghe

259

Organisation des US

- épopée (Epic) et thèmes : fonction globale regroupant plusieurs US
 - mécanisme de groupage
 - vers un aperçu global du projet
 - d'autres appellations sont utilisées : Epic → Feature → US



Thierry Van den Berghe

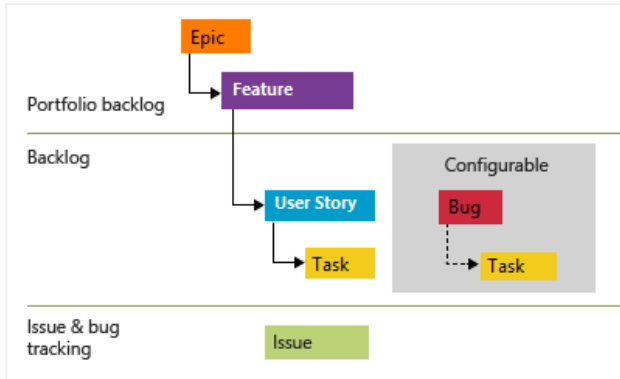
<https://www.slideshare.net/romanpichler/10-tips-for-creating-great-user-stories>
<http://www.romanpichler.com/blog/personas-epics-user-stories/>

260

Exemple : Azure DevOps

<https://docs.microsoft.com/en-us/azure/devops/boards/backlogs/define-features-epics?view=azure-devops&tabs=agile-process>

- "The epics and features that you create should reflect your business focus. As user stories or product backlog items roll up into features, and features roll up into epics—you'll want to name your features and epics with that in mind.
- A feature typically represents a shippable component of software. An epic represents a business initiative to be accomplished."

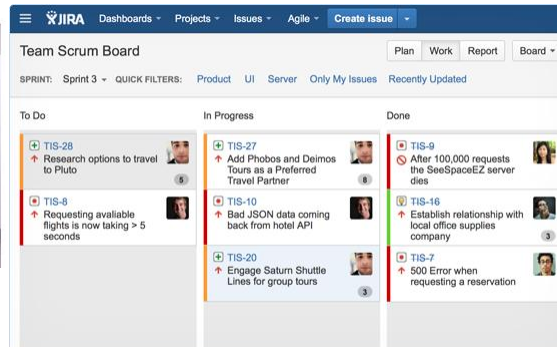
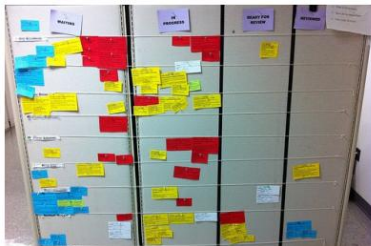


Thierry Van den Bergh

261

Tableaux

- espaces de présentation et d'organisation des US
 - soutient la gestion de projet, par exemple en classant les US selon leur stade de traitement (ex. : à faire, en cours, terminé)



Thierry Van den Bergh

262

Plan



– représentation des exigences

- langage naturel libre
- langage naturel structuré
- user stories (US)
- **langage de modélisation graphique**
- **modélisation**
- **langage formel**

– organisation des exigences

- System Requirement Specification (SRS)
- standards
- cahier des charges

Thierry Van den Berghe

Langage de modélisation graphique (1)

- synthétise les exigences sous forme de modèles

Un bon croquis vaut mieux qu'un long discours.



Napoléon Bonaparte

www.citation-celebre.com

- **langages semi-formels**
 - souvent, les éléments d'un diagramme sont déclarés formellement, et peuvent faire l'objet d'une validation automatique de leur consistance et complétude ...
 - ... mais les propriétés des éléments sont parfois spécifiées avec le langage naturel, et donc informellement

Thierry Van den Berghe

264

Langage de modélisation graphique (2)

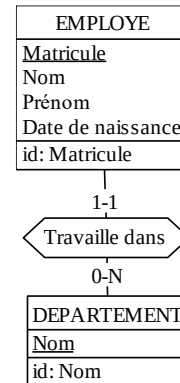
- avantages
 - les représentations graphiques sont plus synthétiques et lisibles → moins d'incohérences, d'ambiguïtés et de redondances
 - facilite la compréhension de systèmes complexes
 - semi formel
- inconvénients
 - moyennement expressif : plusieurs modèles sont nécessaires pour représenter différents aspects du système à construire
 - moyennement accessible : demande un apprentissage
 - → vérification de la consistance entre les différents diagrammes d'un même système

Modélisation

- modèle : représentation abstraite d'un problème ou de sa solution selon un langage défini
 - "représentation abstraite du futur système, dont les facilités clés sont mises en évidence, spécifiées et interconnectées" (van Lamsweerde)
 - abstrait = simplifié, sans détails superflus
- un modèle est exprimé dans un langage de modélisation
 - contrairement aux précédentes formes de représentation
 - standard pour la modélisation : UML (Unified Modeling Language)
- un langage de modélisation précise :
 - une syntaxe : définition des types d'éléments du modèle et des combinaisons valides entre ces éléments
 - une sémantique : définition de la signification d'un type d'éléments

Exemple : modèle entité-association

- **modèle entité-association :**
 - représente les structures de données
- **ex. de syntaxe :**
 - entité, attribut, association, cardinalité, identifiant
 - une association connecte des entités
- **ex. de sémantique :**
 - une entité représente une classe d'instances similaires du domaine
 - un attribut représente une propriété d'une entité ou d'une association
- **ex. de propriété informelle :**
 - un département est la structure organisationnelle principale. Il peut couvrir plusieurs localisations



Thierry Van den Berghe

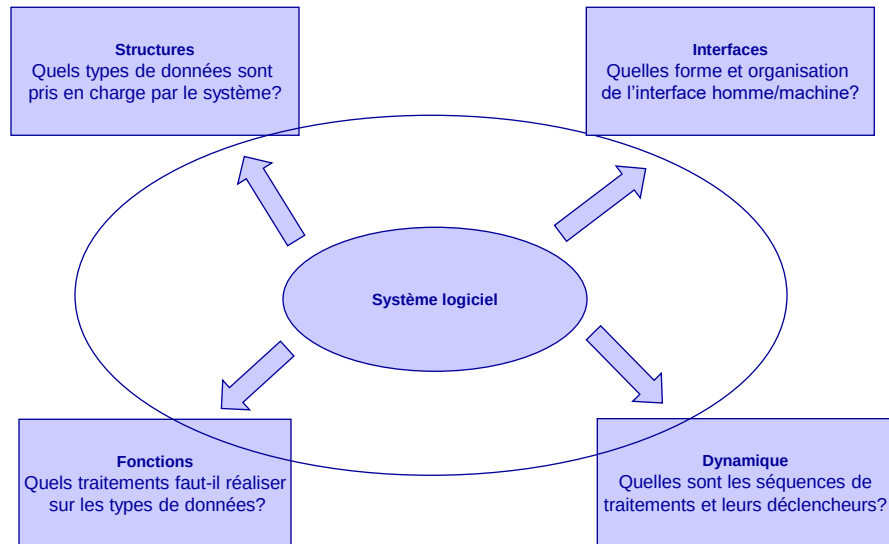
267

Utilité d'un modèle

- permet de visualiser le système avant qu'il n'existe
- offre un support :
 - au processus de création du système
 - à la prise de décisions concernant le système
 - à la communication de l'idée du système
- aide à maîtriser la complexité du système
 - simplification par élimination des détails (abstraction)
 - décomposition du système avec plusieurs modèles
- guide la fabrication du système

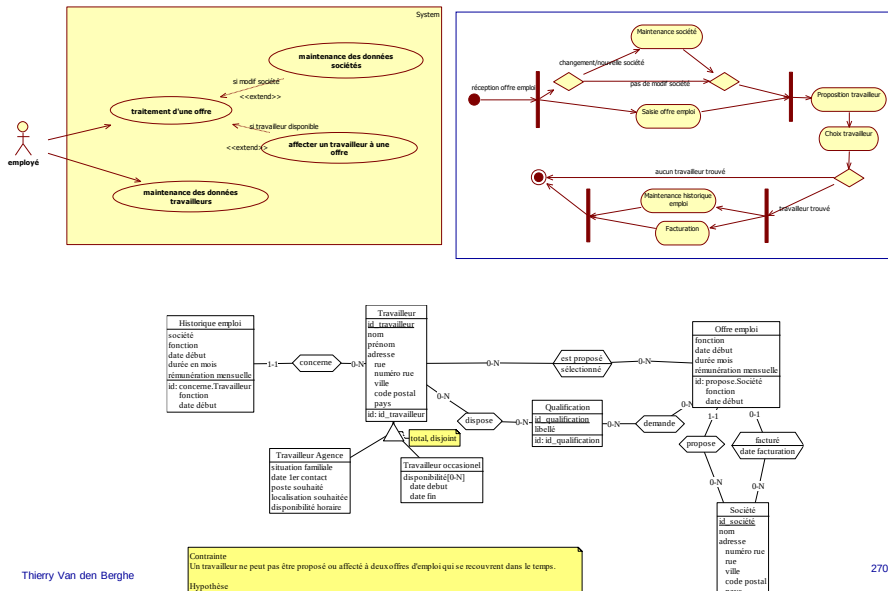
268

Dimensions du logiciel



269

Plusieurs facettes / niveaux de détails

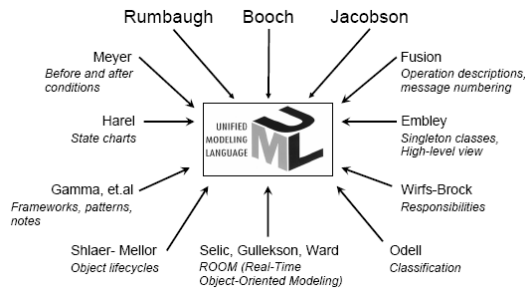


270

Thierry Van den Berghe

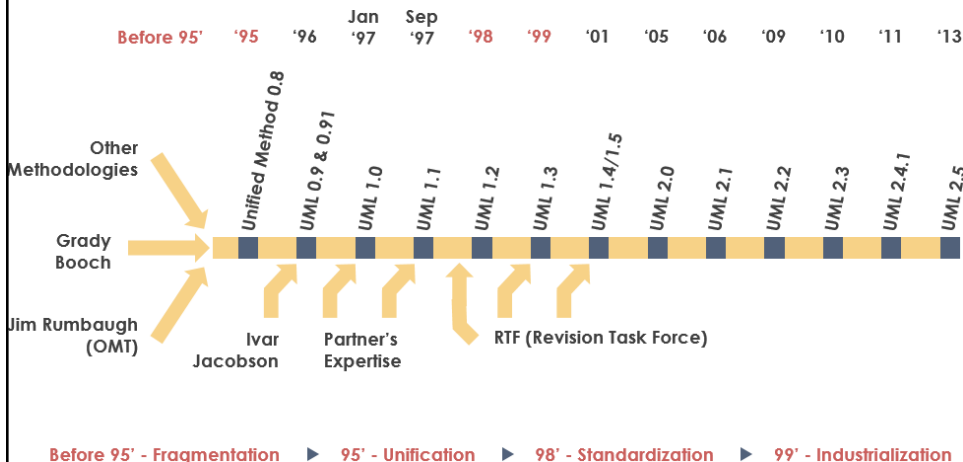
Exemples de langages graphiques : UML

- UML : Unified Modeling Language
- unification des techniques majoritairement existantes
 - développé au départ par Booch, Rumbaugh et Jacobson
 - intention d'universalité...dans un contexte commercial

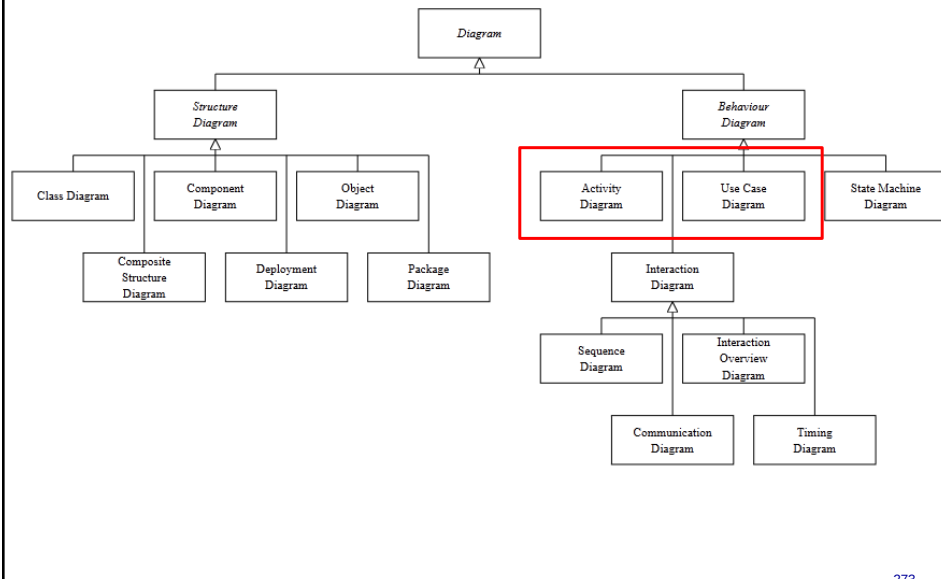


271

Historique d'UML



Composants d'UML (source Wikipédia)



Langage formel

- inspiré des langages mathématiques et de la logique
 - ex. : spécifications des exigences basées sur les états
 - pré/post conditions
 - langage Z
- avantages
 - très formel → analyse mathématique de la qualité des spécifications
 - génération automatique du code plus directe
 - assez expressif
- inconvénients
 - peu accessible → peu populaire auprès des utilisateurs
 - ... malgré le prérequis finalement assez faible

En synthèse

	expressivité	accessibilité	formalisme
langage naturel libre	***	***	-
langage naturel structuré	**	***	*
langage graphique	**	**	**
langage formel	**	*	***

Plan

– représentation des exigences

- langage naturel libre
- langage naturel structuré
- user stories (US)
- langage de modélisation graphique
- modélisation
- langage formel



– organisation des exigences

- System Requirement Specification (SRS)
- standards
- cahier des charges

System Requirement Specification

- comment organiser les exigences?
 - structuration d'un ensemble d'exigences selon des règles au niveau macro
- rédaction d'un SRS : System Requirement Specification (Spécification des Requis du Système)
 - ou document des exigences (RD – Requirement Document)
- sert de base à un cahier des charges à l'intention d'un service informatique ou d'un prestataire de service externe
 - cahier des charges : document souvent contractuel

Objectifs d'un SRS

- transmettre les exigences à la communauté des développeurs
 - référence pour délimiter le système et déterminer son achèvement
 - ce qui n'est pas mentionné dans le SRS ne doit pas être développé
- alimenter la phase de conception
 - mais des allers-retours entre les phases sont fréquentes
- structurer les spécifications en sous-problèmes de taille plus maîtrisable
 - utile dans un contexte de planification et d'estimation d'effort
- supporter la validation du système développé
 - correspondance du système aux exigences

Standards

- différents standards pour organiser le SRS
 - définissent des règles pour contraindre l'organisation du SRS
 - ex. : règles des groupements d'éléments cohérents
 - ex. : par objectifs, par sous-systèmes, par composants logiciels, etc.
- but : standardiser les échanges en acteurs pour faciliter l'estimation des budgets, délais et risques
- IEEE – std 830
- Volere (www.volere.co.uk)
 - variante du IEEE - std 830
 - distinction entre utilisateurs, clients et autres acteurs
 - inclut des sections relatives aux coûts, aux risques ou au planning

279

Modèle SRS selon IEEE-std 830 (1)

source :



Thierry Van den Bergh

Table des matières

1.	Introduction
1.1	Objectif du document
1.2	Portée du document
1.3	Définitions, acronymes et abréviations
1.4	Références
1.5	Vue d'ensemble
2.	Description générale
2.1	Perspectives du produit
2.1.1	Interfaces système
2.1.2	Interfaces utilisateurs
2.1.3	Interfaces matérielles
2.1.4	Interfaces logicielles
2.1.5	Interfaces de communication
2.1.6	Contraintes de mémoire
2.2	Fonctions du produit
2.3	Caractéristiques des utilisateurs
2.4	Contraintes
2.5	Hypothèses et dépendances
2.6	Exigences reportées
3.	Exigences spécifiques
3.1	Fonctionnalités
3.1.1	<Nom de l'exigence fonctionnelle 1>
3.2	Spécification des cas d'utilisation
3.3	Exigences supplémentaires
3.3.1	Utilisabilité
3.3.2	Faisabilité
3.3.3	Performance
3.3.4	Maintenabilité
4.	Contraintes de conception
4.1	<Nom de la contrainte de conception 1>
5.	Sécurité
6.	Exigences de documentation utilisateur et d'aide en ligne
7.	Normes applicables
8.	Classification des exigences fonctionnelles
9.	Annexes

Modèle SRS selon IEEE-std 830 (2)

- **Introduction**

- **Objet du document** → donne une introduction générale
- **Portée** → spécifie le domaine d'application traité et la portée du livrable attendu
- **Définitions, acronymes et abréviations** → précise la terminologie et les concepts spécifiques du domaine d'application
- **Références** → liste des sources de l'élicitation
- **Vue d'ensemble du document** → présente l'organisation de la suite du SRS

Modèle SRS selon IEEE-std 830 (3)

- **Description générale**

- **Perspective produit** → présente les **relations** entre le système à construire et son **environnement** en termes d'interfaces et d'interactions avec les utilisateurs et d'autres équipements matériels ou logiciels
- **Fonctions du produit** → donne un aperçu des fonctionnalités du futur logiciel
- **Caractéristiques utilisateurs** → précise les particularités des utilisateurs, comme leur degré d'expertise
- **Contraintes globales** → identifie les contraintes qui limitent le développeur, comme des réglementations ou des ressources matérielles limitées
- **Hypothèses et dépendances** → documente les facteurs environnementaux qui pourraient impacter les exigences, comme l'émergence de nouvelles règles de sécurité

Modèle SRS selon IEEE-std 830 (4)

- **Exigences spécifiques**

- → cœur du document
- → détaille toutes les exigences
- → différents templates proposés pour organiser cette section

3.	Exigences spécifiques
3.1	Fonctionnalités
3.1.1	<Nom de l'exigence fonctionnelle 1>
3.2	Spécification des cas d'utilisation
3.3	Exigences supplémentaires
3.3.1	Utilisabilité
3.3.2	Fiabilité
3.3.3	Performance
3.3.4	Maintenabilité

Cahier des charges type VOLERE (<http://www.volere.co.uk/>)

III	Contenu du cahier des charges	14
III.1	Fondements du projet – [résumé]	14
1.	But du projet	14
2.	Personnes et organismes impliqués dans les enjeux du projet	15
3.	Utilisateurs du produit	15
III.2	Contraintes sur le projet – [contenu]	15
4.	Contraintes non négociables	15
5.	Glossaire et conventions de dénomination – [résumé]	19
6.	Faits et hypothèses utiles – [résumé]	19
III.3	Exigences fonctionnelles – [résumé]	19
7.	Portée du travail	19
8.	Portée du produit (cas d'utilisations)	20
9.	Exigences fonctionnelles et exigences sur les données	20
III.4	Exigences non fonctionnelles – [contenu]	22
10.	Ergonomie et convivialité du produit	22
11.	Facilité d'utilisation et facteurs humains	24
12.	Fonctionnement du produit	28
13.	Adéquation du produit avec son environnement	34
14.	Maintenance, support, portabilité, installation du produit	38
15.	Sécurité	41
16.	Exigences culturelles et politiques	
17.	Lois et standards influençant le produit	
III.5	Autres aspects du projet – [résumé]	
18.	Questions sans réponse	
19.	« COTS » : Progiciels ou composants commerciaux	
20.	Nouveaux problèmes, créés par l'apparition du nouveau système	
21.	Tâches à faire pour livrer le système	
22.	Contrôle final de qualité sur site (Cutover)	
23.	Risques liés au projet	
24.	Estimation des coûts du projet	
25.	Manuel utilisateur et formations	
26.	Salle d'attente : idées pour les futures versions	
27.	Idées de solutions	
IV	Références	

Qualités du SRS

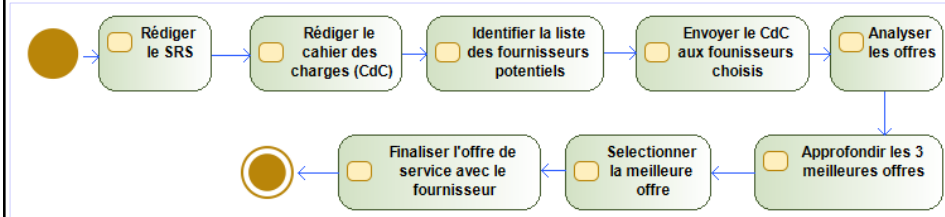
- complétude : couverture complète, suffisamment détaillé pour permettre un développement qui répond aux objectifs fixés
- cohérence : les exigences doivent être compatibles entre elles
- précision : la formulation ne doit pas être ambiguë ou laisser de l'espace à l'interprétation
- pertinence : contribution à l'atteinte d'objectifs uniquement, en évitant de traiter la solution technique
- faisabilité : les exigences doivent être réalisables moyennant les capacités des TIC, les ressources disponibles, le délai et les autres contraintes
- clarté : les exigences doivent être compréhensibles par toutes les parties prenantes

Cahier des charges

- le SRS constitue la base d'un cahier des charges
- appel d'offres vers des prestataires extérieurs → sections ad-hoc supplémentaires
 - présentation détaillée du métier et de l'entreprise
 - historique, mission, valeurs, etc.
 - structure organisationnelle
 - produits et services
 - etc.
 - modalités de l'appel d'offres
 - critères d'éligibilité
 - échéances
 - modalités d'évaluation
 - clauses légales, responsabilités
 - etc.
 - canevas de réponse
 - rubriques à compléter par le soumissionnaire

Processus d'appel d'offres

- côté client :



- côté fournisseur :

1. analyse de faisabilité sur base des exigences, des produits disponibles et de l'expérience
2. prise de décision de remettre une offre ou non en considérant : l'étude de faisabilité, les options stratégiques, la disponibilité des équipes, etc.
3. rédaction d'une offre comprenant une réponse technique et une estimation des coûts
4. négociation avec le client

Thierry Van den Berghe

287

Exemple de contenu de l'appel d'offres

Contenu de l'offre

La proposition comprendra :

1. une brève présentation du fournisseur et de ses compétences ;
2. l'architecture du système proposé et une présentation des technologies mises en œuvre ;
3. une description du processus de développement pressenti, y compris la gestion des risques et de la qualité ;
4. un phasage du projet (graphe Gantt) ;
5. une estimation complète des coûts de réalisation du système, y compris l'analyse détaillée, la conception, l'implémentation, les tests, la mise en production, la formation, l'assistance au démarrage et le référencement ;
6. le coût d'éventuels logiciels de base ;
7. le coût du matériel ;
8. les modalités de garantie et de maintenance ;
9. la liste des éventuelles références dans des domaines similaires à celui du présent cahier des charges ;
10. les CV des membres de l'équipe affectée du projet.

288

Plan



- assurance qualité
- révision des spécifications

Thierry Van den Berghe

Validation des exigences

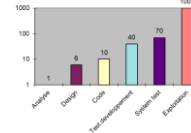
- vérification de la qualité des exigences
- résolution de défauts comme :
 - le manque de correspondances avec les besoins réels
 - des omissions
 - des inconsistances
- pour aller plus loin : cf. tests

Thierry Van den Berghe

290

Assurance qualité

- consiste à détecter, documenter, analyser les défauts, puis à prendre des actions correctrices
 - la recherche de qualité est permanente et elle mérite des tâches dédiées, planifiées et budgétées
 - rappel : les défauts des phases hautes du cycle de développement ont des impacts importants sur les phases en amont



A éviter (1)

- ambiguïtés
 - énoncé relatif soumis à interprétation
 - "etc.", "ainsi de suite", cas particulier → signes d'ambiguïté
 - ex. : les étudiants doivent s'inscrire à 5 cours par session
- exigences multiples
 - une seule exigence par énoncé
 - "et", "ou", "ainsi que" → signes de mélange
 - ex. : un client peut ajouter un article à son panier ou le supprimer
- clauses échappatoires
 - énoncés couvrant plusieurs cas de figure
 - si possible : décomposer les cas
 - "si", "quand", "sécussat" → signes d'échappement
 - ex. : un étudiant doit se présenter à l'examen excepté en cas de mal

Thierry Van den Bergh

A éviter (2)

- énoncés liés aux exigences techniques
 - relève des phases en amont
 - termes techniques, références au matériel → signes de test
 - ex. : les données complètes doivent être stockées dans le c
- spéculations et suggestions
 - exigences qui pourraient être demandées sans certitude
 - "peut-être", "pourrait", "normalement", "souvent" → signes c
 - ex. : normalement, un client doit régler sa facture avant exp
- manque de précision
 - expression de buts ou d'attributs de qualité difficilement me
 - "flexible", "le plus vite possible", "environ" → signes d'impr
 - ex. : un employé souffrant doit prévenir son employeur le pl possible

Thierry Van den Bergh

Qualités du SRS

- complétude : couverture complète, suffisamment détaillée pour permettre un développement qui répond aux objectifs fixés
- cohérence : les exigences doivent être compatibles entre elles
- précision : la formulation ne doit pas être ambiguë ou laisser de l'espace à l'interprétation
- pertinence : contribution à l'atteinte d'objectifs uniquement, en évitant de traiter la solution technique
- faisabilité : les exigences doivent être réalisables moyennant les capacités des TIC, les ressources disponibles, le délai et les autres contraintes
- clarté : les exigences doivent être compréhensibles par toutes les parties prenantes

Thierry Van den Bergh

291

Révision par des tiers

- inspection du SRS par des acteurs autres que les rédacteurs
 - technique applicable à toutes les formes d'exigences
 - efficacité démontrée mais consommateur de ressources
- inspecteurs : indépendants des auteurs du SRS
 - pas de relation d'évaluation entre les personnes
 - focus sur les exigences et pas sur les personnes ni sur les opinions
 - idéalement plusieurs profils : 1 utilisateur, 1 expert du domaine et 1 développeur
- l'inspection peut être :
 - libre, sous l'entière responsabilité de l'inspecteur
 - guidée par des check lists
 - guidée par un processus et des rôles établis
 - quelles tâches et quels acteurs en charge de ces tâches?

Thierry Van den Bergh

292

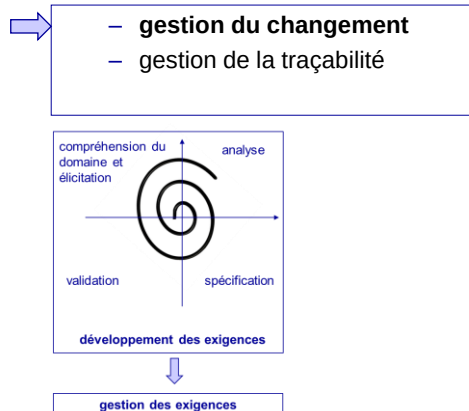
Exemple de check lists

Functional requirement checklist for Requirement ID#:		
F1	Is written in clear, unambiguous, non-technical language.	
F2	Is reasonable and plausible.	
F3	Contains no imprecise terms such as <i>some, sometimes, often, usually, ordinarily, most, mostly</i> .	
F4	The use of absolute terms such as <i>always, every, all, none, and never</i> is realistic.	
F5	The person or system taking the action is clearly understood. (Watch for passive voice.)	
F6	Enumerated list items are stated or obvious. List endings such as <i>etc., and so forth, and so on</i> have obvious meaning to the reader.	
F7	Can be tested to determine whether the requirement is met.	
F8	Contains no implementation details.	
F9	Is consistent with other requirements in this document and with the stated goals of the system.	
F10	Is not redundant with another requirements in this document.	
F11	For and input requirement: The source, format, and range of input values are specified. Reasonableness and range of values edits are stated.	
F12	For and output requirement: The frequency, destination, sorted order, restrictions (filters), output data format, and calculations are specified.	

Requirements Quality

- Does each requirement avoid conflicts with other requirements?
- Does each requirement have a priority? Are the priorities reasonable? (e.g. essential program functionality should not be on a low priority)
- Do the requirements avoid specifying design?
- Are the requirements at a fairly consistent level of detail? Should any requirement be specified in more detail? Should any requirement be specified in less detail?
- Are the requirements clear enough to be turned over to an independent group for implementation and still be understood?
- Is each requirement testable? Will it be possible for independent testing to determine whether each requirement has been satisfied?

Plan



Contexte des organisations

facteurs environnementaux

- globalisation
- conditions de marché
 - concurrence forte
 - automatisation de la relation client
 - vente en ligne
- demande client
 - personnalisation du produit
 - faible loyauté (churn)
 - rapidité, diversité, qualité
- régulation
 - Corporate Social Responsibility
 - certification qualité
 - prescrits légaux (SOX, etc.)

pressions

opportunités

réponse des organisations

- stratégie
- agilité
- partenariats, outsourcing
- productivité
- nouveaux business models
- CRM
- automatisation

- le fonctionnement des organisations est sans cesse revu
- un système est souvent obsolète dès son déploiement

Un terrain mouvant

- les exigences sont intrinsèquement instables et incomplètes
 - → le changement est inhérent aux projets TIC
- les causes de changements sont nombreuses :
 - changements dans l'environnement, nouveaux objectifs métier
 - changements dans les modes d'utilisation
 - ex. : demande de nouvelles navigations, rééquilibrage de contenu dans les écrans, nouveaux terminaux, etc.
- le changement intervient à tout moment
 - lors du développement (d'une itération à l'autre)
 - exigences, conception, implémentation
 - après le déploiement

Plusieurs types de changements

- **correction** : changement pour éliminer un défaut
 - ex. : éliminer les erreurs de programmation
- **amélioration** : changement pour perfectionner un système déjà fonctionnel
 - ex. : rajouter des éléments de navigation directs entre les écrans selon les usages fréquents des utilisateurs
- **adaptation** : changement du système en place pour répondre aux évolutions imposées au système
 - ex. : acter le consentement des prospect lorsqu'ils communiquent leur coordonnées pour répondre au GDPR
- **extension** : changement pour compléter le système par l'ajout de nouvelles fonctionnalités
 - ex. : développement d'un module de suivi de commande d'un e-shop

Thierry Van den Berghe

297

Principales causes de changements

Cause	Changement	Moment
erreurs	correction	à tout moment
nouvelles fonctionnalités	extension	après le déploiement
meilleure compréhension de l'utilisateur	correction, extension	exigences, après le déploiement
amélioration de la qualité	amélioration	après le déploiement
changement des priorités, du planning ou des contraintes de coût	adaptation	exigences, conception, implémentation
<i>changements liés à l'environnement</i>		
nouvelle réglementation	adaptation	après le déploiement
nouvelle technologie	adaptation, extension	après le déploiement
nouvelles opportunités	extension	après le déploiement
nouvelle catégorie d'utilisateurs	adaptation, extension	exigences, après le déploiement

Thierry Van den Berghe

298

Gestion des exigences

1. répondre au changement

- intégration des changements d'environnement
- ex. : nouveaux processus d'affaires
- intégration des changements du comportement utilisateur

2. répercuter les changements à tous les niveaux

- ex. : changement des exigences du domaine
 - changement des exigences utilisateur
 - changement des spécifications techniques
 - nouvelle version logicielle

3. gérer les interdépendances

- traçabilité

Bonnes pratiques

• mise en place d'un processus du changement des exigences

- exemples de pratiques :
 - adopter un cycle de développement adapté (cf. Agile)
 - évaluer les exigences par rapport à leur stabilité estimée
 - définir des procédures de modification d'exigence
 - définir des procédures de propagation du changement

• instaurer une culture du changement

- vision claire et partagée
- engagement de toutes les parties prenantes
- lutter contre la résistance naturelle au changement

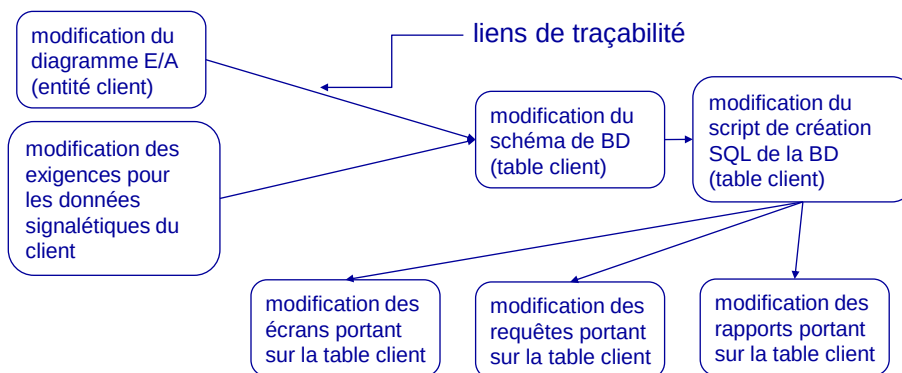
• anticiper les réponses aux changements dans les exigences mêmes

- penser en termes de flexibilité

Exemple d'anticipation du changement

- le président du jury de délibération doit pouvoir valider des crédits d'une UE pour un étudiant qui a obtenu 9/20 à cette UE
-
- le président du jury de délibération doit pouvoir valider les crédits d'une UE pour un étudiant qui a obtenu la cote minimale à cette UE
 - l'administrateur doit pouvoir paramétrer la cote minimale à obtenir pour valider les crédits d'une UE

Exemple de propagation d'un changement



Plan

- Ingénierie logicielle
 - Ingénierie des exigences (IE)
 - Domaine et élicitation
 - Analyse des exigences
 - Spécification des exigences
 - Qualité des exigences
 - **Gestion des exigences**
 - Buts et exigences
 - Cas d'utilisation
 - Diagramme d'activité
 - Introduction à BPMN
 - Tests
- 
- gestion du changement
 - **gestion de la traçabilité**

Thierry Van den Berghe

Gérer la traçabilité

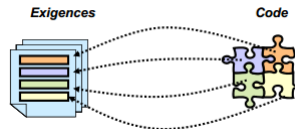
- processus de **définition, d'exploitation** et de **maintenance** des liens de traçabilité entre les exigences
 - IEEE 830 : "An SRS is traceable if the origin of each of its requirements is clear and if it facilitates the referencing of each requirement in future development or enhancement documentation. "
- objectifs :
 - pouvoir identifier l'**impact** d'un changement d'une exigence sur le reste du système
 - mieux détecter les **inconsistances** et **manquements**, comme des exigences non liées à des spécifications techniques
 - mieux comprendre la **motivation** des exigences à travers leurs interdépendances
- coûts :
 - important si le niveau de détail est fin
 - un éventuel retour vient assez tard dans le projet

Thierry Van den Berghe

304

Types de traçabilité

- liens de traçabilité verticaux et horizontaux
 - verticaux : liens entre des éléments de phases successives du cycle de développement
 - ex. : les exigences et le schéma de base de données
 - horizontaux : liens entre des éléments d'une même phase
 - ex. : un client doit pouvoir créer une nouvelle adresse de livraison
un employé doit pouvoir expédier une commande à l'adresse de livraison du client
- liens de traçabilité horizontaux : vers l'avant et vers l'arrière
 - source → cible : quel usage de la source
 - source ← cible : d'où vient la cible, quelle est sa justification
 - ex. :

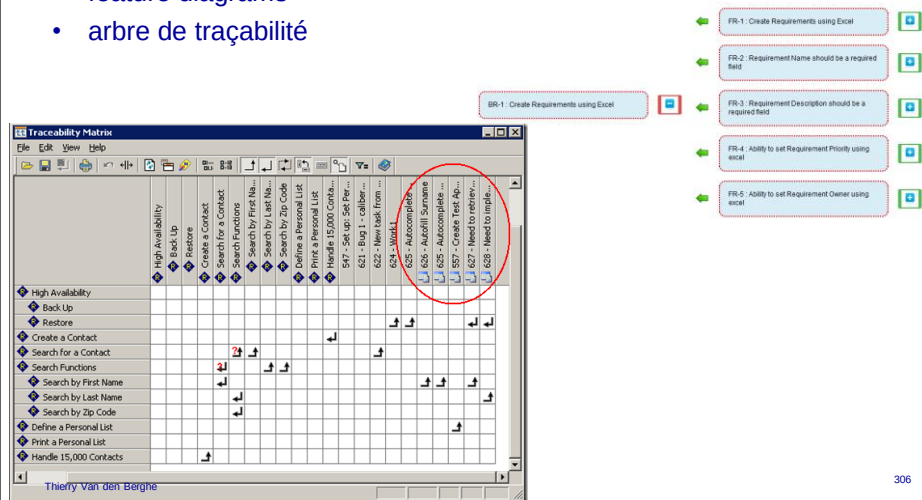


Thierry Van den Berghe

305

Représentation des liens de traçabilité

- représentation matricielle
- feature diagrams
- arbre de traçabilité



306

Outils d'ingénierie des exigences

- **facilités**
 - gestion de projet
 - répertoire des exigences
 - gestion de la traçabilité
 - travail collaboratif
 - interopérabilité
 - etc.
- **exemples**
 - IBM Requisite Pro (Web) – Rational Doors
 - Enterprise Architect - http://www.sparxsystems.com.au/platforms/requirements_management.html
 - TraceCloud - <http://www.tracecloud.com>
 - Jira

Thierry Van den Berghe

307



ICHEC
BRUSSELS MANAGEMENT SCHOOL

21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation**
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Plan



- cas d'utilisation
- acteur
- séquences d'événements
- heuristiques de construction
- exemple
- exercices

Thierry Van den Berghe

Cas d'utilisation : exemple introductif

Quelles sont les utilisations d'un système?



téléphoner

envoyer
des SMS

gérer un
agenda

prendre
des photos

etc.



composer un texte

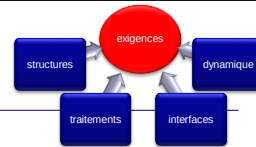
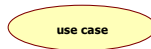
choisir un destinataire

ou introduire le numéro
du destinataire

confirmer l'envoi

310

Cas d'utilisation



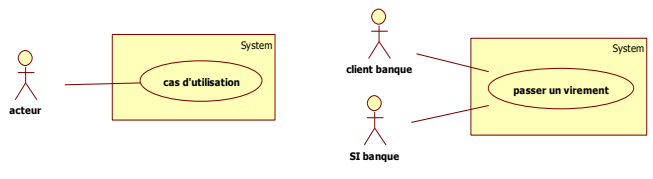
- description du comportement du système du point de vue de l'utilisateur
 - décrit le comportement du système dans son ensemble (boite noire)
 - pas de spécification technique liée à l'implémentation
- un cas d'utilisation = une fonctionnalité qui apporte de la **valeur** à l'utilisateur
 - généralement, l'utilisateur déclenche l'exécution d'un cas d'utilisation
- les cas sont regroupés dans un diagramme de cas d'utilisation
 - les cas sont repris dans le système, qui est lui-même délimité de son environnement

211

Acteur



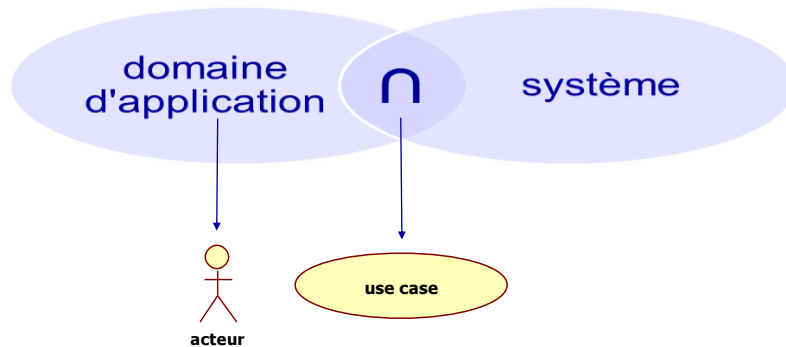
- représente un rôle joué par une entité qui interagit avec le système
 - stimule le système en vue de l'exécution d'un traitement
 - envoie et/ou reçoit de l'information vers/du système
 - participe au déroulement du cas d'utilisation
- est extérieur au système et donc situé dans son environnement
- utilise le système



- acteur principal <> acteur secondaire

212

use case = exigence

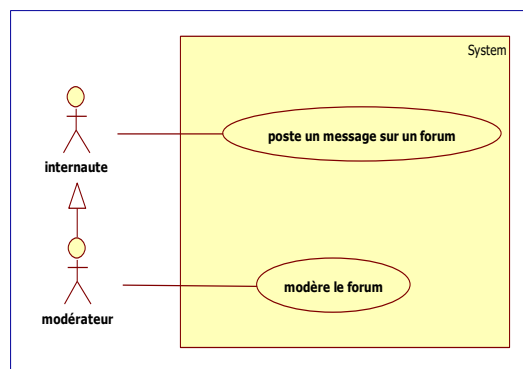


Thierry Van den Berghe

313

Hiérarchie d'acteurs

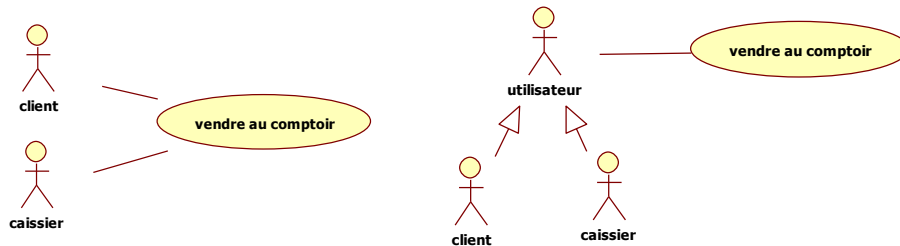
- peut être reprise dans des structures de généralisation / spécialisation
 - un acteur spécialisé hérite de toutes les propriétés de l'acteur qui le généralise
 - un acteur spécialisé possède des propriétés spécifiques propres (non héritées)
 - un acteur peut être spécialisé par plusieurs autres acteurs



314

Différentes situations...

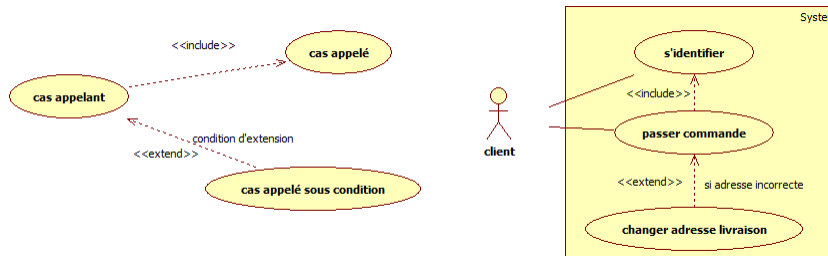
- le client et le caissier doivent participer tous deux à la vente
- soit le vendeur soit le client seul (self scanning) peut faire une vente



315

Structuration des cas d'utilisation

- un cas peut factoriser un comportement qui se répète dans plusieurs autres cas
 - relation `include` : un cas fait toujours appel à un autre cas
 - relation `extend` : un cas fait appel à un autre cas uniquement si une condition est vérifiée
 - un cas appelé peut être exécuté indépendamment de son appelant



316

Détails d'un cas d'utilisation

- première description à partir de rubriques standard
 - liste de rubriques variable
- exemples de rubriques
 - **Nom du cas:** recherche d'ouvrages dans la bibliothèque
 - **Acteurs:** lecteur
 - **Objectif:** sélectionner des ouvrages dans le catalogue sur base de critères de recherche
 - **Description:** Un lecteur introduit des critères de recherche, comme un mot dans le titre ou un auteur, puis le système lui présente les ouvrages correspondants et les nouveautés.
- détail des interactions avec des séquences d'événements

217

Exemple de canevas

Context of use: Buyer issues request directly to our company, expects goods shipped and to be billed.

Scope: Company *Level:* Summary
Preconditions: We know Buyer, their address, etc.
Success End Condition: Buyer has goods, we have money for the goods.
Failed End Condition: We have not sent the goods, Buyer has not spent the money.
Primary Actor: Buyer, any agent (or computer) acting for the customer
Trigger: purchase request comes in.

Main Success Scenario
1. Buyer calls in with a purchase request.
2. Company captures buyer's name, address, requested goods, etc.
3. Company gives buyer information on goods, prices, delivery dates, etc.
4. Buyer signs for order.
5. Company creates order.
6. Company ships ordered goods to buyer.
7. Company ships invoice to buyer.
8. Buyers pays invoice.

Extensions
3a. Company is out of one of the ordered items:
3a1. Renegotiate order.

4a. Buyer pays directly with credit card:
4a1. Take payment by credit card (use case 44)
8a. Buyer returns goods:
8a. Handle returned goods (use case 105)

Technology Variations
1. Buyer may use
 phone in,
 fax in,
 use web order form,
 electronic interchange
8. Buyer may pay by
 cash or money order
 check
 credit card

Project Information			
Priority	Release Due	Response time	Rate of use
Top	Rel. 1.0	5 minutes for order, 45 days until paid	200/day

Alistair Cockburn, Writing Effective
Use Cases, Addison-Wesley, 2000

Thierry Van den Berghie

Séquences d'événements

- séquences de stimuli de l'acteur et de réponses du système
- séquences alternatives
 - branchement vers une séquence parmi plusieurs en fonction du résultat d'un test
- séquences conditionnelles
 - exécution d'une séquence uniquement si une condition est satisfaite
- les séquences alternatives et conditionnelles "complètent" la séquence standard d'un cas d'utilisation

319

Exemple de séquence d'événements : recherche d'ouvrages dans une bibliothèque

Action du lecteur	Réponse du système
1. Le cas est initié lorsqu'un lecteur appelle la page Recherche du site Web.	2. Le système affiche un formulaire de critères de recherche et la liste des nouveautés.
3. Le lecteur indique un nom d'auteur et/ou un mot du le titre.	4. Si des ouvrages sont trouvés, Le système affiche la liste synthétique des ouvrages correspondants à la recherche. 4'. Si aucun ouvrage n'est trouvé, le système affiche à nouveau l'écran de recherche avec le message : 'aucun ouvrage trouvé'.
5. Le lecteur clique sur la cote d'un des ouvrages présentés.	6. Un écran reprenant le détail de l'ouvrage sélectionné est affiché.

alternatives

320

Évaluation des cas d'utilisation

- technique **simple**
- vue **abstraite** du système
- permet une **découpe** des fonctionnalités
- description **informelle** des besoins
 - ➔ recours à d'autres langages plus formels

221

Principes de construction



- formalisme simple et nombre de concepts limité, mais erreurs de conception fréquentes
- démarche créative et incertaine car il s'agit de la première description du système
- nécessité de trouver un consensus fonctionnel parmi les parties prenantes
 - évaluation de la valeur, de la faisabilité et du degré d'innovation des fonctionnalités retenues
- principes de base :
 - construction progressive du diagramme de cas en plusieurs itérations
 - une itération complète le diagramme courant ou le détaille

222

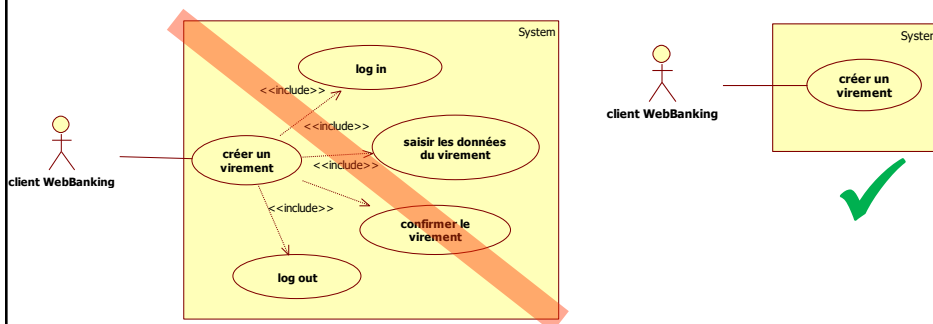
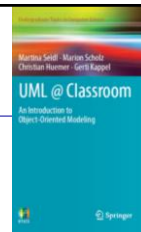
Exemple de démarche de construction

1. identifier les acteurs principaux
2. identifier les cas des acteurs principaux et les décrire selon les rubriques standard
3. identifier les acteurs secondaires
4. identifier les cas des acteurs secondaires et les décrire selon les rubriques standard
5. détailler les cas avec les séquences standard d'événements
6. détailler les séquences alternatives et conditionnelles
7. structurer les cas d'utilisation en factorisant les comportements communs

323

Erreurs à éviter (1)

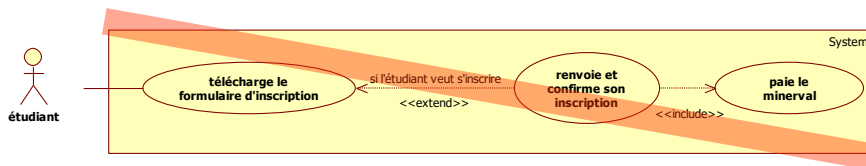
- éviter la décomposition de la fonctionnalité
 - les différentes étapes d'un cas d'utilisation font partie du cas « principal », et ne justifient pas la création d'autres cas qui ne seront appelés que par le cas principal



324

Erreurs à éviter (2)

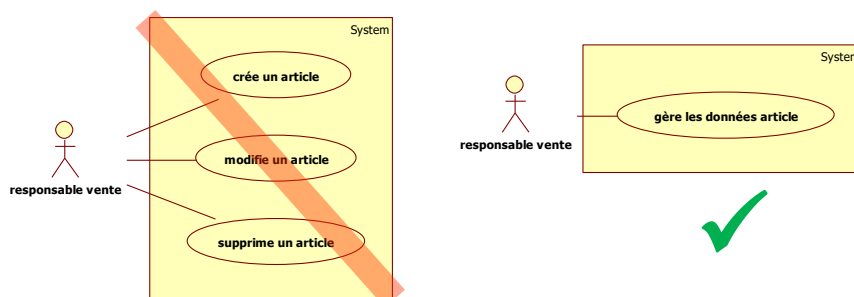
- éviter la modélisation de workflows
 - ne pas représenter l'enchaînement des opérations dans le temps



325

Erreurs à éviter (3)

- ne pas éclater plusieurs cas d'utilisation qui recouvrent le même objectif et définir un seul cas qui recouvre toute la fonctionnalité



326

US (user story) versus UC (use case)

- **intention**
 - US et UC : décrire le comportement du système d'un point de vue fonctionnel
 - US : technique d'éllicitation privilégiée par les méthodes Agile
 - UC : technique d'éllicitation préconisée par le Unified Process
- **niveau de détail**
 - US moins détaillé que UC
 - US complété par des échanges directs utilisateurs/informaticiens
- **forme**
 - UC graphique + modèle structuré de séquences
 - US : langage naturel structuré + détails en langage naturel libre (souvent oral)
- **modalités**
 - UC : encourage la formalisation écrite et la communication asynchrone
 - UC : rarement rédigé par les utilisateurs, plutôt écrit par le Business Analyst
 - US : encourage le travail collaboratif (Agile) et la communication synchrone

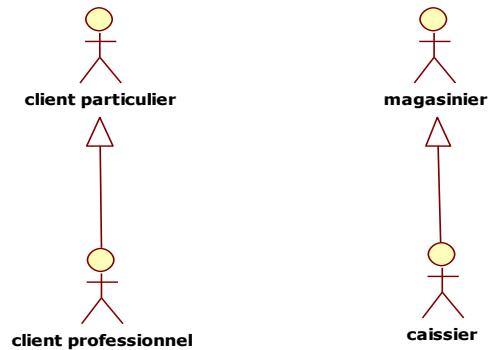
327

Exemple : énoncé simplifié

- Un magasin de vente d'articles pour animaux vend de la nourriture et des accessoires à des clients privés ou professionnels. Deux modes de vente sont proposés : une vente au comptoir et une commande Web pour les professionnels uniquement.
- Les magasiniers se chargent de la préparation des commandes et de la facturation. Ils réassortent le stock et tiennent la caisse.
- De temps en temps, des délégués commerciaux viennent en magasin pour organiser les rayons.

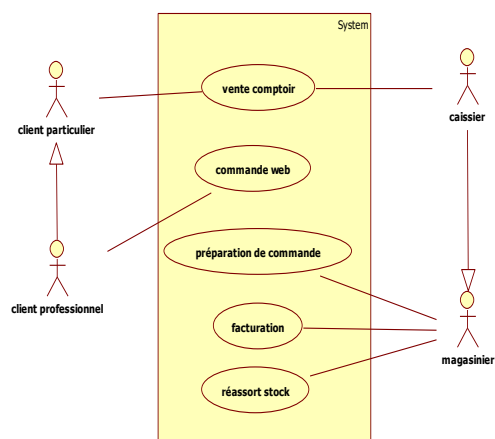
328

Exemple : 1. identifier les acteurs principaux



329

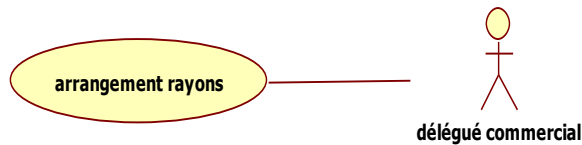
Exemple : 2. identifier les cas principaux



- Nom du cas:** vente comptoir
- Acteurs:** caissier
- Objectif:** saisir une vente et son paiement
- Description:** Un client se présente à la caisse pour régler ses achats. Le caissier enregistre les articles choisis et encaisse le paiement du client. Finalement, le client quitte la caisse avec ses articles.

330

Exemple : 3-4. acteurs et cas secondaires



331

Exemple : 5. détailler les séquences standard (cas vente comptoir)

Action du caissier	Réponse du système
1. Le caissier démarre une nouvelle vente.	2. Le système crée une nouvelle transaction de vente.
3. Le caissier scanne le code barre de chaque article que le client présente au comptoir.	4. Le système détermine le prix de l'article et rajoute l'information relative à l'article dans la transaction de vente en cours.
6. Le caissier signale que la saisie des articles est terminée.	5. La quantité et le prix total de l'article courant sont affichés.
	7. Le système calcule et affiche le total de la vente.
ne concerne pas directement le système	
8. Le client donne les espèces au caissier (d'un montant éventuellement supérieur au montant de la vente). Le caissier enregistre le montant payé.	9. Le système affiche le solde à rendre au client.
	10. Un ticket de caisse est imprimé. La vente est clôturée.

Happy flow

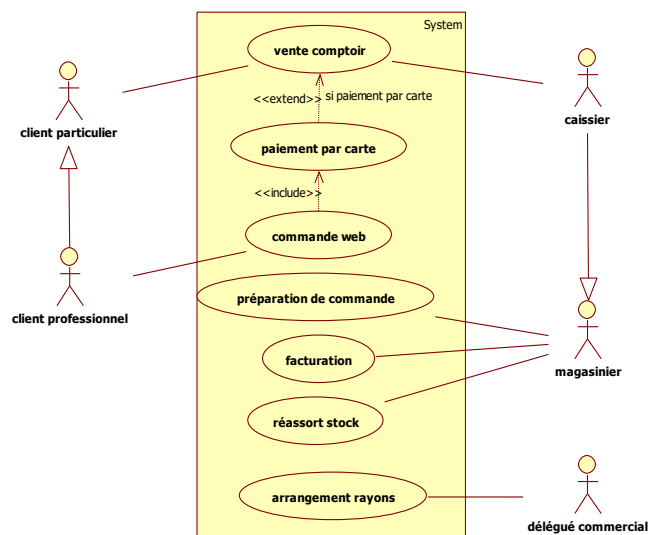
332

Exemple : 6. détailler séquences alternatives et conditionnelles (cas vente comptoir)

- Conditionnelle
 - 3.2 Si plusieurs exemplaires du même article sont présentés, le caissier peut introduire une quantité achetée.
- Alternative
 - Si le client paye en espèce
 - 8. Le client donne les espèces au caissier (d'un montant éventuellement supérieur au le montant de la vente). Le caissier enregistre le montant payé.
 - 9. Le système affiche le solde à rendre au client.
 - Si le client paye par carte
 - 8'. Le client introduit son code
 - 9'. Le système communique avec l'organisme de paiement et débite la carte

333

Exemple : 7. structurer les cas d'utilisation



334



21BA030 - Ingénierie des exigences

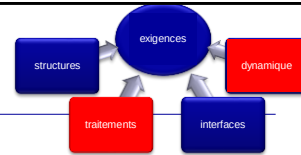
- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité**
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Plan



- éléments de base (UML 1.*)
- extensions UML 2.*

Diagramme d'activité



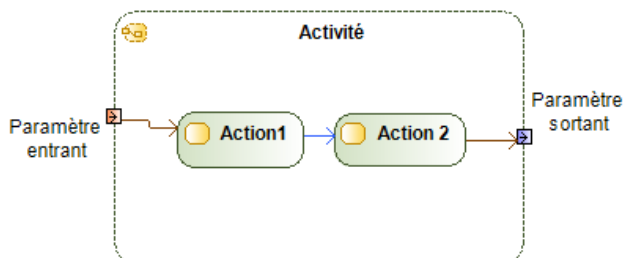
- **modèle des traitements et de leur dynamique**
 - représente des actions et leur séquence d'exécution dans le temps
- **exemples d'applications**
 - spécifier le comportement d'un SI
 - décrire un processus métier
 - compléter la description des cas d'utilisation
- **références**
 - UML @ Classroom
 - <https://www.visual-paradigm.com/VPGallery/diagrams/Activity.html>
 - <http://www.uml-diagrams.org/activity-diagrams-reference.html>

227

Activité



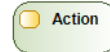
- **description d'un comportement vu par l'utilisateur**
- **une activité = graphe orienté**
 - nœud = action ou une activité incluse
 - arcs = flux de contrôle ou flux d'objet
 - option : paramètres entrants ou sortants



Thierry Van den Berghe

338

Action



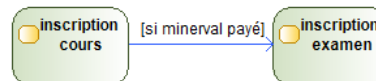
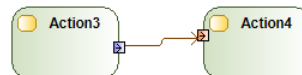
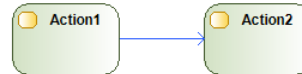
- action = unité fondamentale de comportement
 - traite des entrées pour produire des sorties
 - modifie généralement l'état du système
 - ex. : création d'une nouvelle commande
 - atomique → indivisible
- types d'actions prédéfinis (voir plus loin)
 - Event-based actions
 - Call behavior actions

Détails des actions

- le fonctionnement des actions doit être explicité
- mais pas de règles pour décrire une action
 - description informelle en langage naturel
- exemple de formulation
 - impact de l'action sur l'état des données
 - ex. : post-conditions, i.e., affirmations vérifiées après exécution de l'action
- exemple : action `confirmer une commande`
 - une nouvelle commande est créée et est reliée à un client
 - les quantités disponibles en stocks sont diminuées à concurrence des quantités commandées
 - la liste des commandes à préparer est mise à jour

Arc

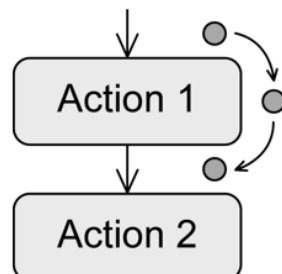
- connecte les activités ou les actions entre elles
- détermine l'ordre d'exécution
- arc de flux de contrôle
 - lorsque l'exécution de l'élément source de l'arc se termine, alors l'exécution de l'élément cible de l'arc démarre
- arc de flux d'objet
 - rajoute un échange de données ou d'objet
 - exprime une dépendance d'entrée/sortie
- condition de garde (optionnelle)
 - doit être vérifiée pour activer le flux



Thierry Van den Berghe

Métaphore de jeton

- mécanisme virtuel de coordination pour décrire l'ordre d'exécution
 - une action qui reçoit un jeton peut s'exécuter
 - une action qui a terminé son exécution passe le jeton à l'action suivante, ce qui déclenche son exécution
 - les conditions de garde peuvent empêcher le passage du jeton



Thierry Van den Berghe

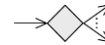
342

Nœuds de début et de fin d'activité

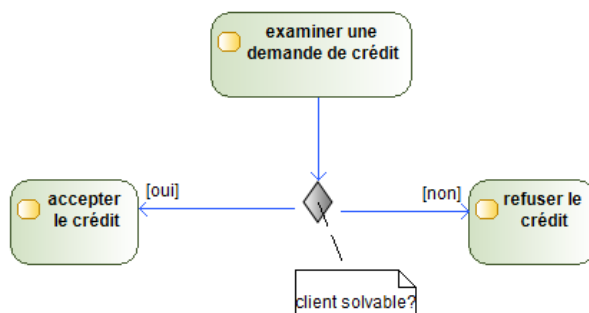
- **nœud initial** 
 - démarre l'activité
 - fournit des jetons à tous les flux sortants du nœud
 - conserve les jetons jusqu'au moment où les nœuds suivants les acceptent
 - plusieurs nœuds initiaux modélisent la concurrence (!)
- **nœud de fin d'activité** 
 - termine tous les flux de l'activité
 - le premier jeton qui atteint un nœud de fin termine toute l'activité
 - sous-branches concurrentes incluses
 - les autres jetons sont détruits
- **nœud de fin de flot** 
 - termine une sous-branche de l'activité
 - n'affecte pas les autres jetons

242

Nœud de décision (choice)



- **définit des branchements alternatifs**
 - aiguillage du jeton entrant
 - une seule voie de sortie suivie par le jeton
- **arcs sortants avec des conditions de gardes**
 - conditions mutuellement exclusives

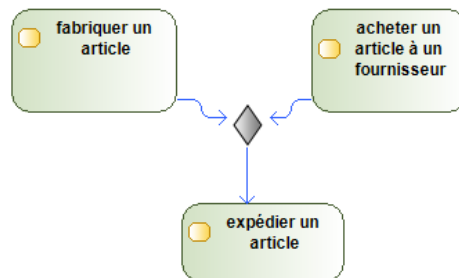


244

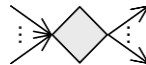
Nœud de fusion (merge)



- fusionne plusieurs sous-branches ensemble
 - passe le premier jeton entrant vers le nœud suivant



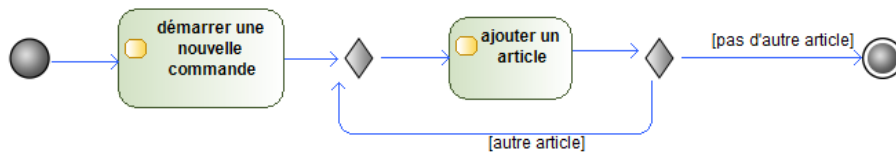
- décision et fusion peuvent être combinées



Thierry Van den Berghe

345

Exemple : boucle



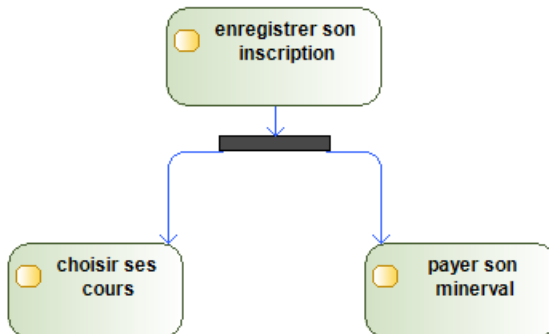
Thierry Van den Berghe

346

Nœud de bifurcation (fork)



- éclate une sous-branche en plusieurs
 - représente le parallélisme
- duplication du jeton entrant pour chaque sous-branche sortante



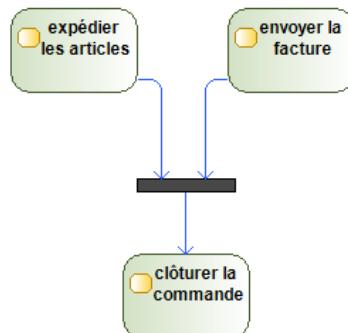
Thierry Van den Berghe

347

Nœud de jointure (join)



- fusionne plusieurs sous-branches entrantes
 - attend tous les jetons des sous-branches entrantes
 - fusionne tous les jetons entrants en un seul sortant



- bifurcation et jointure peuvent être combinées

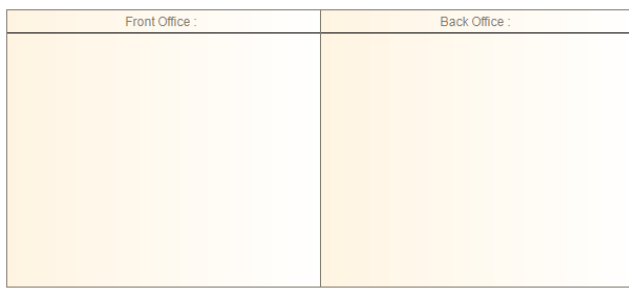


Thierry Van den Berghe

348

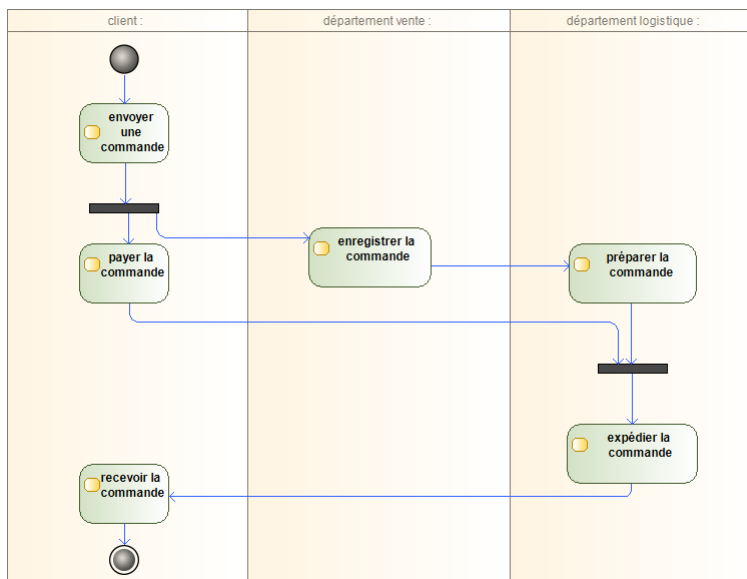
Partition

- définit le contexte de l'exécution
 - qui exécute l'action?
 - où l'action est-elle réalisée?



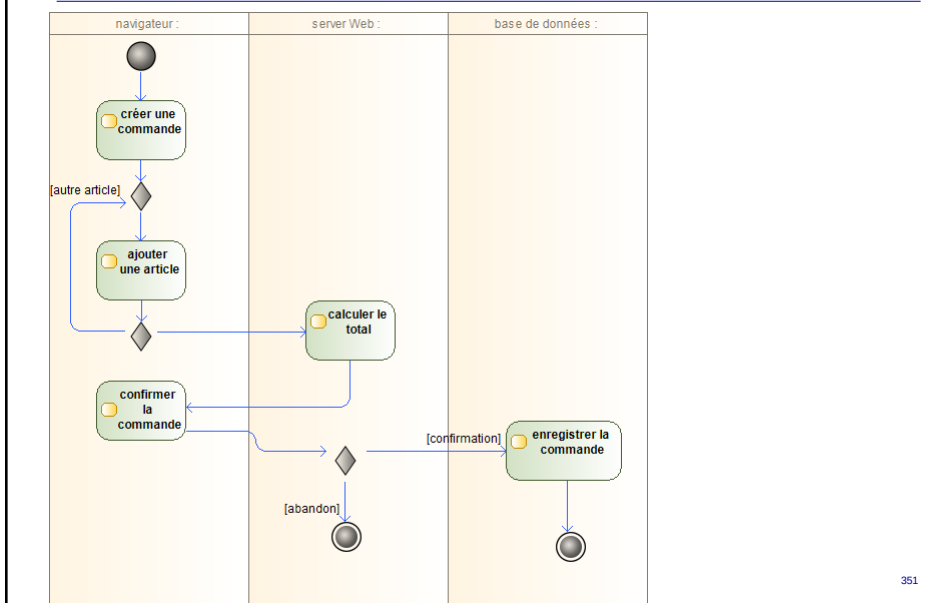
349

Exemple (1)



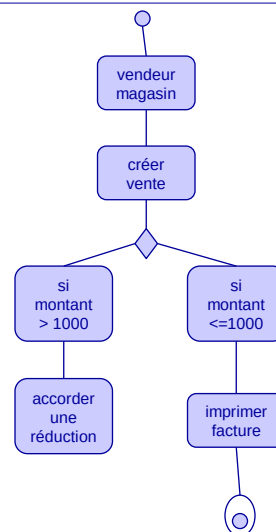
350

Exemple (2)



Erreurs fréquentes

- oubli du sens des flux
- action sans flux sortant
- condition de garde présentée comme une action
- acteur représenté comme action



Aide de l'IA : quelles sont les erreurs dans ce diagramme d'activité UML (grok.com – 02/06/2025)

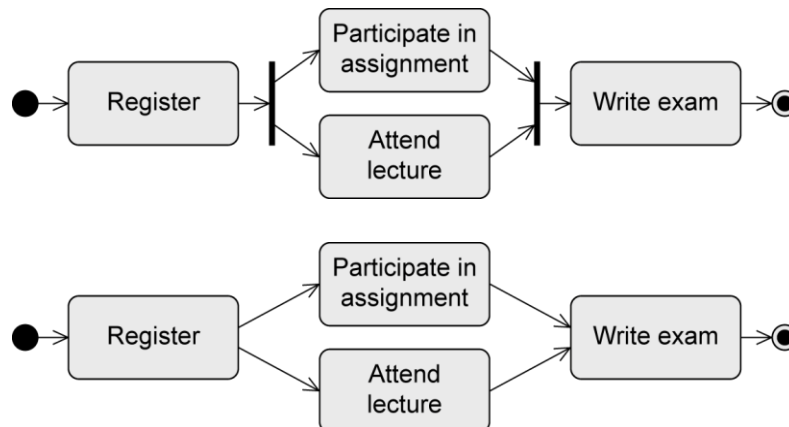
- 1. ****Manque de clarté dans les décisions**** : La condition "si montant > 1000" et "si montant <= 1000" devrait idéalement inclure des gardes (guard conditions) explicites sur les flèches sortant du nœud de décision (losange). Actuellement, les conditions sont dans des nœuds séparés, ce qui n'est pas la norme. Les flèches devraient porter des étiquettes comme "[montant > 1000]" et "[montant <= 1000]".
- 2. ****Absence de flux de sortie complet**** : Après "accorder une réduction" et "imprimer facture", il n'y a pas de nœud final (bull) connecté correctement pour indiquer la fin des deux branches. Chaque branche devrait se terminer par un nœud final pour signifier la fin du processus.
- 3. ****Nœud initial ambigu**** : Le nœud initial (cercle plein) est connecté directement à "créer vente", mais il manque une action ou un événement clair pour démarrer le processus (par exemple, "recevoir commande" ou "ouvrir caisse").
- 4. ****Syntaxe et terminologie**** : Les termes comme "si montant > 1000" et "si montant <= 1000" sont en français, ce qui est acceptable, mais ils devraient être uniformément formatés comme des conditions UML (entre crochets). De plus, "créer vente" pourrait être plus précis (par exemple, "enregistrer vente").
- 5. ****Flux non synchronisé**** : Si les deux branches ("accorder une réduction" et "imprimer facture") doivent se rejoindre pour une étape ultérieure, un nœud de jonction (barre noire) est nécessaire, mais il est absent ici.



353

Parallélisme : 2 notations équivalentes

- ...mais la première est plus explicite

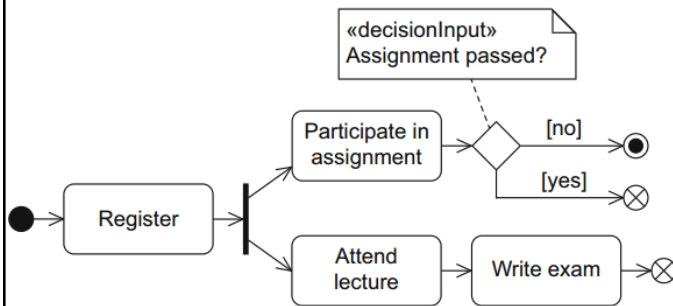


Thierry Van den Berghe

354

Exemple de parallélisme (1)

- "The successful participation in an assignment and the simultaneous attendance of the related lecture combined with taking the exam are two independent execution paths that each end with a flow final node. However, failing the assignment ends the entire activity, meaning that if a student has a negative assignment grade, the student can no longer take the final exam if the student has not done so already"

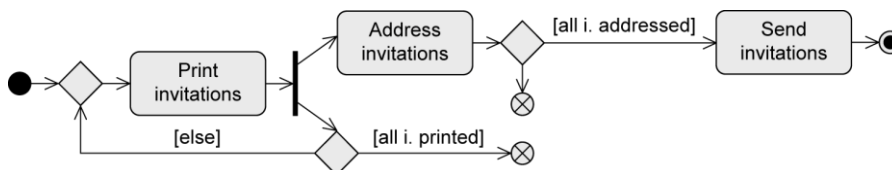


Thierry Van den Berghe

355

Exemple de parallélisme (2)

- While invitations are printed, already printed invitations are addressed.
- When all invitations are addressed, then the invitations are sent.



Thierry Van den Berghe

356

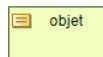
Plan



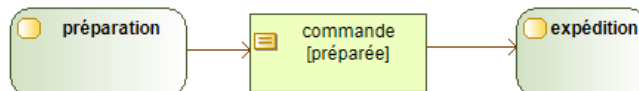
- éléments de base (UML 1.*)
- **extensions UML 2.***

Thierry Van den Berghe

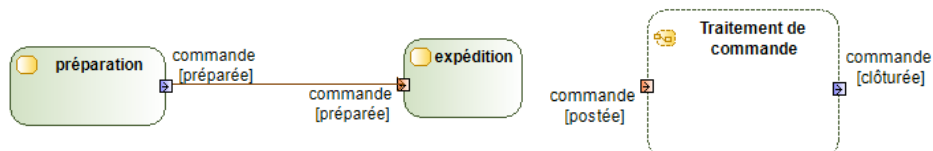
Nœud objet



- matérialise l'échange de données ou d'objets
 - source et cible d'un flux d'objet
 - optionnel : type et état de l'objet



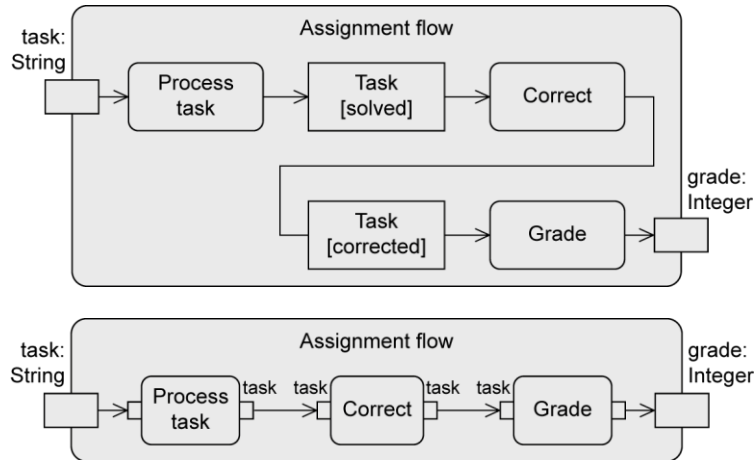
- notation alternative (action et activité) : objets comme paramètres



Thierry Van den Berghe

358

Exemples



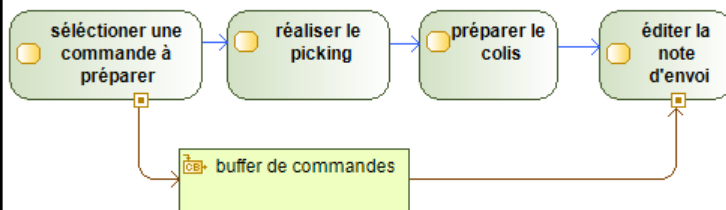
Thierry Van den Berghe

359

Buffer central



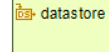
- pour stocker et restituer des objets (ou plus exactement des flux d'objets)
- la consommation d'un objet implique sa suppression dans le buffer
 - mémoire volatile



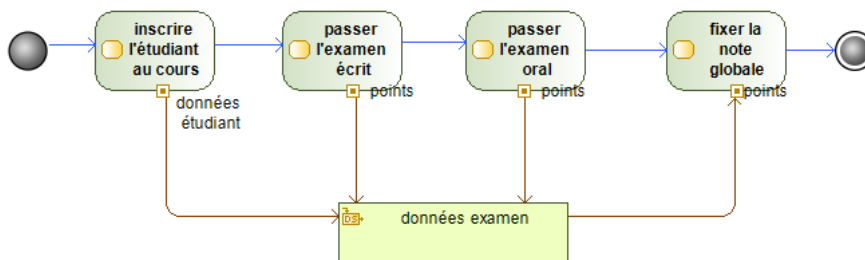
Thierry Van den Berghe

360

Entrepôt de données (Data Store)



- aussi pour stocker et restituer des objets
- mais les objets sont sauvegardés de manière permanente, indépendamment de leur consommation
 - mémoire persistante, comparable à une base de données

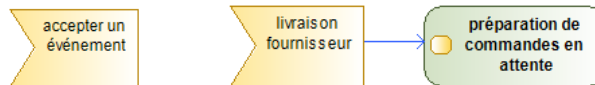


Thierry Van den Bergh

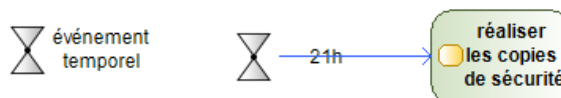
361

Actions événements (1)

- event-based action : permet la transmission de signaux...
 - types particuliers d'actions
- ... soit pour accepter des événements
 - action d'acceptation, qui implique l'attente de l'occurrence d'un événement



- action d'acceptation d'un événement temporel



Thierry Van den Bergh

362

Actions événements (2)

- ... soit pour envoyer des signaux vers un destinataire
 - souvent en combinaison avec l'acceptation d'un événement



Thierry Van den Berghe

363

Enchaînement asynchrone



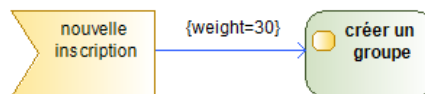
- après qu'une commande aie été préparée, un signal de demande de paiement est émis
- l'activité attends de recevoir le signal de confirmation de paiement pour continuer son exécution
 - enchaînement asynchrone, liés à des événements externes
- l'arc entre "demande de paiement" et "confirmation de paiement" signifie que "confirmation de paiement", soit l'attente d'un signal, ne peut pas s'exécuter sans la fin d'exécution de "demande de paiement"
 - sans flux entrant, une action d'acceptation s'exécute sans préalable

Thierry Van den Berghe

364

Poids de flux

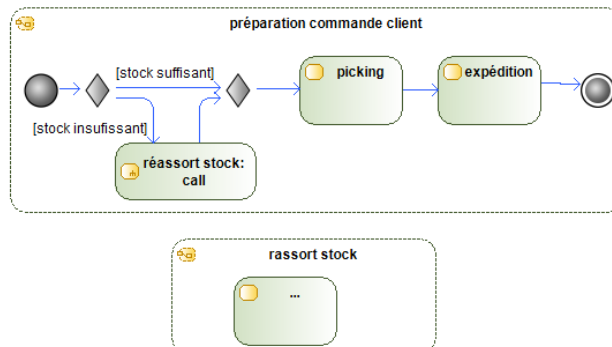
- indique qu'un nombre minimal de jetons doivent être présents pour exécuter l'action cible
- par défaut : poids = 1
- exemple : la création d'un groupe s'exécutera quand 30 occurrences de l'événement "nouvelle inscription" se seront produites



Action d'appel

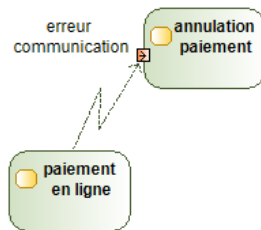


- call behavior action : appel d'une activité indépendante
 - décomposition du diagramme
 - réutilisation



Gestionnaire d'exception

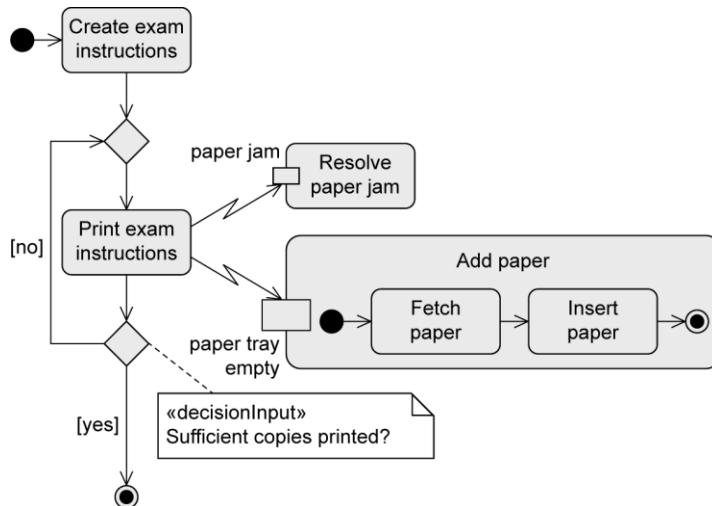
- définit le comportement du système en cas d'erreur
- le gestionnaire d'exception remplace l'action qui provoque l'erreur
 - tous les jetons de l'action sont supprimés
 - le flux d'exécution continue régulièrement après l'exécution du gestionnaire d'exception
 - comparable à la gestion des exceptions dans les langages de programmation



Thierry Van den Berghe

367

Exemple

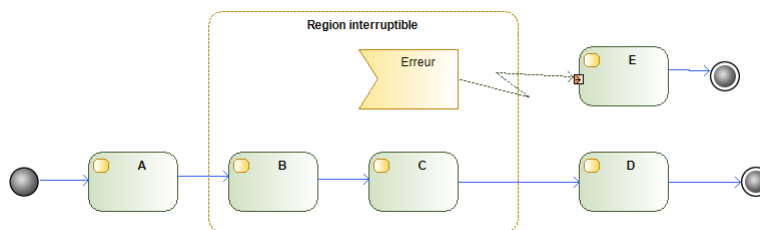


Thierry Van den Berghe

368

Région interruptible

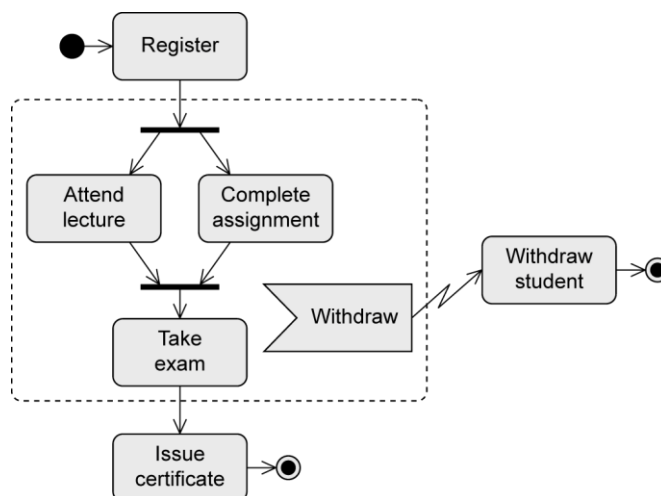
- décrit un groupe d'actions dont l'exécution est arrêtée lorsqu'un événement se produit
- ex. : si l'événement Erreur se produit lors de l'exécution de B ou C :
 - tous les jetons de la région interruptible sont supprimés
 - E est exécuté et D n'est pas exécuté (pas de retour à la fin de l'exécution de l'alternative en cas d'erreur)



Thierry Van den Berghe

369

Exemple (1)

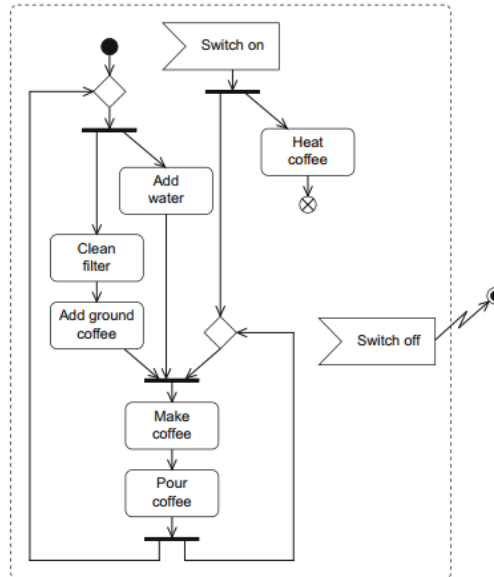


Thierry Van den Berghe

370

Exemple (2)

- cafetière

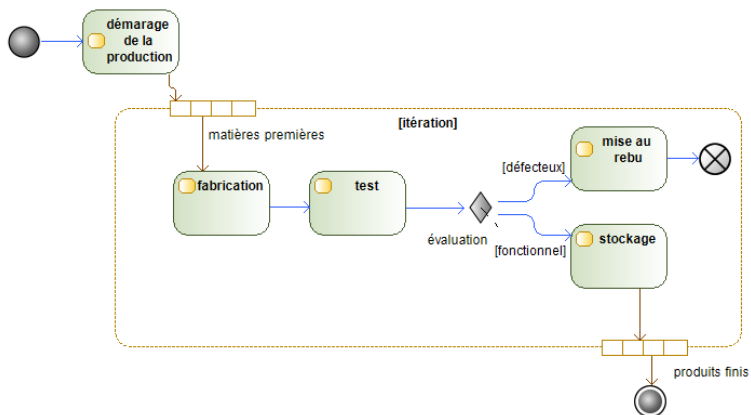


Thierry Van den Berghe

371

Zones d'expansion itératives

- décrit un groupe d'actions dont l'exécution est répétée



Thierry Van den Berghe

372



21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM**
- 7 Modélisation des processus avec BPMN
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Une approche parmi d'autres

- **rappel : multitude d'outils et d'approches à la disposition du Business Analyst**
 - ex. : BABOK recense 50 techniques d'éllicitation, de modélisation, etc.
 - ex. : nombreux langages de spécifications des exigences
 - BPMN : un incontournable historique
- **dans la suite : focus sur une approche pour traiter des processus d'affaires**
 - Business Process Management (BPM) : discipline centrée sur la supervision des processus d'affaires pour s'assurer de leur bon fonctionnement et les améliorer
 - Business Process Management Notation (BPMN) : notations pour modéliser un processus d'affaires
 - BPM et BPMN reprennent de nombreux principes de la Business Analysis
 - dans la suite : focus sur leurs spécificités

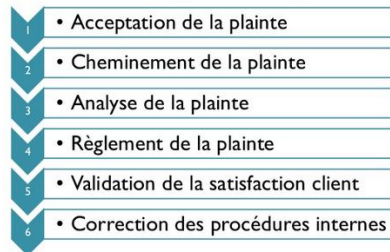
Processus : définition

- processus d'affaires :
 - ensemble d'événements, d'activités et de décisions connectés
 - qui impliquent des acteurs et des ressources,
 - qui apportent ensemble de la valeur à une organisation ou à ses clients
 - exemples : commande client (order-to-cash), appel d'offres (quote-to-order), résolution de problèmes (fault-to-resolution), règlement de plaintes (claim-to-settlement)



Thierry Van den Berghe

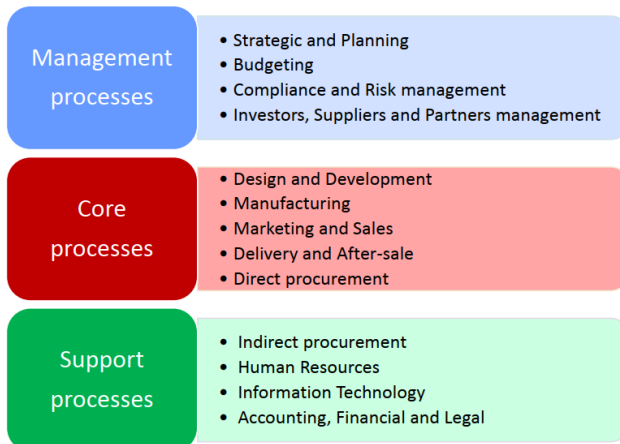
Processus de gestion de plainte



375

Processus : typologie

- processus de management : fixation d'une stratégie/politique
- processus métier : réalisation du produit/service
- processus de support : mise à disposition de ressources/services



376

Exemple : restaurant fast food

- management : stratégie, veille concurrentielle, gestion des implantations, etc.
- métier : approvisionnement, préparation des hamburgers, préparation d'une commande, marketing, vente comptoir, gestion de salle
- support : GRH, IT, comptabilité, etc.



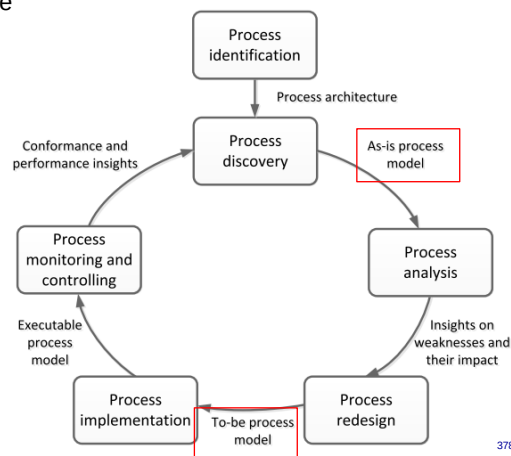
Thierry Van den Berghe

377

Business Process Management (BPM) : définition

- corpus de **principes**, de **méthodes** et **d'outils** pour **concevoir**, **analyser** et **monitorer** les processus d'affaires en vue de les **améliorer**
 - ex. de principe : amélioration continue
 - ex. de méthode : phasage
 - ex. d'outil : notations

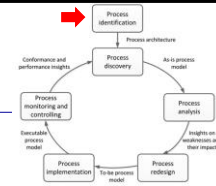
- cycle BPM :



Thierry Van den Berghe

378

Process identification



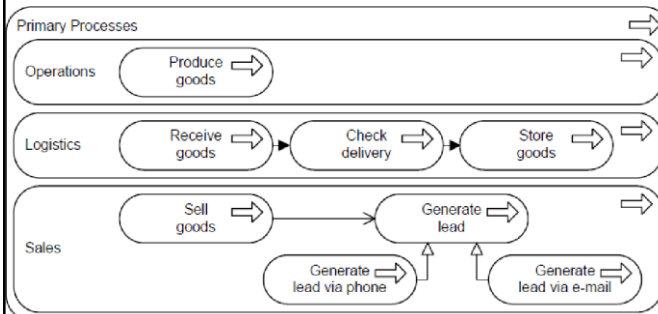
1. **identification des processus en place et de leurs relations**
 - typiquement 10 à 20 processus, mais parfois bien plus dans les très grands comptes (~100's)
 - livrable : architecture des processus
2. **établissement de priorités dans les processus à améliorer**
 - délimitation des processus à optimiser et à l'origine d'un cycle BPM
 - la rareté des ressources empêche une optimisation de tous les processus

Thierry Van den Berghe

379

Architecture des processus

- modèle conceptuel qui montre les processus et leur interrelations
- exemples de relations
 - la sortie d'un processus P1 **est l'entrée** d'un processus P2
 - un processus P1 **est une partie** d'un processus P2
 - un processus P1 **est une spécialisation** d'un processus P2



Thierry Van den Berghe

380

Critères de priorité

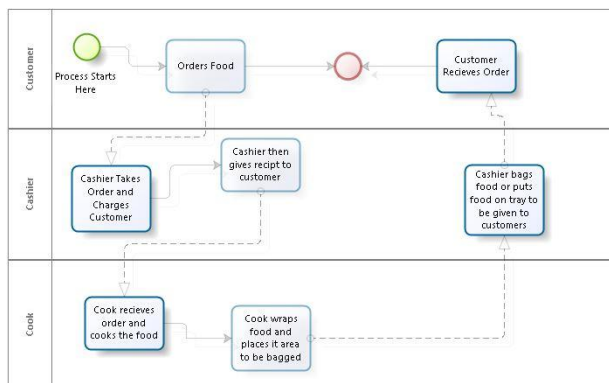
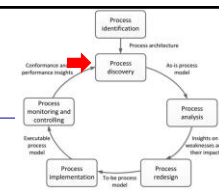
- **importance** :
 - quels sont les processus qui ont contribuent le plus à la réalisation des objectifs ?
 - quels sont ceux qui apportent le plus de valeur?
- **disfonctionnement**
 - quels sont les processus qui posent le plus de problèmes ?
- **faisabilité**
 - quels sont les processus qui sont les plus à même d'opérationnaliser les décisions de gestion en tenant compte des contraintes et des difficultés, notamment de l'adhésion des acteurs ?

Thierry Van den Berghe

381

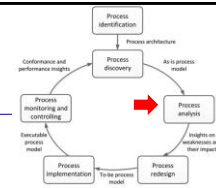
Process discovery

- collecte d'informations à propos d'un processus existant et modélisation de ce processus
 - cf. élicitation et spécification des exigences
- **délivrable** : modèles de processus "as is"
- reflète la compréhension qu'ont les acteurs des processus en place



382

Process analysis

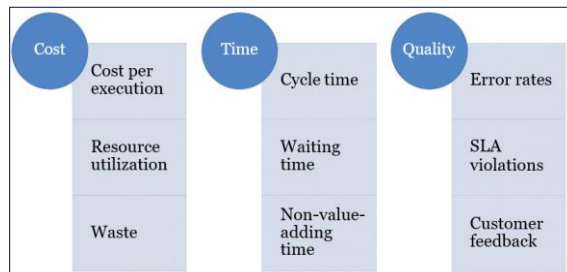


- détection et analyse des faiblesses des processus en place
- analyse quantitative : calcul d'indicateurs de performance du processus (grandeurs mesurables)
 - typiquement liés au temps, au coûts et à la qualité
 - ex. de techniques : analyse de flux, simulation, etc.
- analyse qualitative : réflexion sur le fonctionnement du processus, sur les causes des dysfonctionnements et sur les opportunités d'améliorations
 - ex. de techniques : Lean Management, Root Cause Analysis, diagramme de causes à effets ou d'Ishikawa
- largement abordé par les sciences de gestion

383

Exemple : préparation des hamburgers (1)

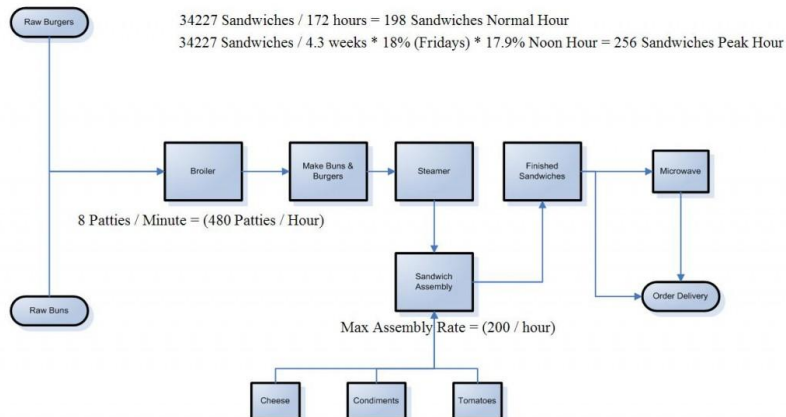
- problème : le processus de préparation des hamburgers ne répond pas à la demande aux heures de pointe
- analyse quantitative :
 - indicateurs : nombre de hamburgers produits à l'heure, temps d'attente moyen des clients, nombre de plaintes concernant le temps d'attente



384

Exemple : préparation des hamburgers (2)

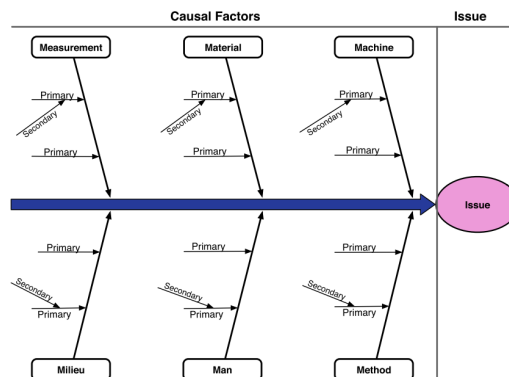
- nombre maximum de hamburgers/heure : 256
- demande maximum de hamburgers/heure : 300
- temp d'attente moyen en heure de pointe : 5,43 min
- nombre moyen mensuel de plaintes liées au temps d'attentes : 24



385

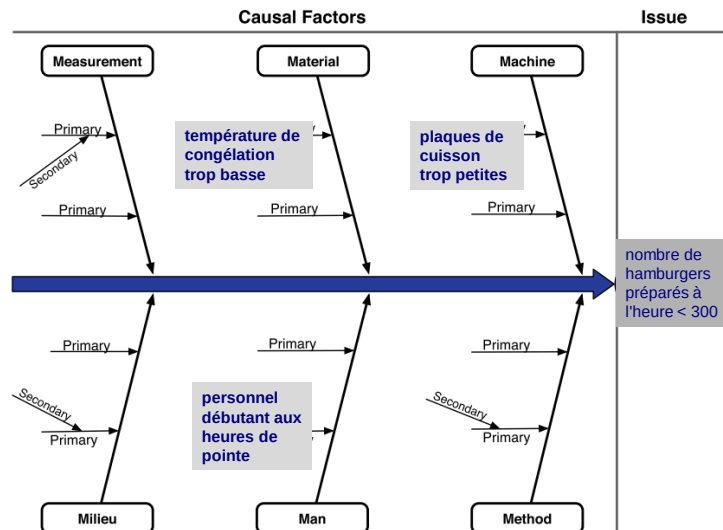
Exemple : préparation des hamburgers (3)

- diagramme de causes à effets
 - décrit les causes d'un effet négatif sur le processus
 - différents facteurs causaux :
 - **technologie (Machine)** : manque de fonctionnalité, mauvaise interface utilisateur, etc.
 - **processus (Method)** : responsabilités mal établies, mauvaise communication, etc.
 - **fournitures (Material)** : mauvaise qualité des matières premières, etc.
 - **personnel (Man)** : manque de qualifications, démotivation, etc.
 - **mesures (Measurement)** : sous-estimation de la demande, etc.
 - **environnement (Milieu)** : mesures de confinement, etc.



386

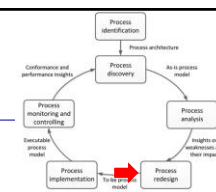
Exemple : préparation des hamburgers (4)



387

Process redesign

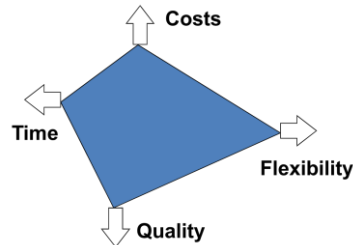
- identification et analyse des modifications et améliorations potentielles pour optimiser les processus
 - éventuellement : choix de solutions alternatives
- proposition d'une version améliorée des processus
 - livrable : modèles de processus "to be"
- plusieurs approches possibles
 - **révision** des processus en place par l'application de recommandations
 - chaque recommandation est un compromis
 - **(re-)construction** du processus en partant du livrable à produire avec le processus
 - les processus en place sont ignorés



388

Révision sur base de recommandations

- directions possibles pour l'amélioration :
 - réduire les coûts et le temps d'exécution
 - augmenter la qualité et la flexibilité (i.e., l'aptitude à la variation)
 - compromis à trouver entre ces directions
- les recommandations portent sur différents facteurs d'amélioration, comme les activités composantes ou les ressources affectées au processus



389

Facteurs d'amélioration et exemples de recommandations (1)

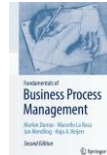
- acteurs internes ou externe du processus
 - ex. : localiser le contrôle au niveau du client (↓qualité, ↑coût)
 - ex. : le client d'une banque enregistre lui-même un prêt
- décomposition du processus en activités
 - ex. : éliminer les activités inutiles ou peu porteuses de valeur (↓qualité, ↑coût, ↑temps)
 - ex. : réduire les tests intermédiaires de qualité
- séquençage en activités
 - ex. : exécuter des activités en parallèle (↓coût, ↑temps)
- gestion des ressources affectées aux activités
 - ex. : attribuer les activités de manière flexible (↓coût, ↑temps, ↑flexibilité)
 - ex. : assurer la polyvalence des collaborateurs pour plusieurs activités différentes

↓ : défavorable ↑ : favorable

390

Facteurs d'amélioration et exemples de recommandations (2)

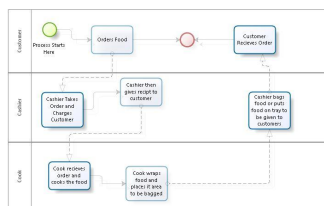
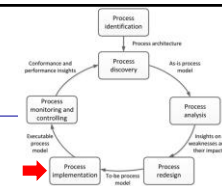
- **information créée ou consommée par le processus**
 - ex. : implémenter des contrôles d'intégrité de l'information (↑qualité, ↓temps, ↓coût)
 - ex. : faire contrôler un contrat de prêt par le directeur de l'agence bancaire
- **technologies mises en œuvre pour réaliser le processus**
 - ex. : automatiser les activités (↑qualité, ↓↑coût, ↑temps, ↓flexibilité)
 - ex. : le client encode lui-même sa commande de repas sur des bornes interactives
- **environnement**
 - ex. : déléguer des activités à des tiers de confiance (↑qualité, ↑coût, ↓flexibilité)
- **autres exemples de recommandations et étude de cas dans :**



391

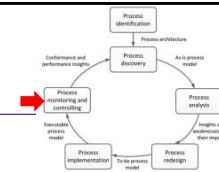
Process implementation

- mise en œuvre des modèles des processus "to be"
- souvent : automatisation accrue
 - → développement IT
- ! gestion du changement
 - expliquer, motiver, impliquer, former



392

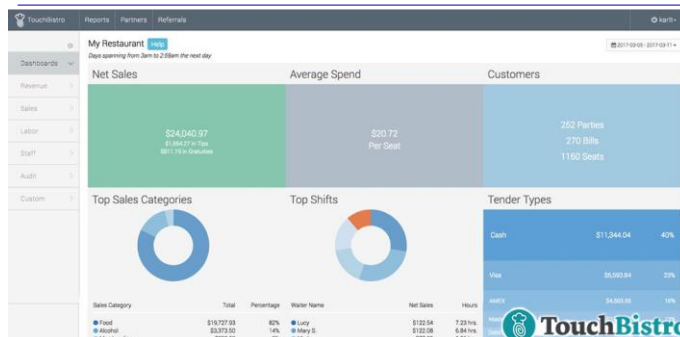
Process monitoring and controlling



- **monitoring** : cf. sciences de gestion
 - Business Activity Monitoring (BAM) : collecte d'informations sur le (bon) fonctionnement des processus, BI, etc.
 - ex. de KPI : temps moyen de préparation, nombre de commandes / heure, satisfaction client, dépense moyenne d'un client, nombre de hamburgers jetés, etc.
 - cf. analyse quantitative
- **controlling** :
 - action correctrices si les attentes ne sont pas rencontrées
 - ex. : paramétrage
 - initiation d'un nouveau cycle BPM si nécessaire

393

Exemples de tableaux de bord





21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN**
- 8 Tests
- 9 IA pour l'ingénierie des exigences

Pourquoi modéliser les processus d'affaires ?

- besoin d'abstraire les processus de la complexité organisationnelle pour les analyser, les améliorer et les implémenter
 - prise de hauteur par rapport au day to day business
 - prise de conscience des processus en place
- requis pour (rappel) :
 - comprendre comment l'organisation fonctionne (**as-is**)
 - définir comment l'organisation souhaite fonctionner dans le futur (**to-be**)
 - **communiquer** sur la manière actuelle/future de fonctionner
 - **guider** le développement d'un support automatisé

BPMN- Business Process Modelling Notation

- notations graphiques pour modéliser un processus d'affaires
 - intention de standardisation
 - développé en 2002 par le Business Process Management Initiative
 - normalisé en 2008 par l'OMG
- 3 sous-modèles :
 - process diagrams : modélisation des processus internes et publics (s'interfaçant avec des activités de tiers)
 - choreography diagrams : modélisation de haut niveau des interactions en plusieurs parties, avec un accent mis sur les échanges de messages
 - collaboration diagrams : représentation des interactions entre plusieurs processus synchronisés
 - conversation : description informelle de haut niveau d'un diagramme de collaboration, représente des ensembles de messages échangés entre des participants

BPMN versus diagramme d'activité UML (DA)

- BPMN orienté description du domaine
- DA orienté comportement visible du système
- DA fait partie d'un ensemble de langages de modélisation - UML
- DA fortement inspiré de BPMN
- mais en pratique : assez interchangeables

Trois niveaux d'abstraction

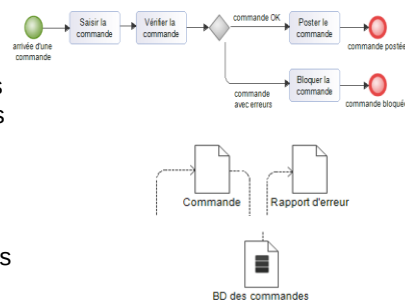
- **niveau descriptif**
 - ensemble basique de symboles
 - focus : cerner les processus d'affaires du terrain avec l'utilisateur
- **niveau analytique**
 - palette complète de symboles, comprenant de nombreux événements et la gestion des exceptions
 - focus : consistance, réalisation, robustesse
 - raffinement du niveau descriptif pour arriver à un niveau détail suffisant pour l'IT
- **niveau exécutable**
 - version du modèle en XML
 - exécutable
 - avancé

Thierry Van den Berghe

399

Trois perspectives combinées (1)

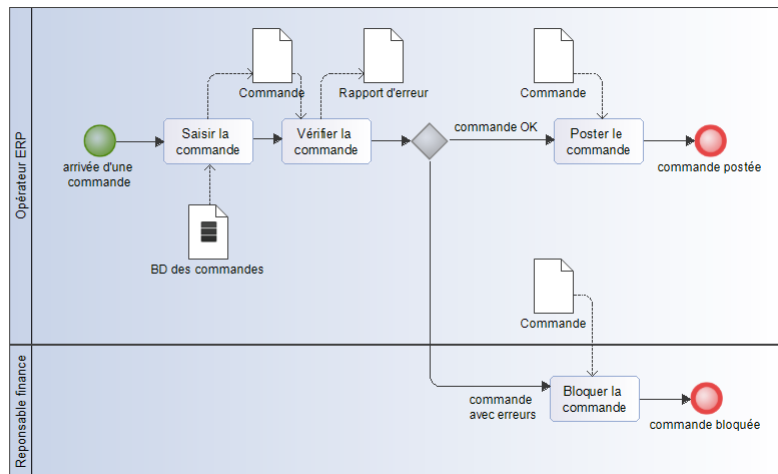
- **perspectives des flux de contrôle**
 - “what needs to be done and when”
 - relation de prédécesseurs/successeurs pour enchaîner activités et événements
- **perspectives des données**
 - “what do we need to work on”
 - relations d'entrée/sortie de données des activités
- **perspectives des ressources**
 - “who's doing the work”
 - acteurs/systèmes qui réalisent les activités et qui génèrent des événements



Thierry Van den Berghe

400

Trois perspectives combinées (2)

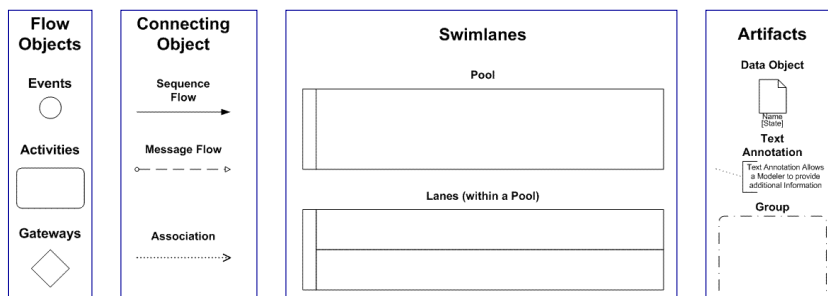


Thierry Van den Berghe

401

Éléments clés

- source principale : M. Dumas, M. La Rosa, J. Mendling Hajo, A. Reijers, Fundamentals of Business Process Management, Springer, 2013



Thierry Van den Berghe

402

Activités

- reflète le travail à accomplir, représente une unité de travail
- types d'activités :
 - tâches : activité atomique (non décomposable)
 - les tâches peuvent être typées selon leur nature
 - sous-processus : activité composée de tâches ou de sous-processus
 - mécanisme de décomposition et de gestion de la complexité
 - appel : activité qui référence à une activité externe au processus courant, et qui est appelée par le processus courant

Tâche

Sous-processus
+

Activité d'appel

Types de tâches

- send : envoie un message à un autre couloir (lane)
- receive : reçoit un message
- service : utilise un Web service ou similaire
- user task : est exécutée à l'initiative d'un acteur humain
- manual task : est exécutée sans application
- business rule : interagit avec un moteur de règles pour analyser des données
- script task : provoque l'exécution d'un script d'instructions

Send

Receive

Service

User

Manual

Business Rule

Script

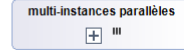
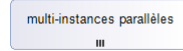
Marqueurs d'activités

- répétition de tâches ou de sous-processus

- séquentielles

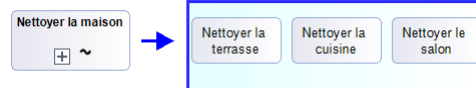
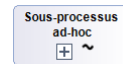


- ou parallèles



- sous-processus ad-hoc, dont les activités peuvent être exécutées ou non, dans n'importe ordre, et éventuellement plusieurs fois

- sous-processus flexible et partiellement spécifié
 - et dont l'exécution s'adapte en fonction des circonstances

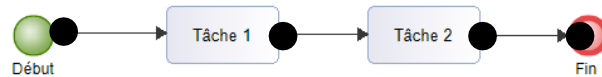


Thierry Van den Berghe

405

Séquencement

- flux de contrôle →
- logique du jeton ●



Thierry Van den Berghe

406

Flux principaux

- flux régulier : exprime une séquence d'exécution
- flux par défaut : activé si toutes les autres conditions de flux réguliers (XOR ou OR) sont fausses
- flux conditionnel



Branchements principaux

- **branchement exclusif (XOR)**
 - division : un seul flux sortant est activé
 - convergence : l'arrivée d'un seul flux entrant suffit à déclencher le flux sortant
- **branchement parallèle (AND)**
 - division : tous les flux sortants sont activés
 - convergence : l'arrivée de tous les flux entrants est requise pour déclencher le flux sortant
- **branchement inclusif (OR)**
 - division : un ou plusieurs flux sortants sont activés
 - convergence : l'arrivée de tous les flux des branches actives est requise pour déclencher le flux sortant



Exclusive Gateway

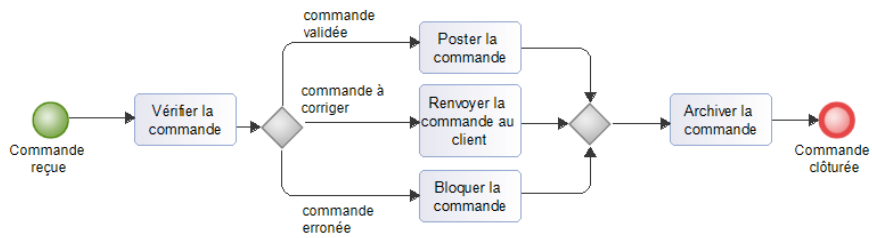


Parallel Gateway



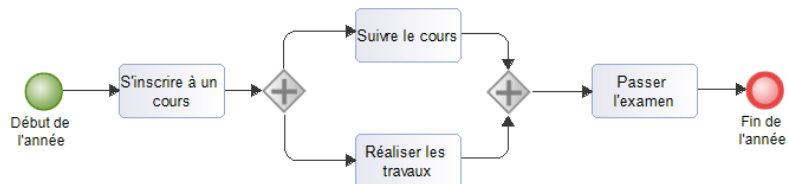
Inclusive Gateway

Exemple : XOR



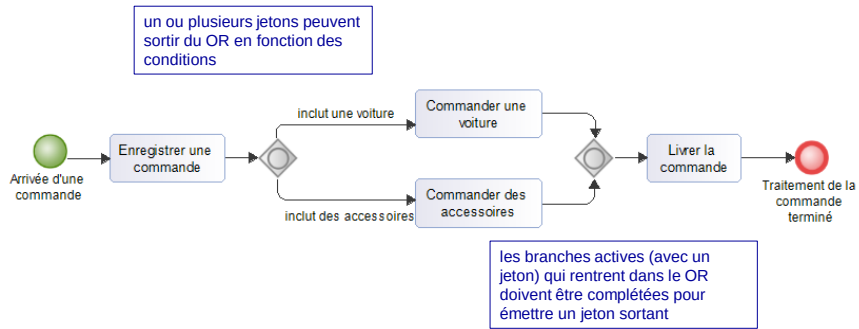
409

Exemple : exécution parallèle avec AND



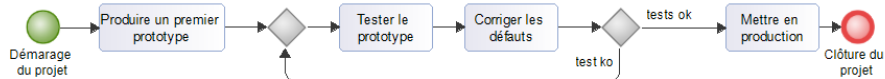
410

Exemple : exécution parallèle avec OR



411

Exemple : répétition avec XOR



412

Principaux artefacts de données

- objet de données : représente de l'information qui transite à travers le processus
- association : connecte un artefact de données à une activité
- dépôt de données : représente un endroit de stockage persistant
- données d'entrée : représente des données entrantes en provenance de l'environnement du processus
- données sortantes : représente des données résultant du processus entier



Object

Association



Store



Input



Output

Annotation

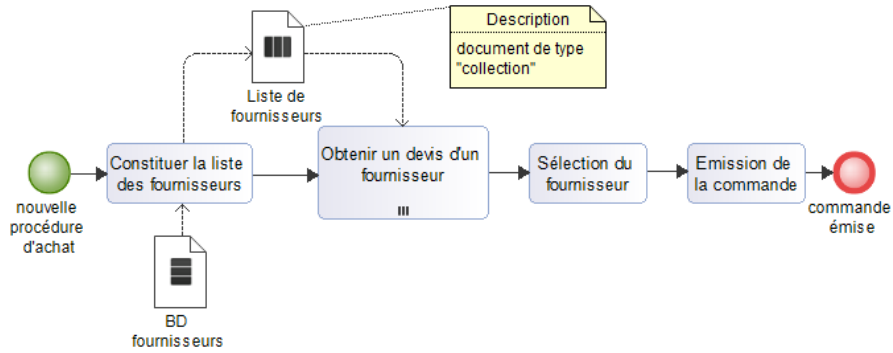
Text Annotation



Announce
Issues for
Discussion

Allow 1 week for the
discussion of the Issues
— through e-mail or
calls

Exemple

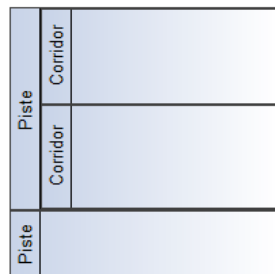


Thierry Van den Berghe

415

Pistes et corridors

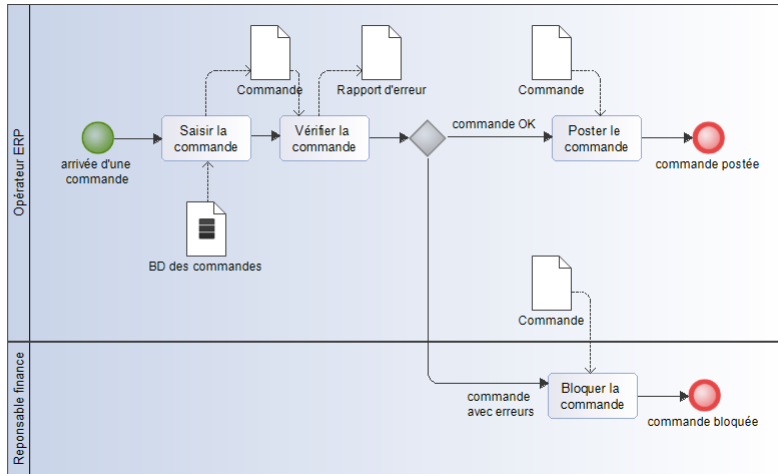
- représente la localisation des activités
 - ex. : acteurs, organisations, départements, équipements, etc.



Thierry Van den Berghe

416

Exemple diagramme de processus (rappel)

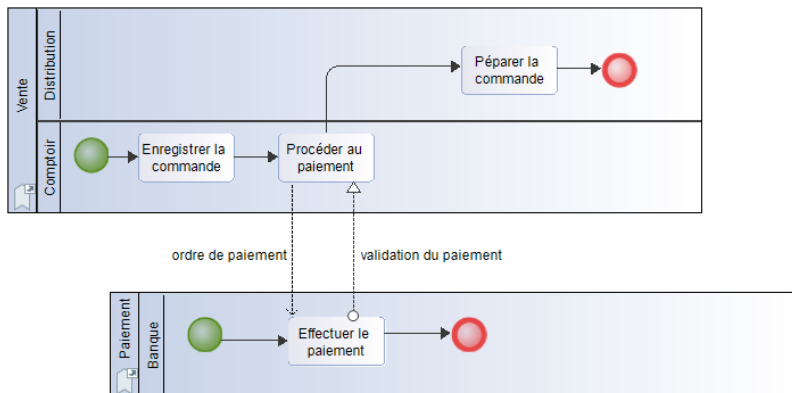


Thierry Van den Berghe

417

Diagramme de collaboration

- des flux de messages représentent des échanges d'informations entre les pistes et/ou corridors de processus différents



Thierry Van den Berghe

418

Événement (1)

- un événement est une occurrence qui survient au début, pendant ou à la fin d'un processus
 - start event : signale quand une instance du processus démarre (création de jetons)
 - end event : signale quand une branche du processus se termine (destruction des jetons de la branche correspondante)
 - intermediate event : intervient pendant le déroulement du processus, et bloque le flux de contrôle tant que l'événement ne s'est pas produit
 - terminate end event : signale quand une instance du processus se termine complètement (destruction de tous les jetons)
 - un événement impacte le déroulement du processus et déclenche des flux
- notation : cercles + annotation interne en fonction de la nature de l'événement

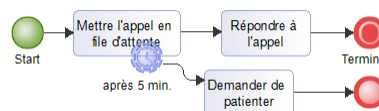
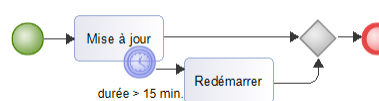


Thierry Van den Berghe

419

Événements (2)

- un événement peut être lié :
 - à un flux
 - ou à une activité (Boundary)
- si attaché à une activité:
 - interrupting : si l'événement se produit, le flux normal est interrompu et seul le flux d'exception lié à l'événement se produit
 - non interrupting : le flux normal est toujours exécuté

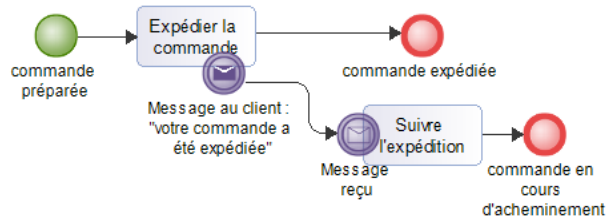


Thierry Van den Berghe

420

Evénements (3)

- un événement peut être :
 - lanceur (throwing) : génération d'un signal déclencheur
 - capteur (catching) : arrêt du processus, attente d'un signal déclencheur puis redémarrage à l'arrivée du signal



Thierry Van den Bergh

421

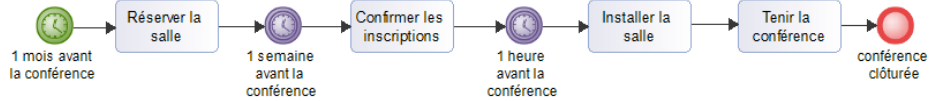
Typologie détaillée

- du niveau descriptif au niveau analytique

	Niveau supérieur	Début	Intermédiaire	Fin
Aucun: indiquent généralement un déclenchement, un changement d'état ou la fin d'un processus.				
Message: réception et envoi de messages.				
Minuterie: cycle temporel, moment déterminé ou délai écoulé.				
Escalade: mesure d'escalade à un niveau supérieur de responsabilité.				
Conditionnel: réaction à un changement de condition ou à une règle d'affaires.				
Lien: liaison de pagination. Deux événements correspondants équivalent à un flux de séquence.				
Erreur: réception ou émission d'erreurs précises.				
Annulation: réaction à l'annulation d'une transaction ou déclenchement d'une annulation.				
Compensation: gestion ou déclenchement d'une compensation.				
Signal: signalisation entre différents processus. Un signal émis peut être capté plusieurs fois.				
Multiple: réception d'un des événements spécifiés. Émission de tous les événements spécifiés.				
Multiple parallèle: réception de tous les événements se produisant en parallèle.				
Arrêt: déclenchement de la fin immédiate du processus.				

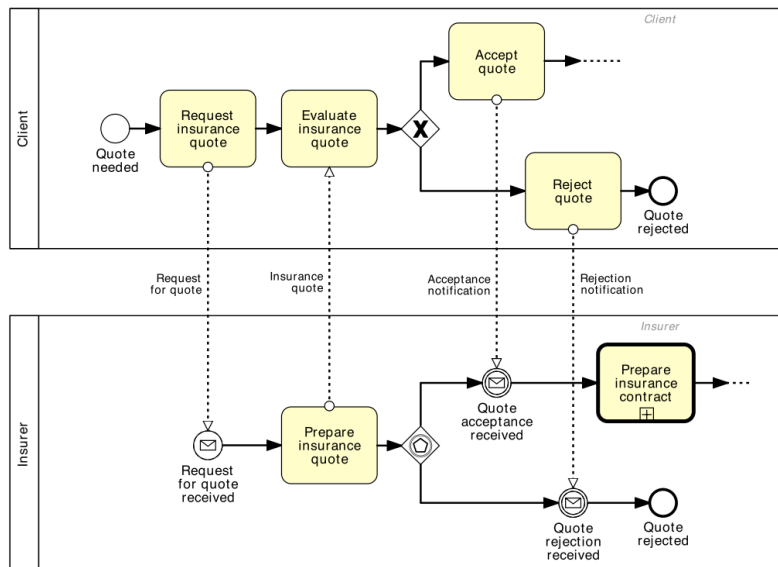
Thierry Van den Bergh

Exemple



423

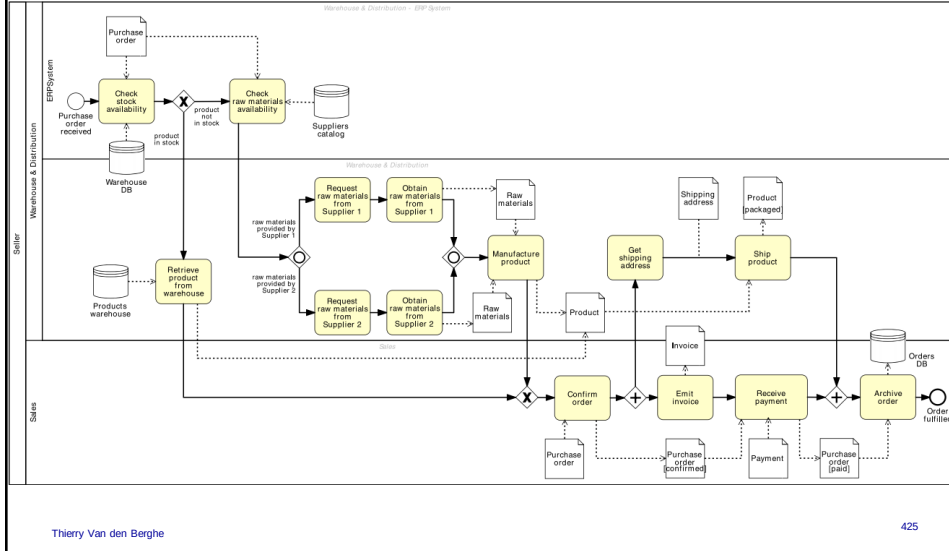
Exemple



Thierry Van den Bergh

424

Exemple



21BA030 - Ingénierie des exigences

- 1 Introduction à la business analyse
- 2 Ingénierie des exigences
- 3 Cycle de gestion des exigences
- 4 Diagramme de cas d'utilisation
- 5 Diagramme d'activité
- 6 Réingénierie des processus avec BPM
- 7 Modélisation des processus avec BPMN
- 8 Tests**
- 9 IA pour l'ingénierie des exigences

Plan



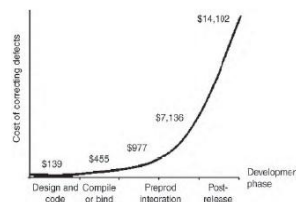
- **définitions et généralités**
- phasage des tests
- niveaux de test
- techniques de test

Thierry Van den Berghe

Avant de commencer...

- **tester un logiciel est difficile**
 - les tests exhaustifs sont quasi-impossibles
 - l'absence d'erreur est une illusion
 - les tests montrent la présence de défauts, par leur absence
- **la plupart des défauts sont présents concentrés dans un petit nombre de modules (règle des 80-20)**
 - tendance, mais pas une règle absolue
- **réaliser des tests en amont économise des ressources :**

Basili and Boehm, Software Defect Reduction Top 10 List, IEEE - Computer Society, vol. 34, (1), Jan. 2001, pp. 135-137.



428

Erreur → défaut → défaillance → effet : définitions

- **erreur : action humaine provoquant l'introduction d'un défaut**
 - ex. : mauvaise compréhension des besoins, mauvais choix technique
- **défaut (bug) : imperfection présente dans un livrable**
 - ex. : exigences erronées
 - pisté par des tests "boite blanche", i.e., examen du livrable lui-même
- **défaillance : déviation par rapport à un comportement requis et attendu**
 - résulte de l'exécution d'un logiciel comportant un défaut
 - ex. : résultat de calcul erroné
 - pisté par des tests "boite noire", i.e., examen du comportement du système lors de son exécution
- **effet : conséquence d'une défaillance**
 - ex. : plaintes d'utilisateurs

429

Test : définition

- **"Le test est l'exécution ou l'évaluation d'un système ou d'un composant du système, par des moyens automatiques ou manuels pour :**
 - vérifier qu'il répond à ses spécifications, ou
 - identifier les différences entre les résultats attendus et les résultats obtenus" (IEEE STD 729 - Standard glossary of software engineering)"
- **! le travail de test ne se réduit pas à exécuter des tests**
 - ensemble d'activités, qui vont de la planification au reporting
- **! test <> débogage**
 - test : mise en évidence des défauts
 - débogage : analyse et correction des défauts (dont l'élimination est confirmée par de nouveaux tests)

430

Tester est un facteur clé de succès

- les tests contribuent au succès de la solution
 - réduction des risques
 - réduction des coûts liés aux effets des défaillances
 - augmentation de la satisfaction / adhésion / confiance des utilisateurs
- les tests déterminent la réception ou le refus des livrables
 - aspect contractuel
- les tests certifient la conformité du système par rapport à une norme ou un règlement
 - aspect légal / réglementaire

431

Psychologie des tests

- identifier des défauts d'un livrable peut être perçu comme une critique par les auteurs du livrable
 - difficulté d'accepter ou de reconnaître ses erreurs
 - le testeur peut être mal considéré par les auteurs (développeurs, etc.)
 - → communication constructive
- bonnes pratiques :
 - insister sur la **collaboration**, en opposition à la confrontation
 - rappeler l'**objectif commun** de toutes les parties : augmenter la qualité
 - énoncer les **bénéfices** et enjeux des tests
 - communiquer sur le **livrable**, pas sur l'auteur
 - **dépersonnaliser** les défauts, i.e., présenter les défauts comme générés par le processus de développement dans son ensemble, et pas par un acteur particulier
 - rappel : les ennemis ne sont jamais utiles

432

Acteurs des tests

- **testeur (BABOK) :**
 - "Testers are responsible for determining **how to verify that the solution meets the requirements** defined by the business analyst, as well as **conducting the verification process**. Testers also seek to ensure that the solution meets applicable quality standards, and that the risk of defects or failures is understood and minimized. An alternate role is **quality assurance analyst**."
 - "The business analyst works with quality assurance (testers) to ensure that technical solutions will meet the business needs as defined by the requirements and other business analysis deliverables."
- **acteurs du projet de développement :**
 - en pratique, presque tous les acteurs participent aux tests, et plus largement à l'assurance qualité
 - développeur, business analyst, utilisateur

433

Plan

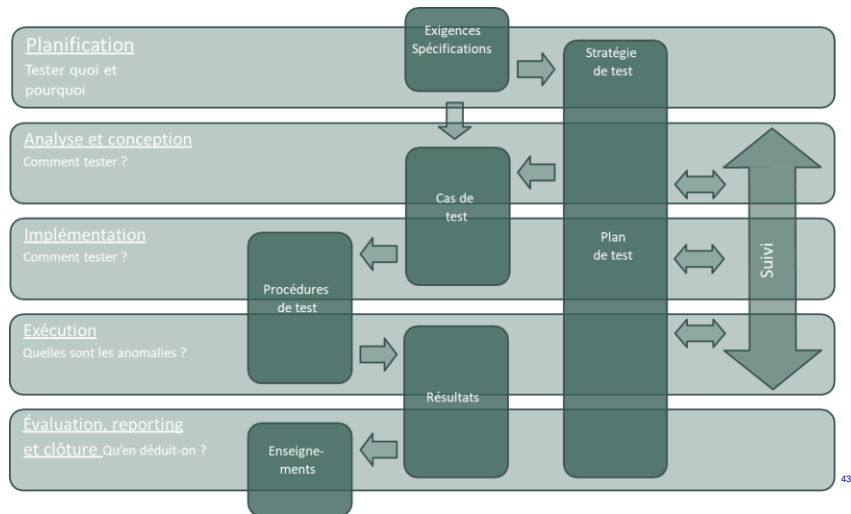
- Ingénierie logicielle
- Ingénierie des exigences (IE)
- Domaine et élicitation
- Analyse des exigences
- Spécification des exigences
- Qualité des exigences
- Gestion des exigences
- Buts et exigences
- Cas d'utilisation
- Diagramme d'activité
- Introduction à BPMN
- Tests



- définitions et généralités
- phasage **des tests**
- niveaux de test
- techniques de test

Tester = ensemble d'activités

- démarche de test = sous projet du projet de développement



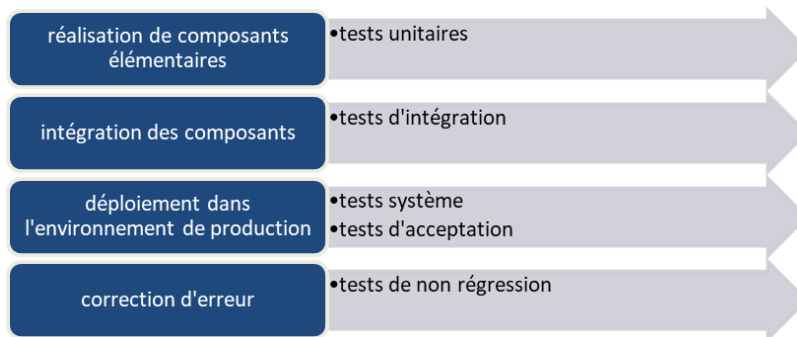
Plan



- définitions et généralités
- projet de test
- types de test**
- techniques de test

Types de tests

- le processus de test connaît plusieurs déclinaisons de tests, correspondant chacun à un niveau de développement du logiciel



437

Tests unitaires

- objectifs :**
 - détecter les défauts et défaillances d'un composant sans considérer les autres composants
 - vérifier si le comportement du composant correspond aux spécifications
- base de test :**
 - composant isolable et testable
 - ex. : modèle de données, classe d'objets, module
- responsabilités :**
 - généralement, le développeur

438

Tests d'intégration

- **objectifs :**
 - détecter les défauts et défaillances dans les interfaces entre composants ou entre systèmes
 - vérifier si le comportement des interfaces correspond aux spécifications
- **base de test :**
 - interactions et interfaces entre des composants intégrés
 - interactions et interfaces entre systèmes, progiciels et micro-services
- **responsabilités :**
 - intégration de composants : généralement, le développeur
 - intégration de systèmes : généralement, le testeur

439

Tests système

- **objectifs :**
 - détecter les défaillances dans le logiciel dans son ensemble
 - s'assurer qu'il correspond aux exigences
- **base de test :**
 - logiciel entièrement intégré, installé et configuré
- **responsabilités :**
 - généralement, testeur indépendant
 - business analyst

440

Tests d'acceptation (1)

- **objectifs :**
 - valider que le système est complet et qu'il fonctionne comme attendu, pour marquer l'acceptation du système pour sa mise en production
 - acquérir la confiance dans la qualité et la capacité opérationnelle du système
 - remarque : la recherche de défaut n'est pas un objectif ici, même s'ils peuvent apparaître
- **base de test :**
 - système tel que dans l'environnement de production
- **responsabilités :**
 - conception et implémentation par le client utilisateur ou son délégué
 - exécution par le client utilisateur, avec l'aide éventuelle de testeurs indépendants

441

Tests d'acceptation (2)

- **test d'acceptation utilisateur (UAT - User Acceptance Testing)**
 - valide l'aptitude du système à l'usage par les utilisateurs prévus dans un environnement opérationnel réel ou simulé
 - renforce la confiance dans les aspects fonctionnels
- **test d'acceptation opérationnelle (OAT - Operational Acceptance Testing)**
 - valide l'aptitude du système à être maintenu et administré par les opérateurs et administrateurs prévus
 - renforce la confiance dans les aspects opérationnels, comme la sauvegarde et restauration, la gestion des comptes utilisateurs, la migration de données, etc.

442

Tests d'acceptation (3)

- **test d'acceptation contractuelle et réglementaire**
 - valide la conformité à des critères d'acceptation d'un contrat ou d'un règlement pour la production d'un logiciel spécifique
 - effectué par les utilisateurs ou des testeurs indépendants
 - les résultats peuvent être observés ou validés par des organismes de réglementation
 - ex. : validation d'une application de tracing des personnes par une autorité en charge des questions de protection des données personnelles

443

Tests d'acceptation (4)

- **test alpha et bêta**
 - évalue un logiciel commercial COTS avant sa mise sur le marché
 - COTS (*Commercial Off-The-Shelf*) signifie : vendu sur l'étagère
 - effectué par les utilisateurs clients potentiels
 - alpha : réalisé sur le site des développeurs
 - bêta : réalisé sur le site des clients

444

Tests de non régression

- **objectifs :**
 - s'assurer que les corrections et les évolutions du logiciel n'ont pas introduit de nouveaux défauts
- **base de test :**
 - logiciel dans sa version courante
- **responsabilités :**
 - généralement, le développeur

445

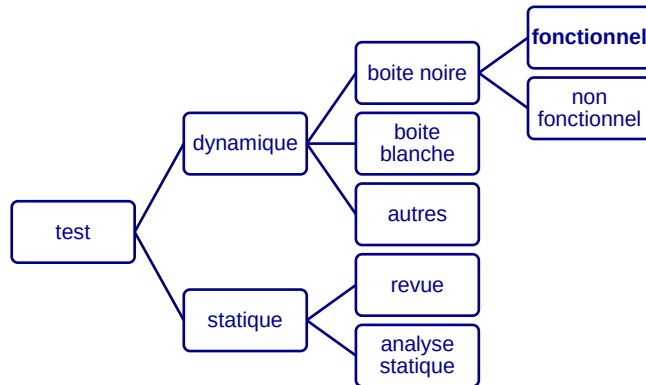
Plan

- Ingénierie logicielle
- Ingénierie des exigences (IE)
- Domaine et élicitation
- Analyse des exigences
- Spécification des exigences
- Qualité des exigences
- Gestion des exigences
- Buts et exigences
- Cas d'utilisation
- Diagramme d'activité
- Introduction à BPMN
- Tests



- définitions et généralités
- projet de test
- types de tests
- **techniques de test**

Techniques de tests



447

Choix des techniques de test

- comme souvent, le choix est lié à plusieurs facteurs
 - complexité de la base de test, objectif du test, outils disponibles, modèle de cycle de développement, expérience antérieure, etc.
- souvent encore, combinaison de techniques de test
- le business analyst est prioritairement impliqué dans des tests fonctionnels
 - vérification de la qualité des exigences
 - vérification de l'implémentation des exigences

448

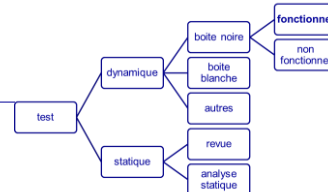
Dynamique versus statique

- **test dynamique : examine le logiciel lors de son exécution**
 - pré suppose que le logiciel soit développé et exécutable
 - focus sur le comportement du logiciel
- **test statique : examine le logiciel sans qu'il soit exécuté, voire ni développé**
 - évaluation des exigences, du code, etc.
 - détection directe des défauts, sans devoir identifier les défaillances causées par ces défauts
 - un défaut peut n'être révélé que très tard par une défaillance
 - réalisation possible dans les phases hautes du cycle de développement
 - → réduction des coûts
 - détection de défauts difficiles à cerner avec les tests dynamiques, comme des défauts de cohérence et de qualité interne
 - ex. : variables déclarées mais inutilisées, défaut de sécurité, etc.

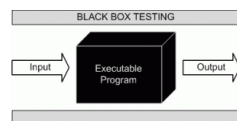
449

Boite noire

- vérifie la satisfaction des exigences, sans considérer la structure interne du logiciel
- met en évidence des défaillances
- réalisé par les utilisateurs et testeurs

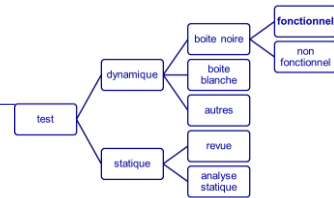


- **test fonctionnel**
 - vérifie la capacité et la couverture fonctionnelles du logiciel
 - nécessite souvent la connaissance du domaine d'application → implication du business analyst
- **test non fonctionnel**
 - vérifie les caractéristiques non fonctionnelles du logiciel



450

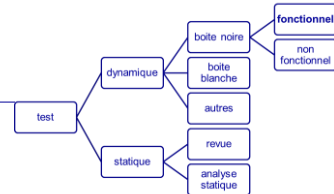
Boite blanche



- examine le comportement du logiciel par rapport à sa structure
 - structure = code, architecture, flux, etc.
 - met en évidence des défauts
 - réalisé par le développeur

451

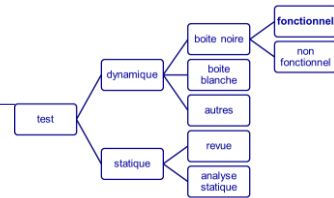
Autres tests dynamiques (1)



- test basés sur l'expérience
 - basé sur les compétences, l'intuition et les expériences comparables du testeur
 - mise en évidence de défaillances
 - ex. : estimation d'erreurs
 - le testeur établit une liste d'erreurs potentielles sur lesquelles les tests seront ciblées
 - réalisé par les utilisateurs et testeurs

452

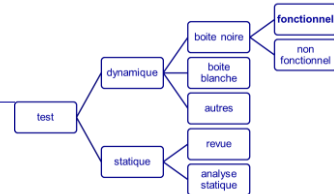
Autres tests dynamiques (2)



- analyse dynamique
 - mesure/surveillance de l'objet sous test pendant son exécution avec des outils
 - mise en évidence de défauts
 - ex. : recherche boucles infinies, de fuites mémoires, etc.
 - réalisé par des outils automatiques

453

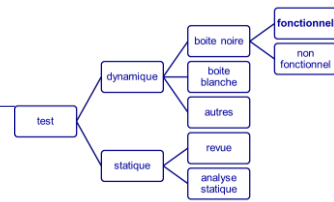
Revue



- met en évidence des défauts
- basé sur la collaboration entre acteurs humains (collègues, facilitateurs, réviseurs, etc.)
- suit un processus formel ou informel
 - ex. : revue informelle à travers des discussions entre développeurs
 - ex. : programmation en binôme, «Over the shoulder review»
 - ex. : revue formelle, avec des rôles et responsabilités bien identifiés, un planning, etc.
 - ex. : inspection, i.e., revue formelle réalisée par une équipe externe

454

Analyse statique



- mise en évidence de défauts
 - ex. : variables non initialisées, mauvaises conventions de nommage, etc.
- réalisé par des outils automatiques
- examine le code source